

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



**MÓDULO DE CONTROL DE NIVELES DE INMERSIÓN Y SOPORTE DE
INTERFACES FISIOLÓGICAS PARA MOTIONS**

Fabián Isaías Urbina Ampuero

Profesor guía: Roberto Ignacio González Ibáñez

Trabajo de titulación presentado en conformidad
a los requisitos para obtener el título de Ingeniero
de Ejecución en Computación e Informática

Santiago – Chile

2017

© **Fabián Isaías Urbina Ampuero, 2017**



• Algunos derechos reservados. Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-Compartir Igual 3.0. Sus condiciones de uso pueden ser revisadas en: <<http://creativecommons.org/licenses/by-nc-sa/3.0/cl/>>

RESUMEN

El objetivo de esta memoria es presentar las distintas fases del proyecto titulado “Modulo de control de niveles de inmersión y soporte de interfaces fisiológicas para MOTIONS”, el cual busca dotar de nuevas funcionalidades a una herramienta de investigación existente. Se detallan también aspectos de su implementación y contexto en el que se sitúa.

Actualmente existen variadas formas en que el ser humano interactúa con objetos de información digital, usualmente mediante interfaces que hacen posible el intercambio de señales entre el usuario y el sistema que ofrece la interacción. Una de estas formas ocurre en el plano de la realidad virtual. Poco se sabe de cómo se dan dichas interacciones en este plano, menos aún, cómo la inmersión las afecta. Para apoyar esta investigación existen herramientas de software y hardware, entre las cuales está MOTIONS, que debe su nombre a “*huMan-infOrmaTion InteractioN experimentation System*”. MOTIONS es un sistema que permite la interacción con objetos digitales de información y que posee la característica de soportar múltiples interfaces de forma simultánea, lo que la convierte en una herramienta versátil. Si bien, MOTIONS es flexible, no soporta cambios en el grado de inmersión que se ofrece al usuario. Es en este contexto que surge el siguiente problema ¿Cómo ofrecer en MOTIONS la posibilidad de trabajar con distintos niveles de inmersión a nivel visual, auditivo y fisiológico?

En este informe se definen las características de un módulo de software que expande las funcionalidades de MOTIONS, para soportar el trabajo con distintos grados de inmersión, a nivel visual y auditivo. Tras esa misma idea, se incluye la capacidad de relacionar señales fisiológicas, recibidas mediante una placa *BITalino*, con acciones, en una simulación. Para abordar ambas mejoras, se utiliza una metodología ágil de desarrollo de aplicaciones basada en RAD, que consiste en la construcción progresiva de prototipos, donde cada uno cumple una o más funcionalidades, llegando, finalmente, al producto completo. Para verificar el funcionamiento de la nueva versión de MOTIONS, se realizaron pruebas unitarias y de cumplimiento de requisitos, las que arrojaron problemas de usabilidad con la interfaz *BITalino*, y de configuración, sin embargo, esta situación no impide el uso de la herramienta en labores de experimentación. El nuevo sistema mantiene las características de modularidad y escalabilidad originales, por lo tanto, queda disponible para posibles nuevas iteraciones.

Palabras Clave: MOTIONS, Realidad Virtual, Inmersión, Señal Fisiológica.

AGRADECIMIENTOS

Agradecimientos a mis familiares y amigos por su apoyo constante. Agradezco especialmente a mi madre, Teresa, por estar siempre preocupada por mí, de que durmiera y comiera bien, por tenerme ropa limpia siempre, por estar disponible para lo que necesitara, pero por sobre todo, por poner todo su amor en cada cosa que hace. A Oscar, mi padre, por el esfuerzo físico y mental que realiza todos los días en el trabajo, pero que se nota que lo hace con todo su cariño y siempre pensando en entregar lo mejor a su familia. Agradezco también al Proyecto DICYT, Código061519GI, Vicerrectoría de Investigación, Desarrollo e Innovación, Universidad de Santiago de Chile, USACH, que financió de manera parcial el desarrollo de este proyecto.

Además, no puedo dejar de agradecer a mi pareja, Thiare. Su apoyo y comprensión durante todo este largo proceso fueron indispensables, sobre todo en momentos en que las cosas parecían no tener solución. Aquellos momentos donde, finalmente, esas palabras de apoyo, que solo ella sabía articular, eran todo lo que necesitaba.

Finalmente, agradezco a mi chinito, Diego. Se que me observas y apoyas desde el cielo.

TABLA DE CONTENIDO

CAPÍTULO 1.	INTRODUCCIÓN	1
1.1	ANTECEDENTES Y MOTIVACIÓN.....	1
1.2	DESCRIPCIÓN DEL PROBLEMA.....	2
1.3	SOLUCIÓN PROPUESTA	4
1.3.1	Características de la solución	4
1.3.2	Propósito de la solución.....	4
1.4	OBJETIVOS Y ALCANCES DEL PROYECTO	5
1.4.1	Objetivo general	5
1.4.2	Objetivos específicos.....	5
1.4.3	Alcances	5
1.5	METODOLOGÍAS Y HERRAMIENTAS UTILIZADAS	6
1.5.1	Metodología a utilizar	6
1.5.2	Herramientas de desarrollo.....	7
1.6	ORGANIZACIÓN DEL DOCUMENTO.....	7
CAPÍTULO 2.	MARCO TEÓRICO.....	9
2.1	MARCO CONCEPTUAL	9
2.1.1	Inmersión	9
2.1.2	Historia de la realidad virtual.....	10
2.1.3	Equipamiento Bioelectrónico	12
2.1.4	Interacción humano-información.....	16
2.2	ESTADO DEL ARTE.....	17
2.2.1	Análisis de respuestas fisiológicas en entornos inmersivos de realidad virtual 17	
2.2.2	Enganche e inmersión en ambientes inmersivos de realidad virtual	17
2.2.3	Interfaces inmersivas para enganche y aprendizaje	18
2.3	RESUMEN.....	18
CAPÍTULO 3.	ANÁLISIS.....	20

3.1	ANTECEDENTES.....	20
3.1.1	MOTIONS	20
3.1.2	Nivel de inmersión.....	21
3.1.3	Interfaces en MOTIONS	24
3.1.4	Acciones en MOTIONS	25
3.1.5	Requisitos	26
3.2	PROTOTIPADO.....	28
3.2.1	Primer prototipo: Inmersión visual básica.....	28
3.2.2	Segundo prototipo: Inmersión visual completa.....	32
3.2.3	Tercer prototipo: Inmersión auditiva completa.....	40
3.2.4	Cuarto prototipo: Integración de niveles de inmersión en MOTIONS	43
3.2.5	Quinto Prototipo: Conexión de Unity3D y BITalino	47
3.2.6	Sexto Prototipo: Integración de recepción de señales fisiológicas en MOTIONS 50	
3.2.7	Séptimo prototipo: Nueva escena y mejoras	53
3.3	RESUMEN.....	55
CAPÍTULO 4.	DISEÑO E IMPLEMENTACIÓN.....	57
4.1	ARQUITECTURA GENERAL	57
4.2	ESTRUCTURA	59
4.2.1	Clases.....	59
4.3	COMPORTAMIENTO	59
4.3.1	Secuencia	59
4.3.2	Inmersión	62
4.3.3	Interfaz BITalino	63
4.3.4	Clases Controller y Loader	64
4.4	ASPECTOS DE IMPLEMENTACIÓN	66
4.4.1	Recursos de inmersión en una escena	66
4.4.2	Inmersión y hardware	67
4.4.3	Asociación de acciones e interacciones.....	68

4.5	RESUMEN.....	69
CAPÍTULO 5. EVALUACIÓN		71
5.1	CARACTERÍSTICAS GENERALES DE LA EVALUACIÓN.....	71
5.2	PRUEBAS DE CUMPLIMIENTO DE REQUISITOS.....	71
5.3	RESUMEN.....	74
CAPÍTULO 6. CONCLUSIONES.....		79
6.1	OBJETIVOS	79
6.1.1	Objetivos específicos.....	79
6.1.2	Objetivo general	81
6.2	RESULTADOS OBTENIDOS.....	81
6.3	IMPLICACIONES Y ALCANCES.....	82
6.3.1	Implicaciones	82
6.3.2	Alcances	82
6.4	TRABAJO FUTURO.....	83
6.5	OBSERVACIONES FINALES	83
GLOSARIO		85
REFERENCIAS BIBLIOGRÁFICAS.....		86
ANEXO A.	ARQUITECTURA DE MOTIONS	88
ANEXO B.	CASO DE PRUEBA.....	89
ANEXO C.	QUALITY SETTINGS	91

ÍNDICE DE TABLAS

Tablas del Capítulo 3.

Tabla 3.1: Requisitos Funcionales.....	27
Tabla 3.2: Requisitos no Funcionales.	28
Tabla 3.3: Datos de implementación para el primer prototipo.....	29
Tabla 3.4: Datos de implementación para el segundo prototipo.	32
Tabla 3.5: Datos de implementación para el tercer prototipo.....	40
Tabla 3.6: Datos de implementación para el cuarto prototipo.	43
Tabla 3.7: Datos de implementación para el quinto prototipo	47
Tabla 3.8: Señales recibidas, identificador y pequeña descripción.	48
Tabla 3.9: Datos de implementación para el sexto prototipo.	51
Tabla 3.10: Datos de implementación para el séptimo prototipo.....	53
Tabla 3.11: Cumplimiento de requisitos funcionales por prototipo.	56
Tabla 3.12: Cumplimiento de requisitos no funcionales por prototipo.	56

Tablas del Capítulo 4.

Tabla 4.1: Descripción de clases	60
--	----

Tablas del Capítulo 5.

Tabla 5.1: Casos de prueba: Evaluación de funcionalidades originales de MOTIONS.....	74
Tabla 5.2: Casos de prueba: Evaluación de niveles de inmersión visual.	76
Tabla 5.3: Casos de prueba: Evaluación de niveles de inmersión auditiva	77
Tabla 5.4: Casos de prueba: Funcionalidades BITalino.	78

Tablas del Anexo B.

Tabla B.1: Caso de prueba CP_V_IMM_08, parte 1.....	89
Tabla B.2: Caso de prueba CP_V_IMM_08, parte 2.....	90

Tablas del Anexo C.

Tabla C.1: Configuraciones disponibles en el menú QualitySettings.	91
---	----

ÍNDICE DE ILUSTRACIONES

Figuras del Capítulo 2.

Figura 2.1: Ciclo general de un equipamiento Bioelectrónico.	13
Figura 2.2: BioRadio, equipamiento de lectura de datos fisiológicos inalámbrico.....	14
Figura 2.3: BITalino, electrodos y batería.....	15
Figura 2.4: BioNoMadix Smart Center	16

Figuras del Capítulo 3.

Figura 3.1: Vista planar en una simulación de MOTIONS.	22
Figura 3.2: Vista esférica en una simulación de MOTIONS.	22
Figura 3.3: Panel de configuración de hardware en MOTIONS.	25
Figura 3.4: Panel de mapeo de acciones en MOTIONS.	25
Figura 3.5: Nivel de inmersión visual cero.....	29
Figura 3.6: Segundo nivel de inmersión visual.	30
Figura 3.7: Tercer nivel de inmersión visual.	31
Figura 3.8: Demostración de efectos del uso de Reflection Probe.....	33
Figura 3.9: Cuarto nivel de inmersión visual.	34
Figura 3.10: Quinto nivel de inmersión visual.....	36
Figura 3.11: Demostración de uso de luz ambiental.....	37
Figura 3.12: Séptimo nivel de inmersión visual.	38
Figura 3.13: Menú de selección de niveles de inmersión y de escena.....	39
Figura 3.14: Panel que muestra al usuario el nivel de inmersión visual actual.	39
Figura 3.15: Recurso de audio en tres dimensiones, donde se delimita su alcance.....	41
Figura 3.16: Integración de inmersión auditiva a las interfaces visuales.	42
Figura 3.17: Inmersión integrada en MOTIONS, con una visualización Planar.	45
Figura 3.18: Inmersión integrada en MOTIONS, con una visualización Esférica.....	45
Figura 3.19: Menú principal de MOTIONS.	46
Figura 3.20: Menú de configuración de los niveles de inmersión en MOTIONS.....	46
Figura 3.21: Datos obtenidos a través de una placa BITalino en una escena de Unity3D.	48
Figura 3.22: Conexión de electrodos de un BITalino para toma de EMG.....	49
Figura 3.23: Datos recibidos al medir EMG en una escena de Unity3D.....	49
Figura 3.24: Lectura e impresión por pantalla de datos leídos desde BITalino.....	50
Figura 3.25: Menú de configuración de BITalino.	52
Figura 3.26: Menú de Action Mapping para BITalino.....	52
Figura 3.27: Comparación entre niveles de inmersión de la nueva escena.....	54
Figura 3.28: Nueva información en el panel de configuración de inmersión.....	55

Figuras del Capítulo 4.

Figura 4.1: Diagrama arquitectural de MOTIONS, parte 1.	58
Figura 4.2: Diagrama arquitectural de MOTIONS, parte 2.	58
Figura 4.3: Extensión al diagrama arquitectural de MOTIONS.	59
Figura 4.4: Extensión al diagrama de secuencia de módulos.	61
Figura 4.5: Diagrama de actividad de niveles de inmersión.....	63
Figura 4.6: Diagrama de actividad de interfaces fisiológicas.	64
Figura 4.7: Diagrama de secuencia de las clases Controller con acciones	65
Figura 4.8: Diagrama de secuencia de las clases Loader con acciones	66
Figura 4.9: Módulo de asociación de acciones en un esquema conceptual.	69

Figuras del Anexo A.

Figura A.1: Diagrama de Arquitectura de MOTIONS completo.....	88
---	----

CAPÍTULO 1. INTRODUCCIÓN

El objetivo de este capítulo es exponer los motivos que impulsan este proyecto, además de entregar lineamientos generales de cómo será abordado y cuáles serán sus objetivos y límites. Para eso, se presentan los antecedentes y la motivación que rodean el problema a abordar, junto con la solución elegida para resolverlo. Se listan los objetivos y alcances de la solución, y se explica en que consiste la metodología propuesta para su desarrollo.

1.1 ANTECEDENTES Y MOTIVACIÓN

Realidad virtual se define como “un ambiente real o simulado, donde quién lo percibe experimenta telepresencia”¹. Telepresencia, se refiere a la experimentación de presencia en un entorno generado por medios artificiales, entendiendo presencia como la sensación de encontrarse en un entorno generado por medios naturales (Steuer, 1992). MOTIONS es un sistema computacional inmersivo de apoyo a la experimentación con objetos de información digital (Gaete, 2016) que permite estudiar la telepresencia. Sin embargo, no lo hace a cabalidad, ya que su implementación actual no contempla que los distintos niveles de las capacidades sensoriales de los usuarios, como lo son la vista y la audición, además de una retroalimentación basada en sus respuestas fisiológicas, pueden influir en esta inmersión. Añadir una forma de poder contemplar los distintos niveles de inmersión auditivos y visuales, además de la retroalimentación a la respuesta fisiológica del participante, como herramientas de control de inmersión, es particularmente importante considerando que la vista y la audición son esenciales en nuestra cultura (Murray & Sixsmith, 1999) y que, como la presencia está definida como la sensación de estar en un ambiente, lo que las respuestas fisiológicas de una persona produzcan dentro de este ambiente, aumenta esta sensación (Steuer, 1992). Además, la evolución de los entornos de interacción en tres dimensiones es tan rápida que ha despertado el interés de muchos estudios, lo cual, hace necesario el poder contar con una herramienta que, de una forma completa, permita a los investigadores estudiar la mayor cantidad de aristas que la realidad virtual posea (Chavan, 2016).

La gente vive los entornos de interacción en tres dimensiones de maneras distintas dependiendo de su cultura. Por ejemplo, una persona que se encuentra en un entorno de realidad virtual en el que puede navegar sin gravedad y sin colisiones, intenta evitar, de todas formas, alejarse de la superficie y esquivar obstáculos dispuestos en la escena (Murray, Bowers, West, Pettifer &

¹ Traducción libre

Gibson, 2000). Los estudios dicen también, que para la cultura occidental es más importante la vista y la audición (en comparación con el resto de los sentidos), mientras que, los orientales enfatizan sus experiencias sensoriales en el olfato (Murray & Sixsmith, 1999). Es por esta evidencia cultural, que es interesante poseer un control sobre los niveles de las capacidades sensoriales de los usuarios en una herramienta dedicada a la investigación como lo es MOTIONS.

La evolución de los entornos de interacción en tres dimensiones ha sido rápida, debido a que mientras avanza la tecnología, se hacen más accesibles las interfaces que permiten interactuar dentro de ellos. A la vez, la experiencia de usuario es más completa en términos de inmersión (Chavan, 2016). Los usos que esta tecnología ofrece son, por ejemplo, en áreas de educación y entrenamiento, donde permiten a los usuarios mejorar en alguna disciplina específica sin comprometer mayores costos operacionales; áreas de retail, donde los compradores pueden, a través de una simulación, dimensionar el espacio que ocupa algún producto que deseen comprar, sin necesidad de estar físicamente en una tienda; medicina y ciencia, ofreciendo simulaciones que permiten desarrollar control de fobias y tratar condiciones de estrés post-traumático. Debido a la efectividad que los entornos de interacción simulados han demostrado en todas las áreas mencionadas, es que ha despertado mayor interés de los investigadores (Chavan, 2016) y resulta importante poder estudiar la totalidad de su telepresencia.

1.2 DESCRIPCIÓN DEL PROBLEMA

La plataforma *HuMan-infOrmaTion InteractioN experimentatioN System*, (MOTIONS), es una herramienta de experimentación. Creada originalmente bajo el nombre de VORTICES, donde permitía realizar estudios de interacción humano-información únicamente en ambientes de realidad virtual (Gaete, 2016). En la actualidad, MOTIONS se ha extendido para soportar interfaces oculares, cerebrales (Pollmann, 2017) y gestuales (Estay, 2017), en ambientes inmersivos, no necesariamente haciendo uso de realidad virtual, lo que la convierten en una herramienta de investigación versátil.

Si bien, en realidad virtual, uno de los grandes desafíos es lograr que el usuario sienta que es parte de la realidad alternativa que se le ofrece (Steuer, 1992), también pueden percibirse distintos niveles de inmersión en entornos interactivos que no necesariamente utilicen realidad virtual. Típicamente, y basados en la cultura, los ambientes de interacción explotan los aspectos visuales, auditivos y táctiles con el fin de alcanzar distintos niveles de inmersión (Murray & Sixsmith, 1999). Si bien, uno de los focos centrales de MOTIONS es la interacción inmersiva con información, la implementación actual no contempla que la inmersión puede tener varios niveles que se asocian a los recursos tecnológicos y las capacidades sensoriales del usuario. Actualmente, posee recursos para trabajar con la vista, la audición y el tacto, pero, estos son básicos y contemplan un único nivel de inmersión. Es en este contexto que surge el siguiente

problema ¿Cómo ofrecer en MOTIONS la posibilidad de trabajar con distintos niveles de inmersión a nivel visual, auditivo y fisiológico?

1.3 SOLUCIÓN PROPUESTA

1.3.1 Características de la solución

La solución consiste en un módulo de software para la herramienta MOTIONS, el cual debe cumplir los siguientes requisitos:

- a. No debe intervenir el correcto flujo de las funcionalidades actuales en el software.
- b. Debe proveer un mecanismo que permita controlar el nivel de inmersión auditiva, vale decir, aumentar o reducir la cantidad y calidad de sonidos emitidos, a través de categorías elegibles por el investigador. Debe contar con una interfaz visual que provea la capacidad de seleccionar alguna de estas categorías.
- c. Debe proporcionar un mecanismo que permita controlar la inmersión visual, en otras palabras, cambiar la cantidad y calidad de los estímulos visuales generados, a través de categorías que ofrezcan niveles de inmersión. Debe contar con una interfaz visual que facilite la elección de las categorías.
- d. Se debe integrar las componentes desarrolladas en los puntos b y c con MOTIONS. Esto implica que, ambas componentes funcionen dentro del ambiente de experimentación. Además, las interfaces visuales del control de inmersión visual y auditivo deben adaptarse al menú del experimentador actual, de modo que permita converger de forma flexible todas las funcionalidades, antiguas y nuevas, del software.
- e. Debe poseer un catálogo de ambientes (escenas) para generar el espacio de realidad virtual.
- f. Debe proveer un sistema que gatille, en base a respuestas fisiológicas del participante, las acciones que permita realizar la simulación.

1.3.2 Propósito de la solución

El propósito de la solución es apoyar las investigaciones relacionadas con realidad virtual e inmersión, a través de un módulo de software que expanda el área de estudio de una herramienta existente para tal propósito. En términos concretos se busca ofrecer un sistema que permita a los investigadores manipular los niveles de inmersión dentro de MOTIONS a través de recursos visuales, auditivos y retroalimentación fisiológica. Esto abre una gama de estudios que hasta el momento son prácticamente imposibles de realizar en este ambiente de investigación, debido a la complejidad involucrada y tiempo requerido para preparar el ambiente necesario para la experimentación dentro de la herramienta.

1.4 OBJETIVOS Y ALCANCES DEL PROYECTO

1.4.1 Objetivo general

Generar un módulo de control de niveles de inmersión para MOTIONS y agregar soporte para retroalimentación a través de interfaces que transmitan señales fisiológicas.

1.4.2 Objetivos específicos

1. Realizar un estudio del marco teórico que rodea el problema a solucionar.
2. Diseñar un módulo para el control de inmersión que sea compatible con la arquitectura de MOTIONS.
3. Desarrollar un componente de control de nivel de inmersión visual.
4. Desarrollar un componente de control de nivel de inmersión auditiva.
5. Integrar los componentes de control de niveles de inmersión.
6. Diseñar e integrar entornos de realidad virtual.
7. Realizar pruebas de cumplimiento de requisitos funcionales y no funcionales.

1.4.3 Alcances

Los alcances y limitaciones se pueden resumir en los siguientes puntos:

- a. Los ambientes (o escenas) serán generados en forma estática por el memorista, es decir, no se utilizará un método de generación procedural para cada ambiente.
- b. La cantidad de ambientes generados será una muestra acotada, con el fin de ejemplificar las funcionalidades del módulo dentro del entorno de investigación.
- c. El módulo queda restringido netamente a la última versión de MOTIONS.
- d. Este trabajo no pretende llegar a un nivel de realismo absoluto, esto considerando que la capacidad de procesamiento disponible limita la calidad y cantidad de recursos gráficos a los que se puede llegar, considerando que un entorno de realidad virtual es un proceso que generalmente requiere mucho trabajo de cómputo (debido a que cada función de renderizado debe realizarse dos veces, una por cada ojo) luego, un nivel de inmersión alto naturalmente irá en desmedro del rendimiento.
- e. Los niveles de inmersión que las tecnologías que existen actualmente no permiten alcanzar el realismo. Acercarse a este requiere más tiempo del comprendido para este proyecto.
- f. La funcionalidad de permitir gatillar acciones dentro de la simulación, en base a señales fisiológicas del usuario, solo toma en cuenta las señales que son captadas por una placa *BITalino*.

1.5 METODOLOGÍAS Y HERRAMIENTAS UTILIZADAS

1.5.1 Metodología a utilizar

La metodología a utilizar está basada en *Rapid Application Development* (RAD). En términos generales, RAD es una metodología enfocada en el desarrollo y que propone como método de trabajo la generación de prototipos desechables que cumplan algún objetivo específico (Martin, 1991). A continuación, se listan las características tomadas de la metodología de RAD y la razón que las hace aplicables a este proyecto:

1. Pone el foco en grupos reducidos de trabajo. Esta característica es aplicable considerando que el proyecto será desarrollado de manera individual, por el alumno.
2. Pone énfasis en el desarrollo y no en la documentación. Si bien, el proyecto sugiere realizar una documentación relativamente extensa, la metodología permite enfocarse en generar productos que satisfagan alguna necesidad real del proyecto en períodos de tiempo relativamente cortos, lo que facilita el desarrollo de ambas partes de forma paralela.
3. La creación de prototipos desechables permite realizar modificaciones regularmente y de manera simple. Esto es altamente deseable considerando que la retroalimentación regular del profesor guía genera cambios, algunas veces estructurales, sobre el desarrollo del proyecto.
4. Los prototipos acercan al desarrollador al producto que se desea alcanzar con el cliente. Considerando que el memorista carece de trabajo previo en creación de módulos de software para realidad virtual, el detalle de los requisitos que debe poseer el sistema no está definido completamente, luego, es conveniente reducir el riesgo de desarrollar un sistema que no esté alineado con los objetivos planteados, esto se logra en gran medida con el uso y evaluación de prototipos desechables.

Debido a los puntos antes mencionados, la elección de RAD como metodología sobre la cual basarse para desarrollar el proyecto, queda respaldada. Cabe destacar que la metodología finalmente utilizada se basa en estos aspectos de RAD, pero no utiliza la RAD como tal. La diferencia más importante guarda relación con el propósito de la metodología RAD, ya que en esta última se busca acercar al desarrollador al producto que se desea alcanzar, y para el marco de este proyecto, se busca obtener producto final, por lo tanto, cuando los prototipos comienzan a acercarse al producto deseado, estos no se desechan y se utilizan como base sobre la cual construir la herramienta final.

1.5.2 Herramientas de desarrollo

Las herramientas de Hardware a utilizar son:

- a. Oculus Rift DK2: Visor montado de realidad virtual para la proyección de contenido tridimensional.
- b. Placa BITalino: Herramienta que permite la captura y posterior procesamiento de señales fisiológicas.
- c. Guantes OpenGlove: Dispositivo desarrollado por Monsalve (2015), para la retroalimentación vibro-táctil.
- d. Computador de escritorio: Equipo compatible con Oculus Rift DK2, el cual está equipado con procesador Intel Core i7, 8 GB en RAM y una tarjeta de vídeo GTX 1070.

Las herramientas de Software a utilizar son:

- a. MOTIONS: Programa que permite interactuar con objetos de información digital en ambientes inmersivos a través del uso de interfaces. Es el software para el cual se diseñará el módulo.
- b. Unity 3D v.5.3.4: Engine de videojuegos que permite crear ambientes 3D y sobre el cual está desarrollado MOTIONS (Gaete, 2016; Estay, 2017; Pollmann, 2017).
- c. C#: Principal lenguaje utilizado para desarrollar código dentro de Unity 3D.
- d. Visual Studio 2015: Entorno de desarrollo integrado (IDE por sus siglas en inglés) para programar en C#.
- e. Word Office 365: Editor de texto utilizado para escribir la memoria.
- f. GitHub: Software que extiende las funcionalidades de Git, el que a su vez es una herramienta, facilita el control de versiones para desarrollo.
- g. GitKraken: Usado como GUI para Git.

1.6 ORGANIZACIÓN DEL DOCUMENTO

El presente documento se compone de seis capítulos. En el Capítulo 2 (marco teórico), se detallan conceptos que son claves para entender este trabajo, además, se recorre el estado del arte, mostrando las tecnologías y estudios que apuntan a resolver un problema similar al que resuelve el proyecto expuesto en esta memoria. En el Capítulo 3 (análisis), se analizan que requisitos son necesarios para llegar a la finalización del módulo de software a construir. Además, se muestran los prototipos utilizados para llegar a la implementación deseada. En el Capítulo 4 (diseño e implementación), se trata la arquitectura de los dos módulos, el módulo de control de inmersión y la integración de *BITalino*, y como se relacionan con el sistema original. En el Capítulo 5 (evaluación), se muestra cómo se evaluó el sistema, considerando la usabilidad y la calidad de la

integración. En el Capítulo 6 (conclusiones), se exponen las conclusiones del trabajo. Se revisa el grado de cumplimiento de los objetivos planteados, además de los resultados obtenidos y las limitaciones halladas el desarrollo del proyecto. Se agregan elementos que podrían suponer mejoras en un futuro.

CAPÍTULO 2. MARCO TEÓRICO

En este capítulo se mencionan conceptos teóricos claves para entender el proyecto, además, se exponen distintas tecnologías existentes que pueden servir de herramientas para el desarrollo de las funcionalidades requeridas. Por otra parte, se realiza una revisión del estado del arte, donde se tratan diversos estudios que se relacionan con la problemática tratada en este proyecto.

2.1 MARCO CONCEPTUAL

2.1.1 Inmersión

La inmersión se define como la “acción de introducir o introducirse plenamente en un ambiente determinado” según (ASALE, s. f.). Para Dede (2009), la inmersión se define como “la sensación subjetiva de participación en una experiencia realista y que ofrece enganche”², lo que puede darse en ambientes digitales de interacción. Según Almond (2011), un ambiente digital e inmersivo se refiere a “un ambiente fuera del mundo físico, el cual habitamos y experimentamos día a día”. Todas estas definiciones comparten dos elementos estrechamente relacionados: la presencia ambiente y el experimentar sensaciones. Los recursos del ambiente son los que provocan sensaciones en el participante. El estudio realizado por Dede (2009), demuestra que estos recursos pueden generar distintos grados de inmersión a través del uso de factores sensoriales, de acción y simbólicos. Un factor sensorial permite al participante de una simulación experimentar localidad dentro de un espacio tridimensional generado digitalmente, haciendo uso, por ejemplo, de interfaces *Head Mounted Display* (HMD) como la que aparece en la Figura 2.1, tecnologías “haptic” o sonido estéreo tridimensional. Los factores de acción facultan al participante para realizar acciones que no pueden realizar en el mundo real y que desencadenan efectos impredecibles, incrementando con esto el grado de concentración y compromiso con el ambiente. Los factores simbólicos implican gatillar asociaciones semánticas y psicológicas a través del contenido de la simulación, como, por ejemplo, sentir miedo en un ambiente que posiciona al participante en una superficie a una altura elevada.

² Traducción libre del autor de la palabra en inglés “*engagement*”.



Figura 2.1: *Head Mounted Display*, diseñado por *Samsung*.

Obtenido de *Samsung Gear VR*³.

2.1.2 Historia de la realidad virtual

La historia de la realidad virtual es amplia. Según la Virtual Reality Society, en un reporte del año 2016 (Virtual Reality Society, 2016), las primeras formas de inmersión aparecieron en las pinturas panorámicas, donde la intención era hacer sentir al observador que era parte de la escena. A partir de ese momento, la realidad virtual ha crecido junto con la tecnología. Los avances más importantes se concentran desde finales del siglo XX en adelante, pudiendo mencionar escasos hitos anteriores como el primer *headset* de realidad virtual (1960), máquinas de arcade (1991) y lentes de realidad virtual (1993), entre otros. En la actualidad, existe una gran variedad de interfaces, generalmente de interacción humano-computador, que evolucionan cada día y que permiten trabajar con realidad virtual de forma completa. Es en este contexto en que los desarrolladores de software crean programas que ofrecen un ambiente inmersivo a través de la utilización de una o más interfaces humano-computador.

Hoy en día existe gran variedad de herramientas que utilizan realidad virtual. Estas herramientas abundan en el mercado, y permiten interactuar con objetos de información digital de forma inmersiva, cada una con sus propias limitaciones (Chavan, 2016). Se pueden identificar 4 áreas

³ <http://www.samsung.com/cl/promotions/galaxynote4/feature/gearvr/>

de interés común donde se utilizan dispositivos de realidad virtual que ofrecen algún control sobre el nivel de inmersión, las cuales se listan a continuación:

1. **Entretención:** tienen como objetivo principal la diversión del usuario, como por ejemplo *Sparc*⁴, *Elite Dangerous*⁵ y *Resident Evil 7: Biohazard*⁶, que son videojuegos. La realidad virtual es provista, generalmente, a través de un HMD, como el *Oculus Rift* o *Samsung Gear VR*. Ofrecen un control complejo en los niveles de inmersión audiovisual, con el fin de adaptar el juego a las capacidades del dispositivo en el cual está siendo ejecutado. *Youtube*⁷, por otra parte, es una plataforma de videos que ofrece la opción de subir o visualizar videos en 360º compatible con lentes de realidad virtual, donde la inmersión se ofrece a través del ajuste de la calidad del video, lo cual compromete imagen y audio. *The Void*⁸ es un sistema de entretenimiento, que ofrece experiencia de inmersión completa, lo que se logra situando al usuario en una instalación física real, que combina sistemas de retroalimentación físicos, como cambios en la temperatura y flujos de aire, con un ambiente digital de realidad virtual entregado al participante a través de una interfaz HMD, y donde los estímulos, recibidos en el plano de la realidad, se condicen con los presentados en el plano virtual, lo que aumenta la inmersión del participante («THE VOID - Step Beyond Reality», s. f.).

En general, en entretenimiento, no se soporta un gran número de interfaces y los softwares suelen no ser de código abierto, por lo tanto, se hace muy complejo el agregar nuevas interfaces y funcionalidades que apoyen la experimentación.

2. **Navegación:** se enfocan en navegación de sistemas computacionales a través de un headset de realidad virtual. *Fulldiver VR*⁹, por ejemplo, es una aplicación de teléfono

⁴ <https://www.ccpgames.com/news/2017/ccp-games-announces-sparc-a-full-body-virtual-sport-only-possible-in-virtual-reality/>

⁵ <https://www.elitedangerous.com/>

⁶ http://residentevil7.com/es/index.html#_about

⁷ <https://play.google.com/store/apps/details?id=com.google.android.youtube>

⁸ <https://www.thevoid.com/>

⁹ <https://play.google.com/store/apps/details?id=in.fulldiver.shell>

celular que ofrece el servicio de navegación web en un entorno virtual. El objetivo único es ser una alternativa a las interfaces usuales de interacción humano-computador, como lo son la pantalla *touch* y el control por voz.

3. **Turismo:** permiten al usuario navegar dentro de un entorno generado artificialmente y basado en alguna localización geográfica existente en el mundo real. Utilizan lentes de realidad virtual para proveer la inmersión del usuario. Un ejemplo de esto es *Google Earth VR*¹⁰, que permite al usuario explorar lugares del planeta en un entorno generado con medios artificiales. Además del headset de realidad virtual, utiliza un dispositivo llamado Touch, que permite manejar, a través de las manos, la navegación por la escena.
4. **Investigación y medicina:** su uso es para crear conocimiento y ayudar a tratar problemas médicos respectivamente. Sistemas como los mencionados en los puntos 2.2.1, 2.2.2 y 2.2.3, ayudan a la investigación ofreciendo inmersión en ambientes de realidad virtual de propósito único. MOTIONS es otro ejemplo de un ambiente de apoyo a la investigación, el cual ofrece la capacidad de interactuar con el entorno de realidad virtual a través de múltiples interfaces como lo son el *Oculus Rift* y *Leap Motion* (Gaete, 2016), *Open Glove* (Monsalve, 2015; Meneses, 2016), interfaces gestuales (Estay, 2017) y oculares y cerebrales (Pollmann, 2017). Gracias a esa cantidad de interfaces, es un ambiente flexible, y si bien, su implementación actual ofrece una funcionalidad básica de ordenamiento de imágenes dentro del entorno virtual, es un software de código abierto, por lo tanto, es escalable a múltiples estudios los cuales pueden aprovecharse de esta versatilidad. MOTIONS en la actualidad otorga un control del nivel de inmersión audiovisual básico.

2.1.3 Equipamiento Bioelectrónico

El equipamiento bioelectrónico es el que permite la adquisición de datos biomédicos. De acuerdo a la Great Lakes NeuroTechnologies (2013), La mayoría de estos dispositivos utilizan un flujo que

¹⁰ <https://vr.google.com/earth/>

comienza con la captura de la señal biomédica en el transductor, el cual puede ser un electrodo, una placa de fuerza o un sensor de estiramiento, entre otros. Luego, la señal obtenida pasa al amplificador, que incrementa su amplitud, posteriormente un filtro de frecuencia toma esta señal y reduce el ruido contenido en ella, con el objetivo de obtener datos más legibles y representativos. La señal filtrada pasa enseguida a un convertidor de analógico a digital (ADC¹¹), el cual toma los datos obtenidos, hace un muestro de estos a una cierta tasa y usa esta información para generar una representación digital, que es fácilmente almacenable y copiada, y permite repetir y automatizar los análisis. Finalmente, estos datos pasan a un post-procesamiento, por ejemplo, filtro por banda de frecuencia, transformada rápida de Fourier o espectrograma, el cual ayuda a la comprensión y estudio de los datos adquiridos. Este último paso es opcional y depende de lo que el investigador desee realizar. Todos los pasos mencionados pueden apreciarse en la Figura 2.2.



Figura 2.2: Ciclo general de un equipamiento Bioelectrónico.

Obtenido de *BioRadio Webinar: BioRadio Webinar: Basics of Biomedical Data Acquisition*.¹²

Los datos biomédicos pueden ser obtenidos de señales fisiológicas (como lo son los electrocardiogramas (ECG), electromiografías (EMG) o electroencefalogramas (EEG)), de la posición y de la respiración, entre otros. Para el alcance de este proyecto, los esfuerzos se centrarán en capturar señales fisiológicas del participante. Algunos de los equipamientos tecnológicos que sirven para recibir estas señales son: *BioRadio*, *BITalino* y *BioNomadix Smart Center*.

BioRadio

El *BioRadio* es un dispositivo bioelectrónico transportable, con canales programables para grabar y transmitir combinaciones de señales fisiológicas humanas. Incluye un SDK¹³ gratuito el cual

¹¹ Traducido de “*Analog to Digital Conversion*”

¹² <https://youtu.be/mKglhrK8V0E>

¹³ Traducido de “*Software Development Kit*”

permite transmitir hacia una aplicación de software, en tiempo real, los datos que definen las señales fisiológicas. En su versión más reciente permite realizar lectura de ECG, EEG, EMG y respiración, además de permitir la inclusión de sensores y transductores externos, con lo que se puede lograr la captura de una mayor combinación de señales (Great Lakes NeuroTechnologies, s.f). En la Figura 2.3 puede apreciarse un dispositivo *BioRadio*.



Figura 2.3: *BioRadio*, equipamiento de lectura de datos fisiológicos inalámbrico.

Fuente: *GLNeuroTech: BioRadio Wearable Tech*¹⁴.

¹⁴ <https://glneurotech.com/bioradio/bioradio-wireless-physiological-monitor/>

BITalino

BITalino es un conjunto de software y hardware, diseñado para poder captar señales emitidas por el cuerpo humano. Posee distintas versiones, las que se diferencian básicamente en la manera en que está distribuido el hardware, pudiendo ser todo en una sola placa capaz de captar todas las señales, o separado en pequeñas placas que reciben señales individuales. Ambas modalidades de hardware poseen ranuras para conectar los transductores (electrodos), que son los que finalmente llevarán la señal emitida por el cuerpo humano, al sistema de recepción de señales de la placa. En cuanto a los sensores incluidos, todas las versiones coinciden en los mismos: EMG, ECG, EEG, actividad electrodérmica (EDA), acelerómetro (ACC) y sensor de luz. En la Figura 2.4, puede apreciarse una placa *BITalino* de placa única, con su batería y electrodos (*BITalino*, s. f.).

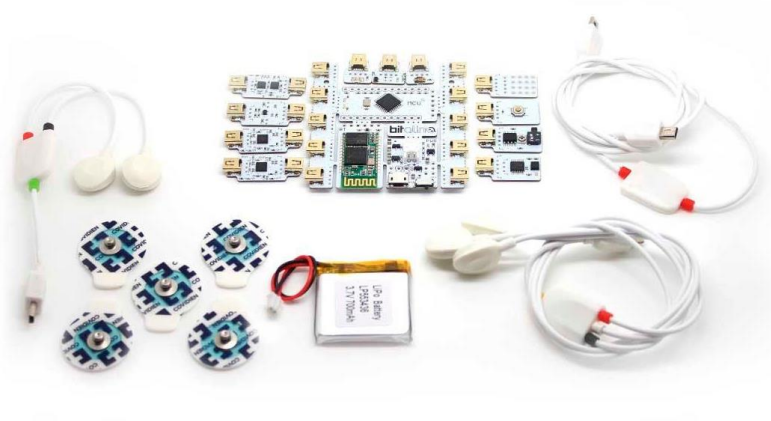


Figura 2.4: *BITalino*, electrodos y batería.

Dispositivos que en conjunto permiten la lectura de datos fisiológicos.

Fuente: *BITalino: (r)evolution Plugged Kit BT*¹⁵.

BioNomadix Smart Center

BioNomadix Smart Center es una interfaz inalámbrica y portátil para medición fisiológica, desarrollado por *BIOPAC Systems*, que sirve de apoyo a la investigación y educación, la cual puede apreciarse en la Figura 2.5. Presenta sensores para ECG, EEG, EMG, EDA, ACC, electrooculograma (EOG), electroglotografía (EGG), respiración (RSP), amplificador de

¹⁵ <http://bitalino.com/en/plugged-kit-bt>

temperatura de la piel (SKT), fotopletismograma (PPG). Posee un software diseñado para la interfaz, llamado *AcqKnowledge*, y que permite realizar mediciones con los sensores ya mencionados. Permite una conexión USB directa con el computador (BIOPAC Systems, 2017).



Figura 2.5: *BioNoMadix Smart Center*

Fuente: Sitio oficial de *BIOPAC*¹⁶

2.1.4 Interacción humano-información

MOTIONS tiene como propósito central, ser una herramienta útil para la investigación de interacciones humano-información en ambientes inmersivos. Una interacción humano-información corresponde a un diseño, el cual provoca una relación entre una máquina, su servicio y el participante. Esta relación se ve afectada por la tecnología que existe en el momento dado de su implementación (Marchionini, 2008).

¹⁶ <https://www.biopac.com/wp-content/uploads/BN-EOG2-T.jpg>

2.2 ESTADO DEL ARTE

Dentro de esta sección se presentan estudios que se han realizado utilizando inmersión en ambientes generados de forma digital. También se detallan las tecnologías que proveen inmersión en la actualidad, considerando hardware y software.

2.2.1 Análisis de respuestas fisiológicas en entornos inmersivos de realidad virtual

Un estudio realizado en el departamento de Ingeniería Biomédica de la Universidad de Hanyang, en Seoul, Korea, utilizó la inmersión para analizar la respuesta fisiológica de participantes no-fóbicos (Jang, Kim, Nam, Wiederhold, Wiederhold & Kim, 2002). El estudio consistió en posicionar a las personas en dos ambientes, uno de manejo y otro de vuelo, buscando demostrar que las respuestas fisiológicas de los participantes se verían afectadas ante estímulos ofrecidos por el ambiente donde se les situaba. Como interfaz de interacción, en ambos casos se utilizó un HMD, adicionalmente, en el caso del simulador de manejo, se agregó un dispositivo que permitía controlar la dirección del vehículo. Para ofrecer retroalimentación vibro táctil al participante, se modificó la silla que sería usada por éste, añadiéndole un equipo *subwoofer*, lo que permitía experimentar vibraciones a quien estuviera sentado en ella. Los resultados del estudio demostraron que la resistencia de la piel y el ritmo cardiaco mostraron excitación en los participantes, y que dicha excitación decrecía con el tiempo una vez finalizada la experiencia.

2.2.2 Enganche e inmersión en ambientes inmersivos de realidad virtual

En el año 2017, se realizó un estudio para comparar el enganche del usuario, la realización de tareas y la distancia recorrida, en un juego, utilizando una visualización de escritorio, y otra con realidad virtual, haciendo uso de un HMD (Alexander, Narayanan, Poys, & Pradeep, 2017). El objetivo del juego era encontrar objetos escondidos lugares estratégicos de un escenario tridimensional. Se podía jugar en tres niveles de dificultad, basados en el tiempo disponible para completar la tarea, y en la cantidad de obstáculos que alejaban al participante del objetivo. Se observó que, en los niveles de dificultad media, la visualización de escritorio permitió a los participantes tardar menos tiempo en completar la tarea que utilizando el HMD, además, la dificultad percibida por los usuarios fue menor, en general, al utilizar la visualización de escritorio. En cuanto al enganche, los usuarios necesitaron mirar una menor cantidad de veces el mini-mapa en la versión de escritorio que al usar HMD, dejando en evidencia, el mayor enganche que tenían con el primer medio de interacción. Luego de analizar los resultados, se llegó a la conclusión de que mayor realismo y campo de visión en una escena no se corresponde directamente con una mejor experiencia de usuario ni desempeño en tareas. Se cree que los resultados negativos que obtuvo la realidad virtual, se produjeron principalmente en los recursos de movilidad ofrecidos por el HMD, por lo que se propone a futuro realizar estudios independientes sobre las acciones de

apuntar y moverse, considerando que ambos aspectos fueron los que reportaron poseer los mayores efectos en los resultados de la evaluación.

2.2.3 Interfaces inmersivas para enganche y aprendizaje

Un estudio acerca de interfaces inmersivas para el enganche y aprendizaje reveló que la inmersión en un ambiente digital puede mejorar el proceso educativo (Dede, 2009). Según el estudio, se puede influir en la educación a través de interfaces inmersivas, de al menos tres formas:

1. **Permitir múltiples perspectivas**, lo que permite comprender fenómenos complejos. Esto se logra cambiando la referencia en una simulación o experiencia. Por ejemplo, primero posicionar al estudiante fuera de donde ocurre cierto fenómeno (perspectiva exocéntrica), y luego posicionarlo dentro del fenómeno (perspectiva egocéntrica). Lo anterior, en algunos casos, solo es posible a través de la realidad virtual, ya sea porque no existen los recursos para lograr este intercambio o porque es físicamente imposible.
2. **Aprendizaje situado**. Esto quiere decir que, para aprender de mejor manera, se recomiendan actividades tácitas, donde el estudiante se encuentre en un entorno de laboratorio, guiado por alguien de mayor conocimiento. Obtener estos entornos puede ser costoso, sin embargo, al utilizar realidad virtual, la inversión suele ser mucho menor.
3. **Transferencia**, que se refiere a la aplicación de conocimiento aprendido desde una situación, en otra. Se demuestra en este estudio, que los estudiantes que han aprendido algo en una simulación, poseen capacidad para resolver problemas similares en situaciones replicadas en el mundo real.

A pesar de lo anterior, en el estudio se menciona que es necesario un estudio para determinar hasta que grado, la inmersión es necesaria para adquirir distintos grados de aprendizaje.

2.3 RESUMEN

En este capítulo se presentó un marco conceptual, donde se aborda el concepto de inmersión y sus distintas definiciones, lo que permite llegar a una idea elaborada de qué es inmersión y como lograr distintos niveles dentro de ésta. Se presenta también, una breve historia de la realidad virtual y se ofrecen tecnologías que permiten interactuar, hoy en día, en ambientes de realidad virtual. En la sección 2.1.3 se entrega una explicación del funcionamiento básico de los equipamientos bioelectrónicos, y las características más importantes de tres de estos dispositivos, *BITalino*, *BioRadio* y *BioNomadix Smart Center*. En la sección 2.1.4, se expone la definición de interacción humano-información. Dentro de este capítulo, también se recorre el estado del arte, mencionando un conjunto de investigaciones que estudian cómo la inmersión en

realidad virtual afecta aspectos del mundo real, como son las respuestas fisiológicas, el aprendizaje y cumplimiento de tareas.

CAPÍTULO 3. ANÁLISIS

En este Capítulo se presentan los antecedentes que rodean el desarrollo a realizar, haciendo énfasis en MOTIONS, la herramienta de software que recibe el módulo de software que permite extender sus funciones.

3.1 ANTECEDENTES

3.1.1 MOTIONS

A continuación, se detallan las distintas iteraciones que han llevado a MOTIONS a ser lo que es hoy en día.

VORTICES

MOTIONS fue concebido, inicialmente, bajo el nombre de *Virtual Reality experimenTation system for Immersive interaCtionS with information*, o VORTICES (Gaete, 2016), con la idea de ser una herramienta versátil y útil para quien desee investigar interacciones con objetos digitales de información en ambientes inmersivos de realidad virtual, soportando las interfaces Oculus Rift, Leap Motion y Open Glove (Monsalve, 2015; Meneses, 2016). En concreto, la tarea de experimentación que el participante de una simulación en VORTICES debe realizar, consiste en buscar, y categorizar imágenes que guarden relación con un tópico o tema en específico, por ejemplo, el “Derrame de petróleo en el Golfo de México”.

BGIIES

El sistema *Body-Gesture Information Interaction Evaluation System*, o BGIIES (González-Ibáñez et al., 2016; Varela, 2015), es un sistema que permite interactuar con objetos de información en ambientes que no necesariamente utilicen realidad virtual a través de una interfaz gestual, Kinect 360 y mouse. BGIIES fue mejorado para cambiar su modelo de interacción, y fue integrado como una extensión de VORTICES (Estay, 2017).

MOTIONS

MOTIONS, además de poseer VORTICES y BGIIES integrados, añade las interfaces de interacción *NeuroSky Mindwave* y *EMOTIV Insight*. Contempla un módulo de integración de interfaces de rastreo ocular y comprende una mejora a la flexibilidad de la herramienta, lo que se ve reflejado en la capacidad de asignar acciones a las distintas interfaces de interacción y de añadir nuevos ambientes de experimentación. Todo esto apoyado por una nueva interfaz de usuario. Si bien MOTIONS posee capacidad de utilizar otras interfaces, en esta sección solo se han nombrado las más relevantes.

A pesar de haber pasado por varias iteraciones antes de llegar al estado actual, el objetivo de MOTIONS se ha mantenido, y es el de ser una herramienta de apoyo a la investigación. Por lo tanto, cada una de sus funciones está pensada para ser usada por alguien que no necesariamente conozca acerca de ciencias de la computación, programación en general, o sobre la lógica interna del software. Es por esto que se otorga ayuda al usuario a través de interfaces simples y configuraciones predeterminadas, cuando es pertinente.

3.1.2 Nivel de inmersión

El nivel de inmersión de MOTIONS, está definido por la retroalimentación visual y auditiva que la plataforma entrega al participante. Para efectos de este trabajo, ambos tipos de inmersión se estudian tomando en cuenta dos aspectos: cantidad de recursos de retroalimentación y calidad de estos recursos.

Inmersión Visual

Los recursos que entregan retroalimentación visual en MOTIONS se listan a continuación:

1. **Skybox:** corresponde a una envoltura que rodea una escena, de forma que crea la ilusión de que la simulación se extiende más allá de los límites de su geometría.
2. **Plataforma:** es un plano que genera la ilusión de que el participante esta sobre una superficie sólida, en lugar de estar flotando en el espacio tridimensional.
3. **Imágenes:** son los objetos digitales de información, y están dispuestos en la escena dependiendo del tipo de visualización que se haya elegido para la simulación. En este momento MOTIONS cuenta con dos visualizaciones, la vista Planar (Figura 3.1) y la vista esférica (Figura 3.2). La vista planar, implementada en BGIIES, muestra las imágenes ordenadas en uno o más planos virtuales, (dependiendo de la cantidad de imágenes a proyectar) dispuestos frente al participante.
4. **Iluminación:** corresponde a la luz que se proyecta en la escena, la cual permite visualizar los objetos a partir de cómo estos la reflejan. MOTIONS utiliza una iluminación básica, obtenida a través de una luz general llamada *Directional Light*, y que es el tipo de luz tradicionalmente usado cuando solo se desea contar con una única fuente de iluminación. Sin la presencia de luz, sería imposible divisar ninguno de los objetos generados en la simulación.

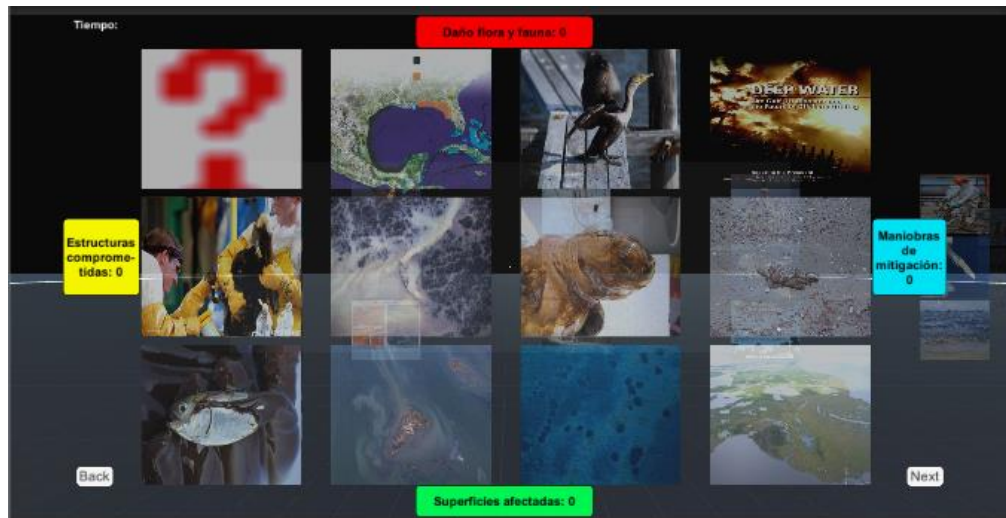


Figura 3.1: Vista planar en una simulación de MOTIONS.

Fuente: Elaboración propia, 2017.

La vista esférica, implementada en VORTICES, dispone las imágenes en un o más esferas virtuales, (también dependiendo de la cantidad de imágenes definidas) alrededor del participante.

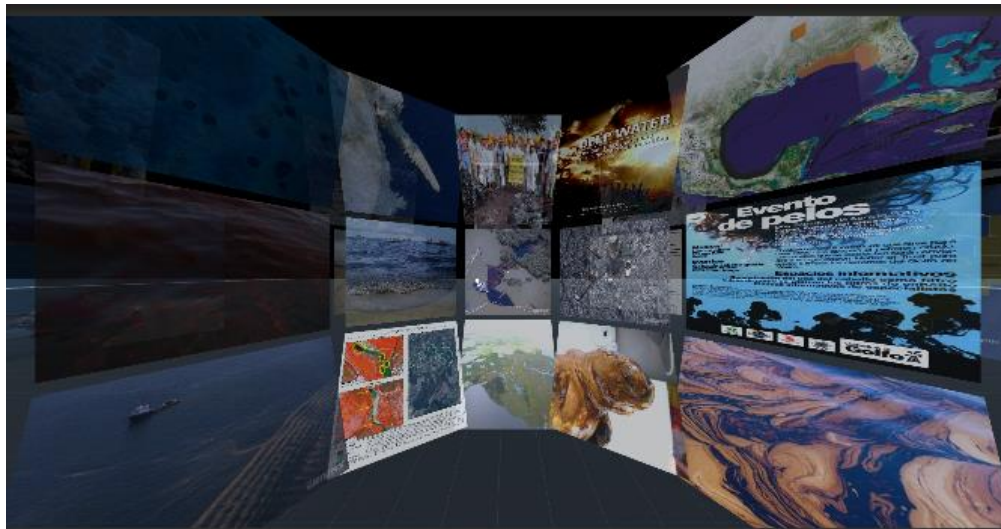


Figura 3.2: Vista esférica en una simulación de MOTIONS.

Fuente: Elaboración propia, 2017.

En cuanto a la calidad de los recursos de retroalimentación visual, en la siguiente lista se detallan los elementos que inciden en ésta:

1. **Rendering Path:** define como se aplica la luz dentro de la escena, y que *Shader Pass* es utilizado. El *Shader Pass* es el encargado de provocar que la geometría de un objeto sea renderizada. Cuando algo es renderizado en *Unity*, primero, la información acerca de

los vértices que definen la geometría del objeto (*Mesh* o Malla) es procesada por una función de vértices, conocida como '*Vertex Shader*', la cual, a grandes rasgos, calcula la posición relativa de los objetos. Luego, esta información es enviada a la función de fragmentos, llamada '*Fragment Shader*' o '*Pixel Shader*', la cual procesa de manera individual cada uno de los píxeles que se desea mostrar por pantalla y retorna la información, que generalmente es un color. La combinación de los *Shaders* y el *Mesh* a utilizar, recibe el nombre de *Shader Pass*. Un *Shader* puede estar compuesto de distintos *Shader Passes*. Un objeto, potencialmente puede pasar por varios *Rendering Path*, dependiendo de la cantidad de luces que lo afectan. MOTIONS actualmente utiliza *Forward Rendering Path*, que es el camino de renderización tradicional (Owens, 2013).

2. **Quality Settings:** las configuraciones de calidad o *QualitySettings*, son un conjunto de elementos que permiten distintos niveles de realismo, principalmente pensando en el rendimiento de la aplicación, pero que afectan la inmersión visual. *Unity* empaqueta elementos de configuración en distintos niveles con nombres descriptivos, de forma que sea posible obtener distintas configuraciones de calidad, sin tener mayor conocimiento de que función realiza cada elemento modificado individualmente. La implementación actual de MOTIONS está configurada en la calidad “fantástica”, que corresponde a la más alta. A grandes rasgos, se configuran características de renderizado y sombreado (Unity3d, 2017).

Si bien existen otros elementos que definen la calidad de los recursos visuales de un programa en *Unity3D*, no es relevante analizarlos en este punto, considerando que la implementación actual de MOTIONS posee muy pocos recursos, y el hacer cambios en estas configuraciones no se ve reflejado en un cambio que influya en la inmersión.

Inmersión Auditiva

En cuanto a recursos que entregan retroalimentación auditiva, MOTIONS presenta:

1. **Sonido al pulsar un botón:** cada vez que el usuario presiona uno de los botones de interacción dentro de la simulación, el sistema reproduce un audio que simula el sonido de un “clic”, con el objetivo de informar el éxito de la acción.

La calidad de los recursos de audio se define a partir de los siguientes elementos:

1. **Modo de reproducción:** en *Unity3D* existen dos modos de reproducción, *Stereo*, donde el audio puede ir de forma diferenciada por el canal de salida de audio izquierdo o derecho, y *Mono*, en el cual los canales de salida de audio izquierdo y derecho reproducen los mismos sonidos (*Mc Squared System Design Group*, s.f.). La reproducción en modo *Stereo* es la que ofrece mayor inmersión al usuario, ya que, con su uso, es posible crear la sensación de que el sonido proviene de alguna dirección en

particular, añadiendo un factor espacial a la simulación. MOTIONS presenta sonido de tipo *Stereo*.

2. **Frecuencia de muestreo:** la frecuencia de muestreo se define como “el número de muestras por unidad de tiempo, tomado desde una señal continua para hacerla discreta o digital”¹⁷ (*Federal Agencies Digitalization Guidelines Initiative*, s. f.). A mayor frecuencia de muestreo, la reconstrucción de la señal original es mejor, y, por lo tanto, el sonido es más real, lo cual se traduce en una mayor inmersión. La implementación actual de MOTIONS presenta una frecuencia de muestreo de 44kHz.

Existen más elementos que permiten influir en la inmersión auditiva, sin embargo, al existir un recurso de inmersión auditiva único y de corta duración, los cambios en los elementos que controlan calidad de los recursos de audio no influyen de mayor manera en la inmersión del usuario dentro de una simulación, por lo tanto, no es relevante analizarlos en este punto.

3.1.3 Interfaces en MOTIONS

Las interfaces que posee el sistema MOTIONS se dividen en dos categorías: las que permiten realizar acciones y las que solo enriquecen la inmersión. Utilizando el panel de configuración de hardware en MOTIONS, que se muestra en la Figura 3.3, es posible establecer los parámetros que rigen el funcionamiento de algunos de los dispositivos, como la calibración del *Eye Tracker*, o el entrenamiento de los comandos mentales en el caso del *EMOTIV: Insight*, además, en dicho panel es donde se decide si utilizar o no cierto dispositivo marcando el “checkbox” que lo acompaña. MOTIONS ofrece también un panel de *mapeo* o creación de relaciones, que se muestra en la Figura 3.4, donde se elige, de entre los dispositivos disponibles para la simulación, que acción se desea gatillar, bajo que comando del dispositivo y con que umbral. Los datos que han sido recibidos por las interfaces quedan guardados en un log para su posterior análisis. El total de interfaces se lista a continuación:

1. Emotiv: Insight
2. NeuroSky Mindwave Mobile.
3. Microsoft Kinect One.
4. Teclado.
5. Mouse.
6. Leap motion.

¹⁷ Traducción libre.

7. The EyeTribe (rastreo ocular).
8. XBOX Gamepad.

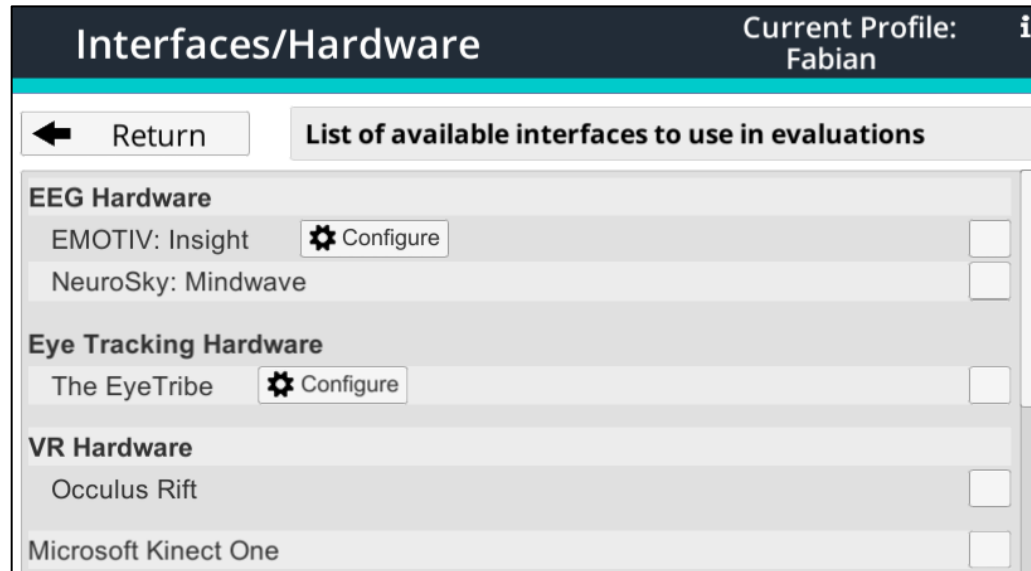


Figura 3.3: Panel de configuración de hardware en MOTIONS.

Fuente: Elaboración propia, 2017.

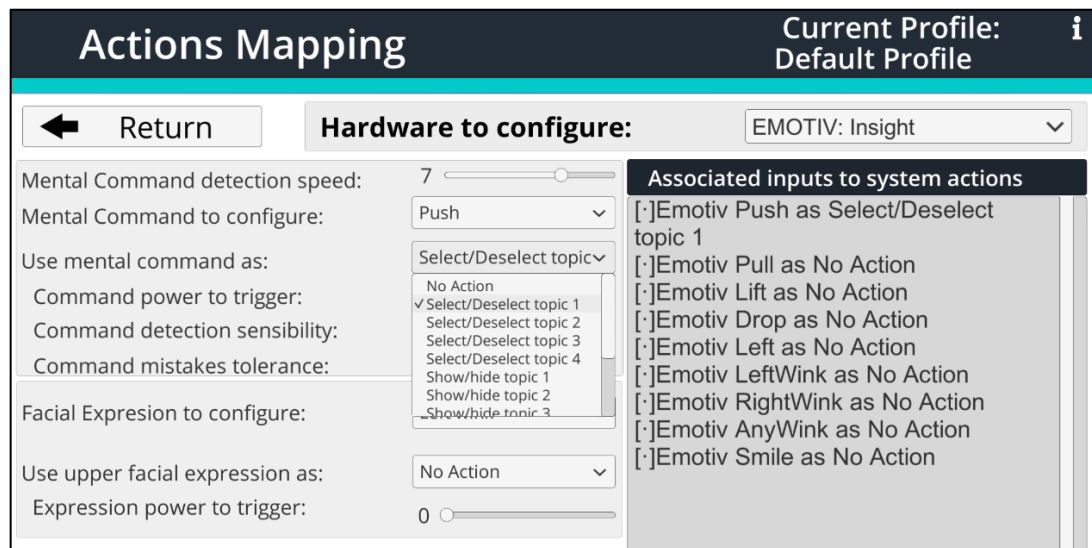


Figura 3.4: Panel de mapeo de acciones en MOTIONS.

Fuente: Elaboración propia, 2017.

3.1.4 Acciones en MOTIONS

MOTIONS permite al participante realizar una lista de acciones definidas, las que se dividen en dos categorías: acciones de la visualización y acciones de la imagen. Las acciones de la

visualización están en función del tipo de visualización elegida para comenzar la simulación, como se listan a continuación:

Acciones de la visualización Planar (BGIIES)

1. Seleccionar tópico.
2. Deseleccionar tópico
3. Mostrar tópicos.
4. Ocultar tópico.
5. Cambiar al plano siguiente
6. Cambiar al plano anterior.

Acciones de la visualización Esférica (VORTICES)

1. Cambiar a la esfera siguiente.
2. Cambiar a la esfera anterior.

Las acciones de la imagen son las que permiten interactuar con el objeto de información sin importar el tipo de visualización elegida.

Acciones de la imagen

1. Seleccionar y deseleccionar imagen
2. Hacer un acercamiento o alejamiento de la imagen (*Zoom in* y *Zoom out*)

Las acciones dentro de la plataforma son independientes de las interfaces, por lo tanto, es posible relacionar distintas interfaces a cualquiera de las acciones listadas. Todas las acciones realizadas por el participante quedan registradas en un log para su posterior lectura y análisis si se requiere. En la actualidad, el sistema solo permite realizar estas relaciones solo con las interfaces *Emotiv Insight*, *Microsoft Kinect One*, *NeuroSky Mindwave Mobile* y Teclado, sin embargo, la arquitectura hace posible agregar nuevas interfaces en el futuro.

3.1.5 Requisitos

A partir de los antecedentes, se desprenden los requisitos funcionales que se muestran en la Tabla 3.1, y que deben ser cubiertos en la implementación del módulo de control de inmersión y en la integración de la nueva interfaz de recepción de señales fisiológicas:

Tabla 3.1: Requisitos Funcionales.

Fuente: Elaboración Propia, 2017.

ID	Descripción	Origen
RF001	El sistema debe poseer siete niveles de inmersión visual donde cada nivel representa cierta calidad y cantidad de recursos gráficos	Inicio
RF002	El sistema debe poseer siete niveles de inmersión auditiva donde cada nivel representa cierta calidad y cantidad de recursos auditivos	Inicio
RF003	El sistema debe permitir la elección de alguno de los niveles de inmersión, visual o auditiva, a través de una interfaz gráfica	Inicio
RF004	El módulo que permite controlar los niveles de inmersión visual y auditiva puede utilizarse en cualquier escena de forma sencilla.	Inicio
RF005	El sistema debe permitir al usuario conocer detalles acerca de cada nivel de inmersión si es que lo desea	P002
RF006	El sistema debe mostrar al usuario el nivel de inmersión que está siendo utilizado dentro de una simulación.	P002
RF007	El sistema debe poseer dos escenas que implementen los niveles de inmersión	Inicio
RF008	El sistema debe permitir al usuario elegir que escena desea utilizar para la simulación	Inicio
RF009	El sistema debe permitir guardar la configuración del ambiente de evaluación, de forma que pueda utilizar las mismas opciones en una nueva sesión	Inicio
RF010	El sistema debe poder recibir las señales emitidas por una placa <i>BITalino</i> .	Inicio
RF011	El sistema debe permitir relacionar las acciones disponibles (ya sea de la visualización o de la imagen), con las señales ECG, EMG, ACC y EDA, recibidas por la interfaz <i>BITalino</i>	Inicio
RF012	El sistema debe proveer al usuario una interfaz donde pueda configurar, con que umbral o en que rango, de la señal recibida por el <i>BITalino</i> , desea gatillar una acción	Inicio
RF013	La interfaz gráfica para configurar el <i>BITalino</i> debe mostrar gráficos en tiempo real, que muestren la señal recibida desde <i>BITalino</i> , y la posición del o los umbrales	P006
RF014	El sistema debe poseer una interfaz gráfica que facilite al usuario la configuración del <i>BITalino</i>	Inicio
RF015	El sistema debe poseer una configuración predeterminada para el dispositivo <i>BITalino</i> con los valores de configuración más típicos	P005

Las restricciones bajo las cuales deben regirse los requisitos funcionales, se detallan en la Tabla 3.2 a través de requisitos no funcionales:

Tabla 3.2: Requisitos no Funcionales.

Fuente: Elaboración Propia, 2017

ID	Descripción	Origen
RNF001	Para la recepción de señales fisiológicas, el sistema debe utilizar una placa <i>BITalino</i>	Inicio
RNF002	El sistema debe integrarse a MOTIONS, sin obstaculizar el correcto funcionamiento de la herramienta original, ni de ninguna de sus características	Inicio
RNF003	El sistema debe implementarse utilizando la misma versión de Unity 3D que utiliza MOTIONS	Inicio
RNF004	El sistema debe proveer configuraciones intuitivas y con nombres descriptivos, de forma que su uso no requiera comprender el funcionamiento interno de la herramienta para poder utilizarla.	P002

3.2 PROTOTIPADO

En esta sección del documento, se presentan los distintos prototipos que permitieron resolver los requisitos funcionales planteados, además de generar otros que no habían sido contemplados en primera instancia.

3.2.1 Primer prototipo: Inmersión visual básica

Dentro de la Tabla 3.3, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. El objetivo de este prototipo fue el de lograr un primer acercamiento a lo que son los niveles de inmersión visual dentro de un ambiente de experimentación. Luego, el foco estuvo en comprender los recursos que entrega Unity3D para lograr distintos niveles de inmersión y como estos recursos se relacionan con los objetos que integran una escena, a través de la creación de un entorno que sitúa al participante en una estación espacial. Para este propósito, se creó una escena en la que se agregaron tres *GameObjects* (clase base de todas las entidades de *Unity3D*), un personaje con forma humana, simulando ser un sujeto de prueba en una experimentación, una esfera, para tener una idea de cómo las distintas configuraciones afectan a los objetos de una escena, y una luz direccional, que permite la visibilidad de los objetos en la escena (de otra forma, los objetos se encontrarían en la oscuridad, y no sería posible diferenciarlos dentro de la escena). Para lograr una impresión más precisa de lo que cada nivel de inmersión ofrece, se creó un script de C# que permite mover la cámara y al personaje en forma simultánea y libre, de manera que se pueda recorrer la totalidad de la escena en cada nivel de inmersión. En el nivel más básico de inmersión visual, o inmersión cero, la cantidad de recursos de inmersión es nula, como se muestra en la Figura 3.5, por lo tanto, la calidad de los recursos es irrelevante.

Tabla 3.3: Datos de implementación para el primer prototipo.

Fuente: Elaboración propia, 2017.

ID Prototipo	P001
Nombre Prototipo	Inmersión visual básica
Objetivos	Implementar niveles básicos de inmersión visual y una de las escenas de experimentación.
Descripción	En este prototipo se muestra una escena de Unity 3D que permite la selección de los tres niveles más bajos de inmersión visual dentro de una de las escenas de experimentación.
Requisitos Funcionales Abordados	RF001, RF004, RF007
Requisitos no Funcionales Abordados	RNF003

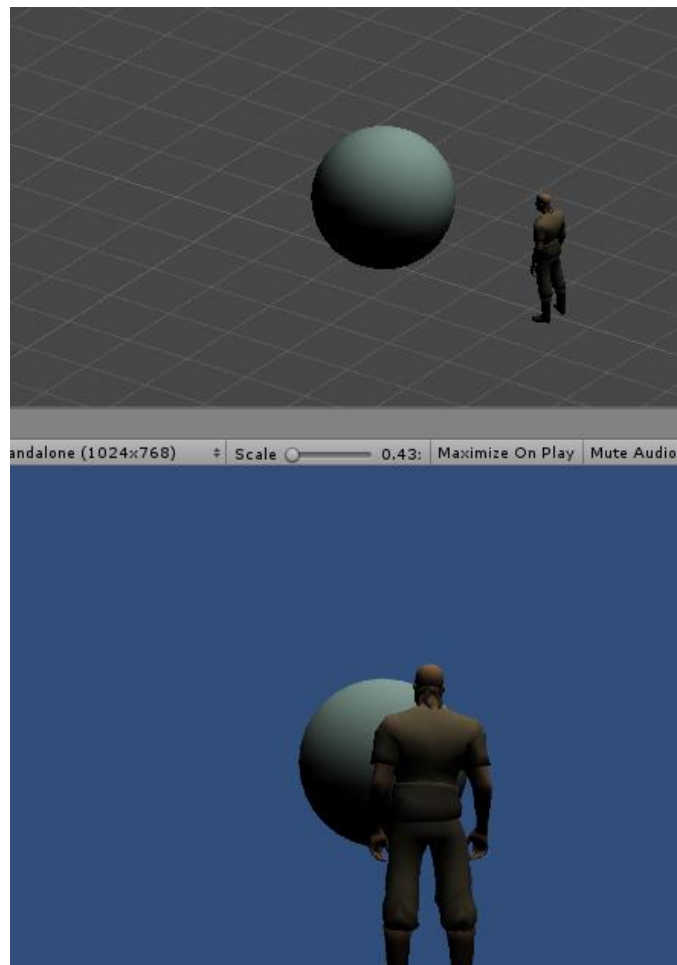


Figura 3.5: Nivel de inmersión visual cero.

Fuente: Elaboración propia, 2017.

Para el nivel dos de inmersión visual, se añadió un *skybox*, el cual otorga la sensación de que la escena se extiende más allá de los límites de su geometría, además de otorgar un contexto espacial al usuario, como se muestra en la Figura 3.6. Es posible modificar la resolución del *skybox*, lo que tiene implicancias para propósitos de resolución. La resolución se dejó al mínimo, considerando que se está en un nivel bajo de inmersión.

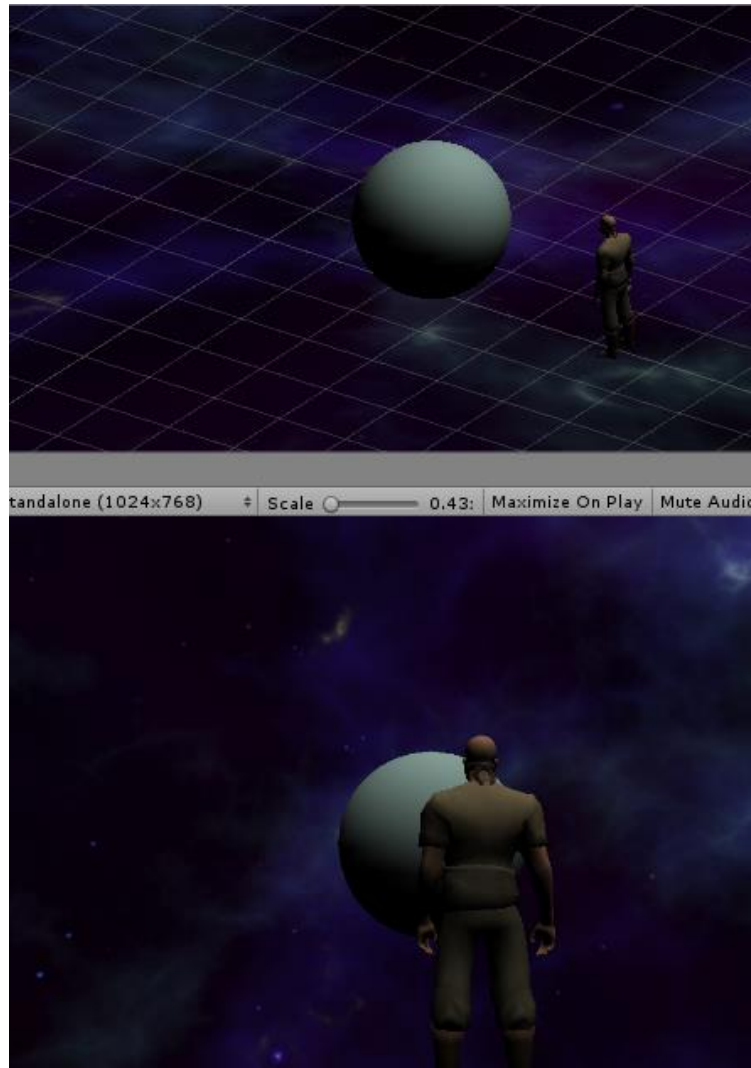


Figura 3.6: Segundo nivel de inmersión visual.

Fuente: Elaboración propia, 2017.

En un tercer nivel de inmersión visual, se agregó una estructura donde está posicionado el personaje, de forma que otorgue la sensación de encontrarse en un contexto físico, donde existe una superficie que sostiene al personaje, y murallas que acotan el espacio, como se puede apreciar en la Figura 3.7. Cabe destacar que los niveles de inmersión son acumulativos, es decir,

el nivel de inmersión tres posee el *skybox* del nivel dos implementado, sin embargo, su resolución es mayor.

Todos los cambios mencionados, referentes a los niveles de inmersión, se realizan a través de código, en un script que hace las veces de controlador, por lo tanto, podría aplicarse a cualquier escena, siempre y cuando, se señalen las configuraciones mínimas a utilizar, en este caso, que *skybox* es el que se desea aplicar y cuál es la estructura a generar.

En resumen, el requisito funcional RF001, se cumplió parcialmente, quedando pendiente cuatro niveles de inmersión adicionales. El RF004 está considerado en este prototipo, ya que basta con utilizar el controlador en la escena sobre la cual se desean aplicar los niveles de inmersión visual. El RF007 también se cumple de manera parcial, considerando que la escena sobre la cual se realizaron las pruebas, es la escena que, cuando esté lista, se utilizará en las simulaciones de MOTIONS.

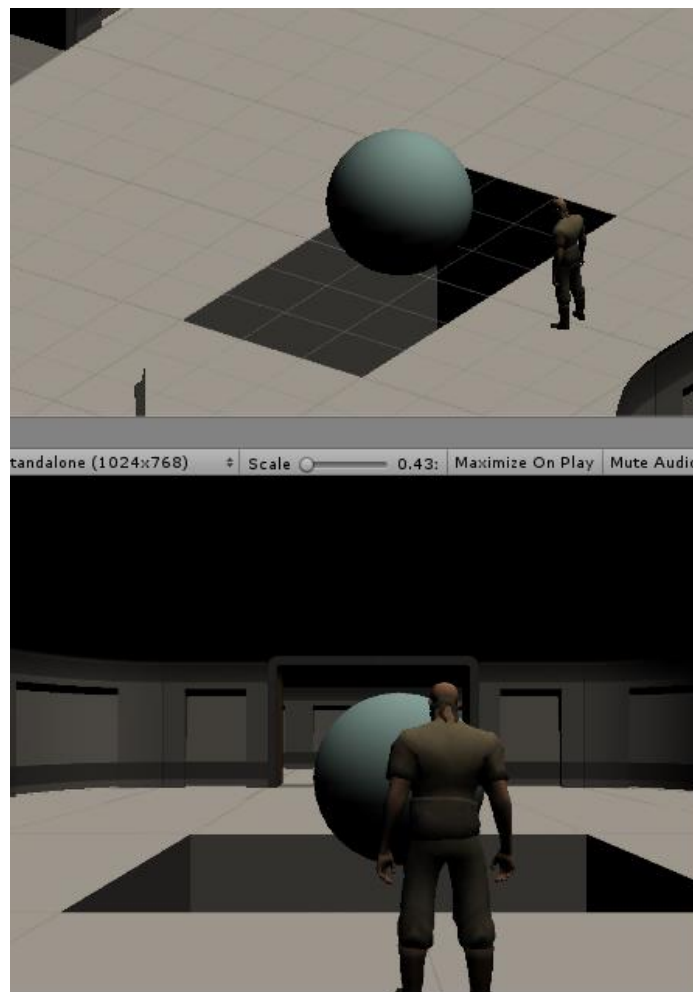


Figura 3.7: Tercer nivel de inmersión visual.

Fuente: Elaboración propia, 2017.

3.2.2 Segundo prototipo: Inmersión visual completa

Dentro de la Tabla 3.4, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. En esta iteración, se agregan cuatro nuevos niveles de inmersión visual a la escena generada en el prototipo P001, esta vez considerando aspectos más complejos de Unity3D, que permiten alcanzar mejores resultados en cuanto a calidad visual.

Tabla 3.4: Datos de implementación para el segundo prototipo.

Fuente: Elaboración propia, 2017.

ID Prototipo	P002
Nombre Prototipo	Inmersión visual completa
Objetivos	Implementar niveles avanzados de inmersión visual y menú de selección de niveles de inmersión.
Descripción	En este prototipo se añaden los cuatro niveles de inmersión visual más altos a la escena creada en el prototipo P001. Además, se añade un menú para seleccionar cada nivel de inmersión.
Requisitos Funcionales Abordados	RF001, RF003, RF004, RF006, RF009
Requisitos no Funcionales Abordados	RNF003, RNF004

En el cuarto nivel de inmersión visual, se agrega iluminación a la escena, como se aprecia en la Figura 3.9, lo que permite distintos grados de luz para diferentes áreas, situación que, con la iluminación que existía en el prototipo P001, era imposible. Además, se incluye un *GameObject* llamado “*Reflection Probe*”, objeto invisible que provoca que los elementos que posean materiales con características reflectivas, reproduzcan las imágenes que los rodean, creando la sensación de que existe un reflejo, como se da con los materiales reflectivos en la vida real. Nótese que el concepto “material” se refiere a un material, de Unity3D, que es una interfaz que permite al desarrollador modificar características de los *shaders*, las que finalmente darán las directivas a la unidad de procesamiento gráfico acerca de cómo los objetos serán renderizados y mostrados por pantalla (Unity3d, 2017a). En la Figura 3.8 pueden apreciarse dos escenas que muestran el interior de un cubo, el cual posee una fuente de luz en la parte superior, y una esfera construida con materiales reflectivos centrada en la base. La escena que se muestra en la parte izquierda de la figura, no posee un *Reflection Probe*, por lo tanto, la esfera no refleja los colores de las murallas del cubo, ni el foco de luz de la cara superior de éste. La escena de la parte derecha de la figura, si presenta un *Reflection Probe*, por lo tanto, puede notarse cómo la esfera refleja los elementos que la rodean, a diferencia del caso anterior.

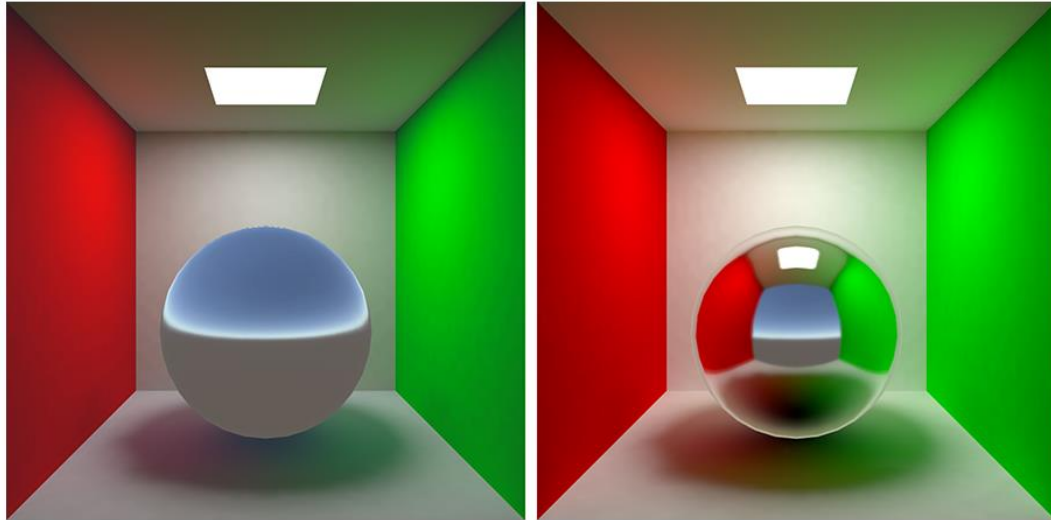


Figura 3.8: Demostración de efectos del uso de *Reflection Probe*.

Comparación entre una escena sin *Reflection Probe* (izquierda) y una con *Reflection Probe* (derecha).

Fuente: *Unity3D: Tutorials, graphics, reflections*¹⁸.

Por otra parte, se introduce el concepto de *Rendering Path*, que es el encargado de definir cómo se aplica la luz dentro de la escena, y que *Shader Pass* es utilizado. Existen tres *Rendering Path* ofrecidos por *Unity3D*, *Legacy Vertex Lit*, *Forward Path* y *Deferred Path*. *Legacy Vertex Lit* es el *Rendering Path* con la menor fidelidad de luces, además no soporta sombras en tiempo real, por estas características se aplica esta configuración a los niveles cero, uno y dos de inmersión visual. *Forward Path*, es el camino de renderización tradicional, gráficamente no otorga tanta fidelidad como el *Deferred Path*, sin embargo, permite trabajar con luces en tiempo real. *Deferred Path* es el camino de renderización que otorga la mayor fidelidad en cuanto a sombras y luces, lo cual es importante si se están utilizando muchas luces en tiempo real, por estas razones, será usado en los tres niveles más altos de inmersión visual. Cabe destacar, que, si el equipo donde se está

¹⁸ <https://unity3d.com/es/learn/tutorials/topics/graphics/reflections>

corriendo MOTIONS no soporta el *Rendering Path* que el nivel de inmersión visual exige, se utilizará el siguiente *Rendering path* que utilice menos recursos, esto se debe principalmente a que las tarjetas de video antiguas no poseen múltiples *Render Targets*, característica que es requerida para la utilización del camino de renderizado *Deferred Path*. Un *Render Target* es “un buffer de memoria para renderizar pixeles¹⁹” (Microsoft, s. f.).

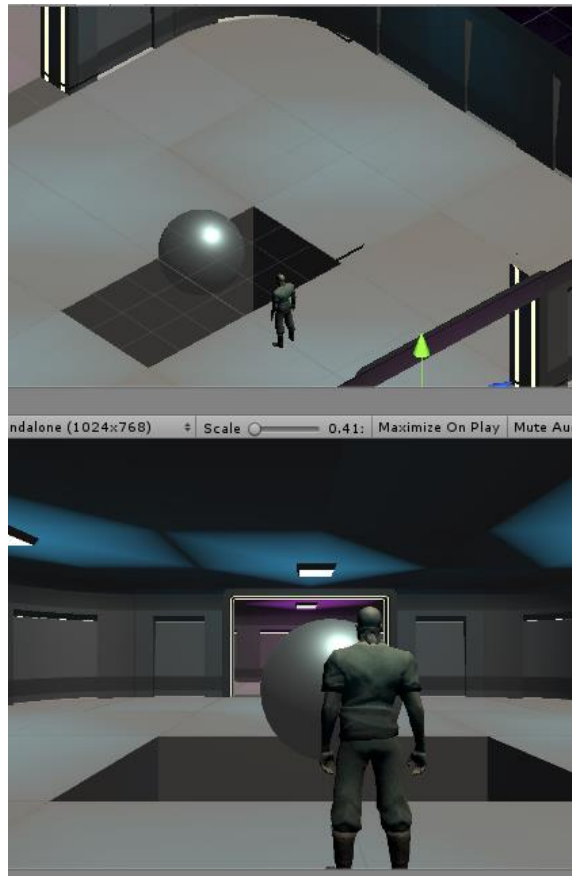


Figura 3.9: Cuarto nivel de inmersión visual.

Fuente: Elaboración propia, 2017.

¹⁹ Traducción Libre.

En el quinto nivel de inmersión visual, se introducen nuevos *GameObjects*, que corresponden a objetos físicos dentro de la escena, relacionados a la temática de ésta, lo que genera un mayor contexto espacial. En este punto cabe mencionar que, para la implementación de los objetos basados en el grado de inmersión, se introduce un *Tag*²⁰ a cada objeto, que representa el nivel de inmersión al que pertenece, por lo tanto, si alguien quisiera desarrollar su propia escena, podría establecer que los objetos aparezcan en el nivel de inmersión de su preferencia, y no necesariamente en el quinto nivel de inmersión. Por otra parte, dentro de este nivel, pueden notarse como los cambios en el *Rendering Path* (recordar que para este nivel y los dos niveles posteriores se utiliza *Deferred Path*, que es el que mejor calidad visual otorga), influyen a la iluminación, obteniendo como resultado imágenes con mayor definición, como se puede ver en la Figura 3.10.

En el sexto nivel de inmersión visual se agrega una luz ambiental. La luz ambiental es una fuente de iluminación global que afecta todos los objetos en la escena en todas las direcciones. Es importante para dar un aspecto más realista a las escenas, puesto que, de no existir, los resultados de luz y sombra son absolutos, es decir, si un objeto obstruye el paso de la luz en algún punto, todo lo que está después de esa obstrucción se verá completamente oscuro, como se ve en la Figura 3.11, donde la pared izquierda de la estructura se ve completamente negra en el caso sin luz ambiental, y parcialmente más oscura (respecto al resto de la estructura) en la escena con luz ambiental.

Para el séptimo nivel de inmersión visual se introduce el concepto de *QualitySettings*, que es un conjunto de configuraciones de calidad gráfica agrupados en distintos niveles. Entre los efectos que se controlan dentro de estas configuraciones están el renderizado y las sombras (puede verse una tabla completa con los efectos controlados por los *QualitySettings* en el Anexo C). En renderizado, puede elegirse si utilizar o no texturas anisotrópicas (las que sirven “para que las texturas luzcan mejor si se ven desde ángulos pequeños”²¹ (Unity3d, 2017b)) y en que momento, elegir distintas calidades de texturas, y configurar el nivel de *antialiasing* a utilizar (el cual “aplica un conjunto de algoritmos diseñados para otorgar una apariencia más suave a los gráficos”²² (Unity3d, 2016)), si es que se usa alguno. Para el sombreado, puede elegirse si se desea que haga sombreado, solo sombreado fuerte, o sombreado fuerte y suave. Además, se puede

²⁰ Un *Tag* es una etiqueta que puede estar relacionada a uno o más *GameObjects* y que puede ser utilizada para aplicar funciones generales a todos los objetos con el mismo *Tag*.

²¹ Traducción libre.

²² Traducción libre.

configurar la resolución del sombreado, la distancia a dibujar, entre otras configuraciones. En la Figura 3.12 se muestra cómo queda la escena con una calidad alta de *Quality Settings*, es decir, texturas en calidad alta resolución anisotrópicas, un *Anti-Aliasing* de *2x Multi Muestreo*, sombras fuertes y suaves y de alta resolución, entre las configuraciones más importantes. Para los niveles anteriores de inmersión (desde el uno al seis) se configuraron los *Quality Settings* de forma progresiva, lo que muestra un cambio de calidad visual para cada nivel, orientado a su grado de inmersión.

Para la selección de los niveles de inmersión visual se creó un menú con un *slider* que permite elegir entre los siete niveles de inmersión, indicando, con nombres descriptivos, cada grado. Además, se le añadió un selector de tipo *dropdown*, para poder elegir la escena sobre la cual se quiere iniciar la simulación, como muestra la Figura 3.13. El nivel seleccionado en este menú se guarda en una estructura de *Unity3D* llamada *PlayerPrefs*, que genera un documento que es independiente de la escena donde se esté trabajando, lo que permite el traspaso de información de una escena a otra.



Figura 3.10: Quinto nivel de inmersión visual.

Fuente: Elaboración propia, 2017.



Figura 3.11: Demostración de uso de luz ambiental.

Comparación entre una escena sin luz ambiental (izquierda) y una con luz ambiental (derecha).

Fuente: Elaboración propia, 2017.

Adicional a todo lo anterior, se agrega, dentro de la simulación, un indicador para saber en que nivel de inmersión se encuentra el participante, además, se añade un botón para volver al menú de configuraciones, como es posible apreciar en la Figura 3.14.

En resumen, el requisito funcional RF001, se cumplió completamente, no quedando pendiente ninguno de los siete niveles de inmersión visual. El RF004 se siguen cumpliendo en este prototipo, ya que la implementación de cada nivel de inmersión visual se realizó siguiendo el mismo esquema del prototipo anterior. El RF006 se cumple casi completo en este prototipo, al ofrecer al usuario un panel que muestra el nivel actual de inmersión visual, pero faltando el nivel actual de inmersión auditiva (que aún no ha sido implementada). El RF007 también se cumple de manera parcial, considerando que la escena sobre la cual se realizaron las pruebas, es la que se utilizará en las simulaciones dentro de MOTIONS.

Al prototipo trabajado, le falta incorporar inmersión auditiva, y una segunda escena completa. Como crítica a este prototipo, es que si bien, es simple de usar para un investigador, no es posible obtener una configuración propia con combinaciones de configuraciones, esto ya que los niveles de inmersión están estructurados de manera fija. Además, no otorga los detalles de cada configuración al usuario.

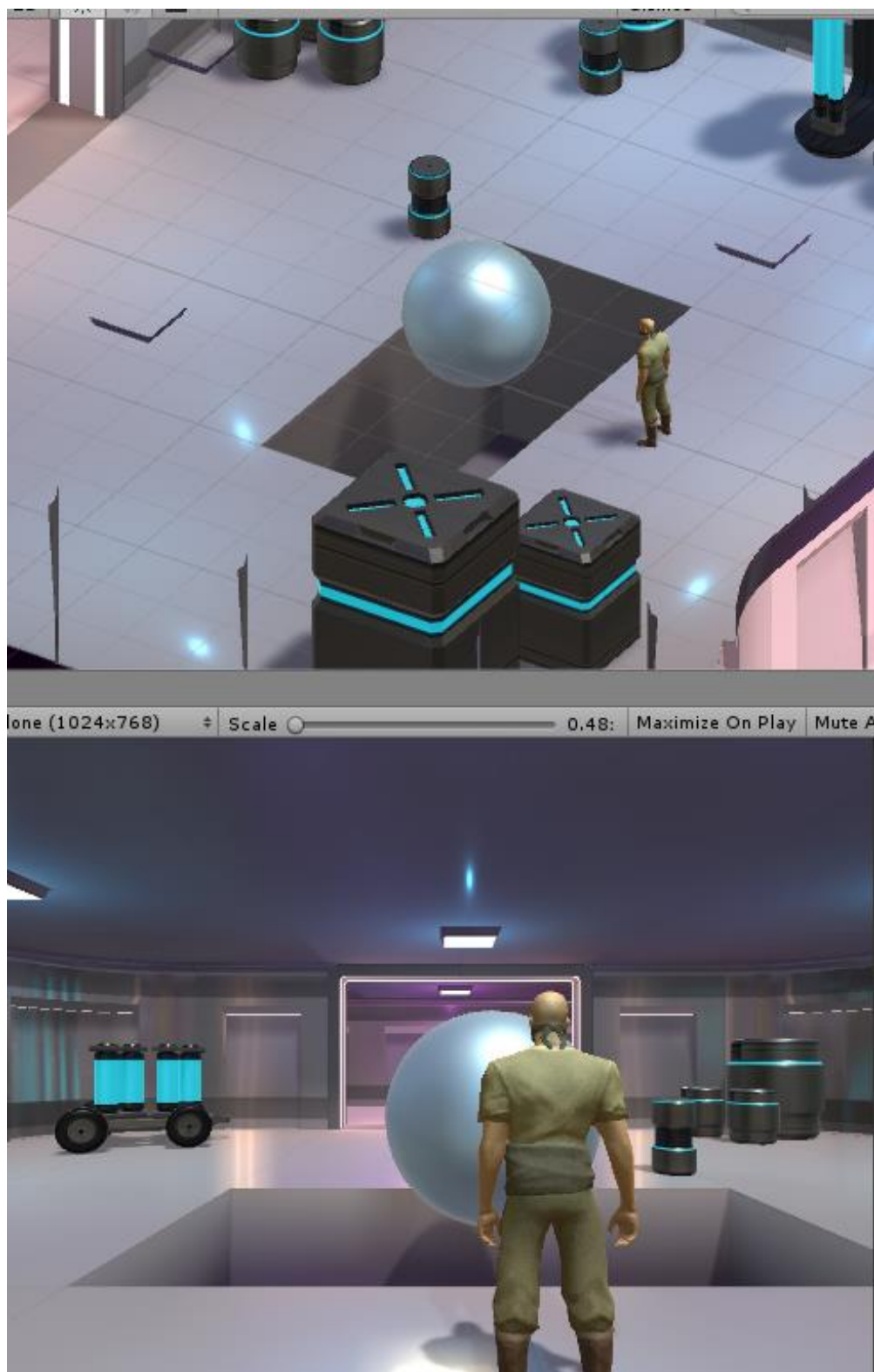


Figura 3.12: Séptimo nivel de inmersión visual.

Fuente: Elaboración propia, 2017.



Figura 3.13: Menú de selección de niveles de inmersión y de escena.

Fuente: Elaboración propia, 2017.



Figura 3.14: Panel que muestra al usuario el nivel de inmersión visual actual.

Fuente: Elaboración propia, 2017

3.2.3 Tercer prototipo: Inmersión auditiva completa

Dentro de la Tabla 3.5, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. Para la inmersión auditiva, los recursos de inmersión a utilizar dependen netamente de la escena, pero en general son dos, un sonido ambiental, como, por ejemplo, el sonido del viento, y sonidos emitidos por objetos en la escena, como una radio que reproduzca una canción. En cuanto a la calidad de los sonidos, los elementos que controlan este factor son: el modo de sonido, ya sea *Mono* o *Stereo*, la frecuencia de muestreo, y la presencia o ausencia de sonido en tres dimensiones. Al igual que ocurre con la inmersión visual, para el primer nivel de inmersión auditiva, no existe ningún tipo de retroalimentación auditiva.

Tabla 3.5: Datos de implementación para el tercer prototipo.

Fuente: Elaboración propia, 2017.

ID Prototipo	P003
Nombre Prototipo	Inmersión auditiva completa
Objetivos	Implementar la inmersión auditiva completa, y añadirla al menú de selección de niveles de inmersión.
Descripción	En este prototipo se añaden los seis niveles de inmersión auditiva a la escena creada en el prototipo P001. Además, se agrega la opción de elegir el nivel de inmersión auditiva al menú de selección de niveles de inmersión.
Requisitos Funcionales Abordados	RF002, RF003, RF004, RF006
Requisitos no Funcionales Abordados	RNF003, RNF004

En el segundo nivel de inmersión auditiva existe un sonido ambiental, que se reproduce de manera *Mono*, es decir, por ambas salidas de audio (izquierda y derecha) se reproduce exactamente el mismo sonido. Además, la frecuencia de muestreo se establece en 22050 Hz, lo que es considerado una calidad baja, como ejemplo, corresponde a la mitad de la frecuencia que utiliza un CD de audio. Para el nivel tres el modo de reproducción continúa siendo *Mono*, sin embargo, la frecuencia de muestreo se establece en 32000 Hz, frecuencia que es utilizada, por ejemplo, por casetes de audio. Desde el nivel cuarto hasta el séptimo, se establece el modo de audio en *Stereo*, además, la frecuencia aumenta a 44100 Hz, 48000 Hz, 88200 Hz y 96000 Hz, respectivamente. Al igual que en la inmersión visual, los recursos de audio están marcados con un *Tag*, de forma que, si se desea lograr inmersión de forma progresiva, se marcan recursos de audio por nivel, especificando a que grado de inmersión corresponden.

Los recursos de audio pueden configurarse para ser reproducidos en tres dimensiones. Esto crea la sensación de que los sonidos provienen de una dirección particular, por lo tanto, agregan espacialidad a la simulación. Para lograr esto, el recurso de audio se reproduce de forma dirigida hacia los canales de salida izquierdo y derecho en función de la proximidad y orientación del usuario respecto al sonido. En *Unity3D*, se puede elegir hasta que punto de la escena se reproducirá el audio de forma completa, en que punto comenzará a decaer, si existirá o no efecto *Doppler*, y bajo que tipo de curva se comportará el decaimiento. En la Figura 3.15, se puede ver el área que delimita la reproducción de audio a volumen completo.



Figura 3.15: Recurso de audio en tres dimensiones, donde se delimita su alcance.

Fuente: Elaboración propia, 2017.

Cabe destacar que para que el sonido ambiental se reproduzca de forma *Stereo*, es necesario que la fuente de audio este en este formato, de lo contrario, será imposible reproducir este modo. Además, como se ve en la Figura 3.16, se ha agregado un indicador de nivel de inmersión auditiva al panel creado en el prototipo P002, y un *slider* al menú creado en el prototipo P002 para controlar los niveles de inmersión auditiva. Por otra parte, es importante mencionar que las configuraciones del muestreo deben realizarse antes de iniciar una escena, debido a una restricción interna de *Unity3D*.



Figura 3.16: Integración de inmersión auditiva a las interfaces visuales.

Estado de inmersión auditiva al panel de la simulación (izquierda) y al selector de nivel de inmersión (derecha).

Fuente: Elaboración propia, 2017.

En resumen, el requisito funcional RF002, se cumplió completamente, no quedando pendiente ninguno de los siete niveles de inmersión auditiva. El RF004 se siguen cumpliendo en este prototipo, ya que la implementación de cada nivel de inmersión auditiva se realizó siguiendo un esquema similar al de la inmersión visual, lo que permite aplicar el módulo en distintas escenas. El RF006 se cumple en este prototipo, al ofrecer al usuario un panel que muestra el nivel actual de inmersión visual y también auditiva. El RF007 también se cumple de manera parcial, considerando que la escena sobre la cual se realizaron las pruebas, es la que utilizará en las simulaciones dentro de MOTIONS, faltando únicamente una segunda escena completa. Al igual que en la inmersión visual, la inmersión auditiva es simple de usar para un investigador, sin embargo, no es posible obtener una configuración propia con combinaciones de configuraciones, esto ya que los niveles de inmersión están estructurados de manera fija, por otra parte, los detalles de cada configuración no son visibles para el usuario.

3.2.4 Cuarto prototipo: Integración de niveles de inmersión en MOTIONS

Dentro de la Tabla 3.6, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. Antes de presentar el prototipo, es necesario comprender el funcionamiento de las escenas dentro del sistema. Para que una escena en MOTIONS funcione, es necesario iniciar sesión, o utilizar el perfil predeterminado. Una vez dentro, se debe realizar las configuraciones necesarias, es decir, ingresar la ruta donde se encuentran los objetos de información (las imágenes), el prefijo que posee cada objeto de información, un documento formato csv que posee metadatos sobre los objetos de información, la cantidad de objetos de información a proyectar, el tipo de visualización (planar o esférica), los hardware a utilizar y la relación entre los hardware y las acciones (todo esto, en el menú que aparece en la Figura 3.19, y los submenús que se desprenden de éste). Una vez establecidos estos parámetros, es posible iniciar una simulación presionando el botón *Start Evaluation*, lo que abre una escena donde se generan distintos *GameObjects*.

Tabla 3.6: Datos de implementación para el cuarto prototipo.

Fuente: Elaboración propia, 2017.

ID Prototipo	P004
Nombre Prototipo	Integración de niveles de inmersión en MOTIONS
Objetivos	Añadir los dos módulos de inmersión a MOTIONS.
Descripción	Se añaden los módulos creados en los prototipos P001 (mejorado en el P002) y P003 a MOTIONS, de forma que puedan ser usados en una escena de experimentación.
Requisitos Funcionales Abordados	En este prototipo no se aborda ningún requisito funcional.
Requisitos no Funcionales Abordados	RNF002, RNF003, RNF004

A continuación, se listan los *GameObjects* generados al iniciar una simulación. Además, se analiza donde debería ir el controlador generado en los prototipos P001, P002 y P003:

1. **Loaders**, que contiene a los encargados de relacionar datos de las configuraciones de la evaluación (asociada a un perfil) con entradas de los dispositivos activos. No existe relación entre los *Loaders* y el controlador de niveles de inmersión, ya que si bien, se debería configurar el nivel de inmersión en el menú principal asociado a un perfil, es independiente de las entradas de los dispositivos activos.
2. **Action Manager**, que controla el intercambio entre las entradas y las acciones del sistema. Tampoco existe relación con un controlador de niveles de inmersión, debido a que sus propósitos son muy dispares.

3. **Interface Manager**, encargado de iniciar los módulos que inician los servicios de las interfaces seleccionadas para la simulación. Típicamente los módulos poseen el nombre “<Interfaz>Manager”, donde <Interfaz> es el nombre del dispositivo. El controlador de niveles de inmersión no es una interfaz de simulación, por lo tanto, no debería ir en el Interface Manager.
4. **Visualizations**, que poseen los *GameObjects* necesarios para organizar y mostrar los objetos de información, ordenados de acuerdo a la visualización elegida en el menú de configuración. Considerando que este *GameObject* está enfocado únicamente para visualizaciones, no debería existir un controlador de niveles de inmersión aquí, puesto que el nivel de inmersión es independiente de la visualización utilizada.
5. **MOTIONS Manager**, el cual maneja la inicialización de una evaluación, y como se generan los archivos de salida de datos. Si bien el controlador de niveles de inmersión podría ubicarse dentro del MOTIONS manager, no es necesario configurarlo antes de iniciar la simulación, por lo tanto, es mejor no sobrecargar MOTIONS Manager con tareas que podrían efectuarse posteriormente. Sin embargo, la configuración de la frecuencia de muestreo del nivel de inmersión auditiva debe realizarse aquí, debido a que no se puede cambiar una vez que ya se ha inicializado la escena.
6. **Managers**, que contiene al *InteractionManager*, el que controla las relaciones de los dispositivos con los objetos de la simulación. y el *InformationObjectManager*, encargado de controlar la inicialización de los distintos módulos de creación de objetos digitales de información. El propósito de este *GameObject* es el de almacenar *Managers* en general, por lo tanto, el controlador de niveles de inmersión debería ir aquí, además, para mantener consistencia, su nombre se cambia a *ImmersionManager*.

Como se muestra en la Figura 3.19, dentro del menú principal de MOTIONS está comprendida la inmersión, sin embargo, no posee ninguna funcionalidad. En este prototipo, el botón de inmersión, que aparece desactivado, se deja activo, y se le añade la función de abrir un menú similar al de los prototipos P001, P002, y P003, pero siguiendo la estética de MOTIONS, como se ve en la Figura 3.20. Una diferencia que se tuvo que implementar, es que en MOTIONS se utiliza una estructura llama *GLPlayerPrefs* para efectos de configuraciones, que permite estructuras de dato más complejas dentro de las preferencias, por lo tanto, se tuvo que adaptar el *PlayerPrefs* original para calzar con las configuraciones de *GLPlayerPrefs*. Una vez configurado el nivel de inmersión, basta con agregar el *ImmersionManager*, las estructuras y *GameObjects* con sus respectivos *Tags*, a la escena donde se realizará la simulación, de esta forma, se pueden experimentar los distintos niveles de inmersión, visual y auditiva, actuando, como se ve en la Figura 3.17 y en la Figura 3.18.



Figura 3.17: Inmersión integrada en MOTIONS, con una visualización Planar.

Fuente: Elaboración propia, 2017.

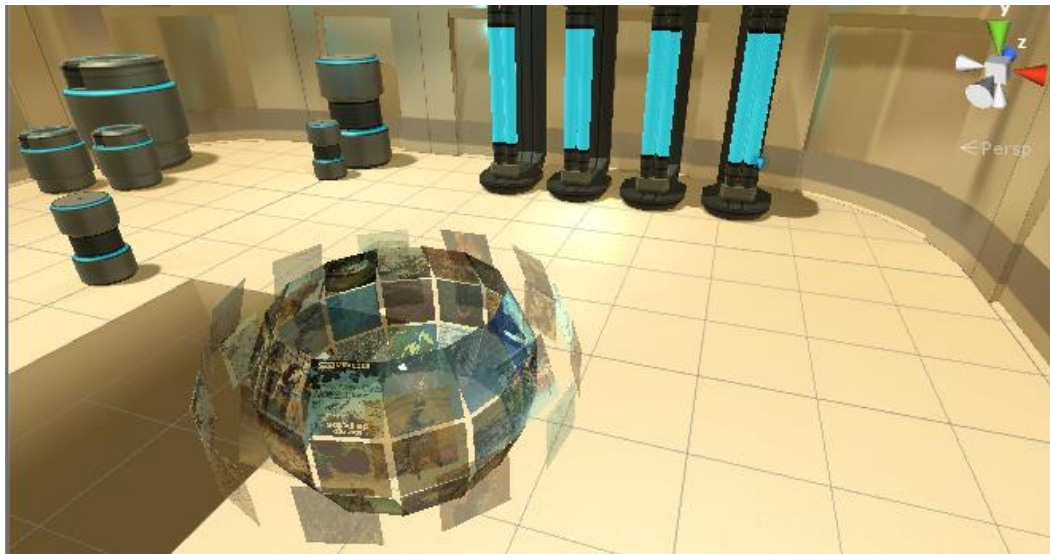


Figura 3.18: Inmersión integrada en MOTIONS, con una visualización Esférica.

Fuente: Elaboración Propia, 2017.

Output path

Information Object

Visualization

Immersion

Hardware

Action Mapping

User Interface

Profile: Default Profile
 +
 edit
 delete

Evaluation: Default Evaluation
 +
 edit
 delete

User ID: 157
 edit

Information object: PanelImage

Visualization: Plane

Immersion:

Hardware (interfaces): see all

Actions mapped: see all

Evaluation Time: 1
 Start Evaluation

Figura 3.19: Menú principal de MOTIONS.

Fuente: Elaboración propia, 2017.

Return

Visual Immersion

Max

Select the scene

Sci-Fi Scene

Auditive Immersion

Min

Figura 3.20: Menú de configuración de los niveles de inmersión en MOTIONS.

Fuente: Elaboración propia, 2017.

En resumen, este prototipo no comprendía el cumplimiento de ningún requisito funcional. En su lugar, se logró completar el requisito no funcional RNF002, ya que la integración con MOTIONS fue exitosa.

3.2.5 Quinto Prototipo: Conexión de Unity3D y BITalino

Dentro de la Tabla 3.7, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. Para lograr la conexión entre *Unity3D* y *BITalino*, se utilizó una API creada por Verhulst y Paulin Doux (2014), que permite recibir datos de la placa en una escena. Para esto es necesario agregar el *GameObject* de nombre *BITalino*, el cual posee los scripts de C# necesarios para la configuración de puertos de conexión por *Bluetooth* (*BITalinoSerial Port*), el manejador de la interfaz (*ManagerBITalino*) y el lector de datos (*BITalinoReader*). La API provee una escena a modo de demostración, la cual muestra los valores recibidos por la placa *BITalino* en una línea de texto. Lo que se aprecia en la Figura 3.21, es una versión modificada de la escena de demostración incluida en la API, leyendo datos recibidos por los receptores EMG y ECG. A la escena se le agregó el indicador de cada señal recibida (cada indicador y su significado pueden revisarse en la Tabla 3.8). Adicionalmente, se separó cada señal en función del formato del dato recibido, es decir, los que se reciben en su versión análoga (parte superior de la Figura 3.21) o en su versión digital (parte inferior de la Figura 3.21).

Tabla 3.7: Datos de implementación para el quinto prototipo

Fuente: Elaboración propia, 2017.

ID Prototipo	P005
Nombre Prototipo	Conexión de Unity3D y <i>BITalino</i>
Objetivos	Reconocer los requisitos para conectar una placa <i>BITalino</i> con Unity3D, y lograr esta implementación en la escena creada en los prototipos anteriores e implementada en MOTIONS.
Descripción	En este prototipo se realiza una conexión entre Unity3D y una placa <i>BITalino</i> , utilizando una API creada por Adrien Verhulst y Paulin Doux ²³ . Además, a modo de prueba, se muestran datos obtenidos del <i>BITalino</i> en una de las escenas que posee la inmersión integrada dentro de MOTIONS.
Requisitos Funcionales Abordados	RF010
Requisitos no Funcionales Abordados	RNF001, RNF002, RNF003

²³ <http://bitalino.com/en/development/apis>

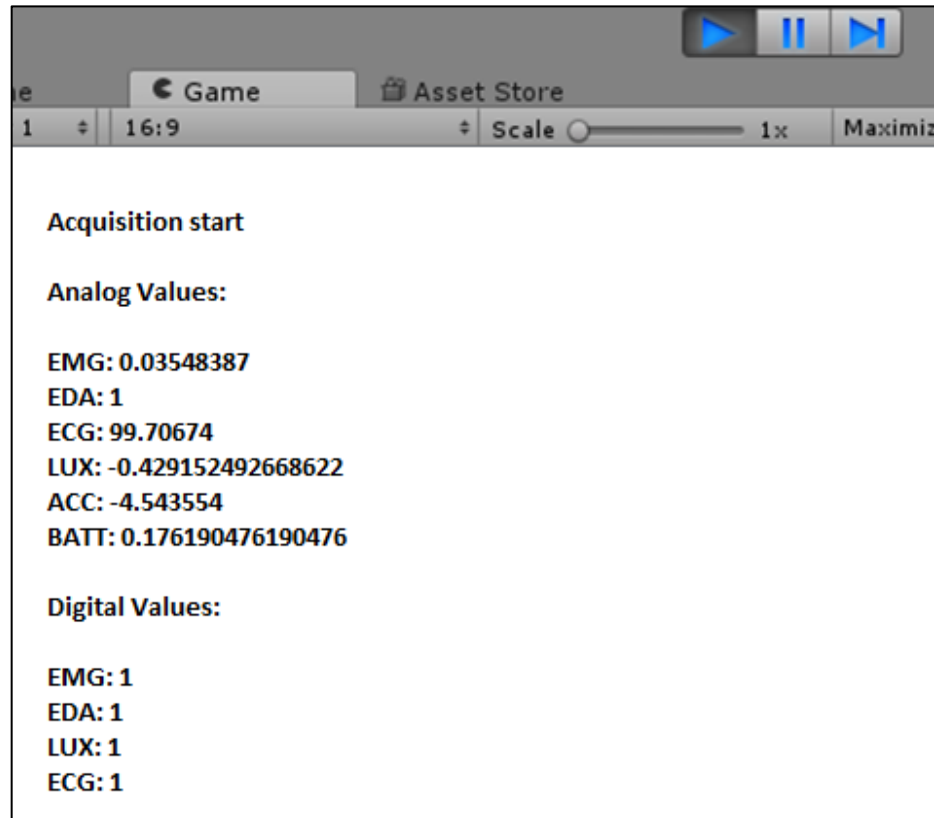


Figura 3.21: Datos obtenidos a través de una placa *BITalino* en una escena de *Unity3D*.

Fuente: Elaboración propia, 2017.

Tabla 3.8: Señales recibidas, identificador y pequeña descripción.

Fuente: Elaboración propia, 2017.

Señal Recibida	Identificador	Descripción
Electromiografía	EMG	Mide la actividad eléctrica muscular.
Actividad Electrodermica	EDA	Mide los cambios en actividad ecléctica de la piel o conductividad eléctrica de ésta.
Electrocardiograma	ECG	Mide el ritmo cardiaco.
Sensor de luz	LUX	Sirve para detectar cambios en la luz ambiental.
Acelerómetro	ACC	Mide la aceleración producida por el movimiento.
Batería	BATT	Mide la cantidad de carga restante en la batería

Además, en esta implementación, se realizó la prueba de medir EMG, conectando los electrodos de la placa como se muestra en la Figura 3.22, y levantar una alarma (por consola) cada vez que la lectura fuera mayor que 1, o menor que -1 (como se aprecia en la Figura 3.23, en la cual

aparece la consola de *Unity3D*, que imprime la medición de EMG, y adicionalmente un indicador si es que ese EMG escapa del rango entre -1 y 1).



Figura 3.22: Conexión de electrodos de un *BITalino* para toma de EMG.

Fuente: Elaboración propia, 2017.

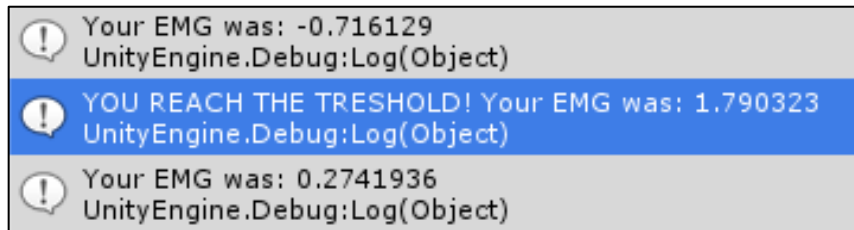


Figura 3.23: Datos recibidos al medir EMG en una escena de *Unity3D*.

Fuente: Elaboración propia, 2017.

A modo de prueba, se agregó la lectura de datos del *BITalino* dentro de una escena de experimentación de MOTIONS, para lograr un primer acercamiento a la integración de la nueva interfaz. Los resultados pueden verse en la Figura 3.24.

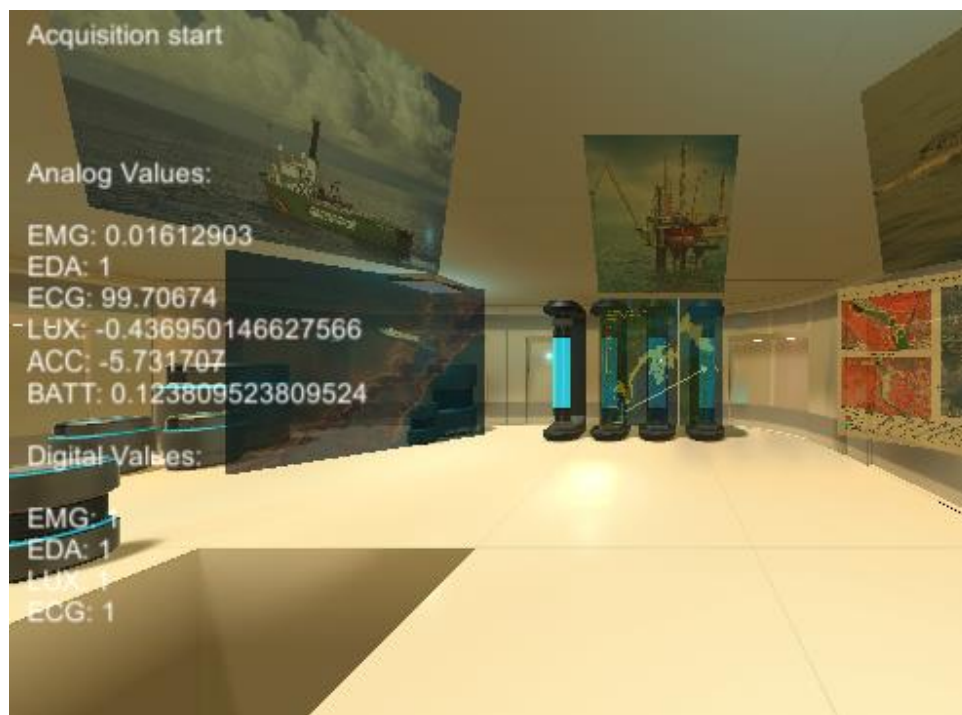


Figura 3.24: Lectura e impresión por pantalla de datos leídos desde *BITalino*.

Fuente: Elaboración propia, 2017.

En resumen, se cumplió el requisito funcional RF010, ya que el sistema permite recibir señales desde una placa *BITalino*. Además, el prototipo se rige bajo los requisitos no funcionales RNF001, ya que la recepción de señales se hace efectivamente a través de una placa *BITalino*, y RNF002, ya que, al recibir señales de esta placa no se interfiere en ninguna de las funciones de MOTIONS.

3.2.6 Sexto Prototipo: Integración de recepción de señales fisiológicas en MOTIONS

Dentro de la Tabla 3.9, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. La arquitectura de MOTIONS permite la conexión y configuración de interfaces de forma independiente a las acciones, y a la lógica de la tarea de experimentación, esto gracias al trabajo realizado por Pollmann (2017). Para configurar *BITalino*, se agregó una opción en el menú de hardware de MOTIONS, que permite seleccionar si interfaz, y bajo que parámetros se registrará, como muestra la Figura 3.25. En este caso, “*Communication Port*” es el puerto de *Bluetooth* donde esta pareado el *BITalino*, “*Baud Rate*” o tasa de baudios, corresponde a la tasa de recepción de dato, “*Sampling Rate*” es la frecuencia de muestreo, bajo la cual se reducirá la señal continua del *BITalino* a una señal discreta, y “*Buffer Size*” es el tamaño del buffer donde se almacenarán temporalmente los datos leídos. Todos estos campos poseen una configuración predeterminada, donde si bien, no se garantiza el correcto funcionamiento, se ofrecen valores que en la mayoría

de los casos suelen ser los adecuados. Cuando se presiona el botón “Save”, cada valor seleccionado se guarda en el perfil del usuario.

Tabla 3.9: Datos de implementación para el sexto prototipo.

Fuente: Elaboración propia, 2017

ID Prototipo	P006
Nombre Prototipo	Integración de recepción de señales fisiológicas en MOTIONS.
Objetivos	Añadir a MOTIONS, la capacidad de configurar las preferencias de una placa <i>BITalino</i> , y de relacionar las señales recibidas con acciones de MOTIONS.
Descripción	En este prototipo se añade la capacidad, en MOTIONS, de seleccionar la interfaz <i>BITalino</i> para su utilización. Esto se cumple, primero, agregando un menú de configuración, para poder establecer los parámetros que rigen el funcionamiento de la placa <i>BITalino</i> , luego, se agrega la opción de relacionar, a través de una interfaz gráfica, las señales recibidas con acciones de MOTIONS, bajo algún parámetro que funcione de umbral.
Requisitos Funcionales Abordados	RF011, RF012, RF013, RF014, RF015
Requisitos no Funcionales Abordados	RNF001, RNF002, RNF003, RNF004

Una vez agregada una nueva interfaz a MOTIONS, el sistema se encarga de dejarla disponible para ser elegida en el menú “*Action Mapping*”, que es donde se unen las interfaces con las acciones. Si bien, las acciones en MOTIONS se relacionan con las interfaces, no lo hacen de forma directa, sino que lo hacen con señales propias de cada interfaz, que en este caso pueden ser ECG, EMG, ACC y EDA. Cada señal puede activar una acción basada en el cumplimiento de alguna condición. Para todas las señales se agregó dos tipos de condición: el umbral, que es un valor mínimo que debe alcanzar la señal para poder gatillar la acción, y el rango, que son dos valores entre los cuales puede moverse la señal para disparar la acción.

Teniendo esto en consideración, se generó la vista que se aprecia en la Figura 3.26, donde el usuario puede seleccionar la interfaz, la señal, el disparador, y la acción. Además, se añade un gráfico que otorga una representación gráfica de la señal seleccionada, y líneas, que delimitan los valores mínimos y máximos seleccionados para el caso del rango, o solo el valor mínimo, para el caso del umbral, lo que facilita la configuración. Todos los valores elegidos en este menú, quedan guardados en el perfil del usuario actual.

Por otra parte, se agregaron scripts propios de la interfaz *BITalino*, para que la conexión y captura de datos siguiera el formato del resto de las interfaces ya implementadas.

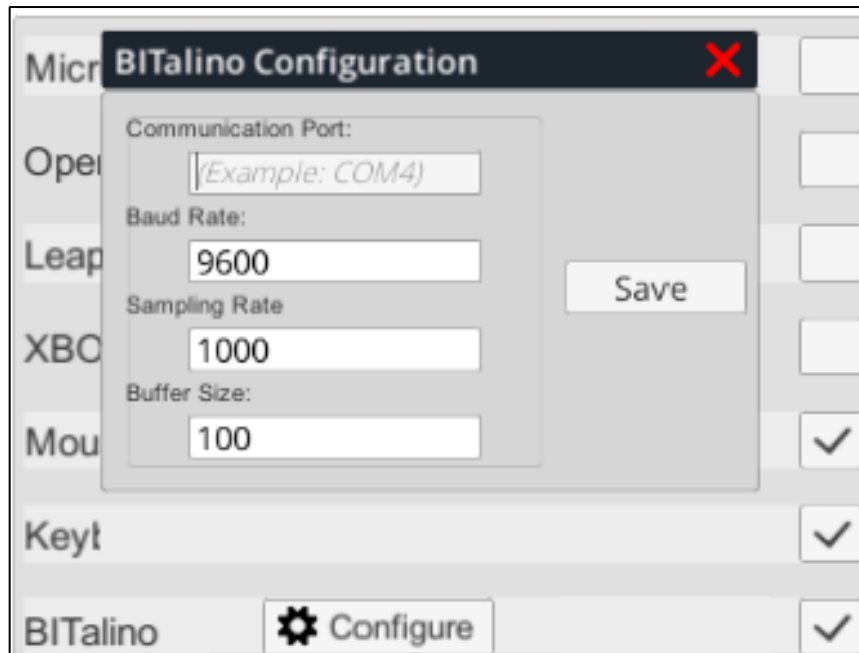


Figura 3.25: Menú de configuración de *BITalino*.

Fuente: Elaboración propia, 2017.

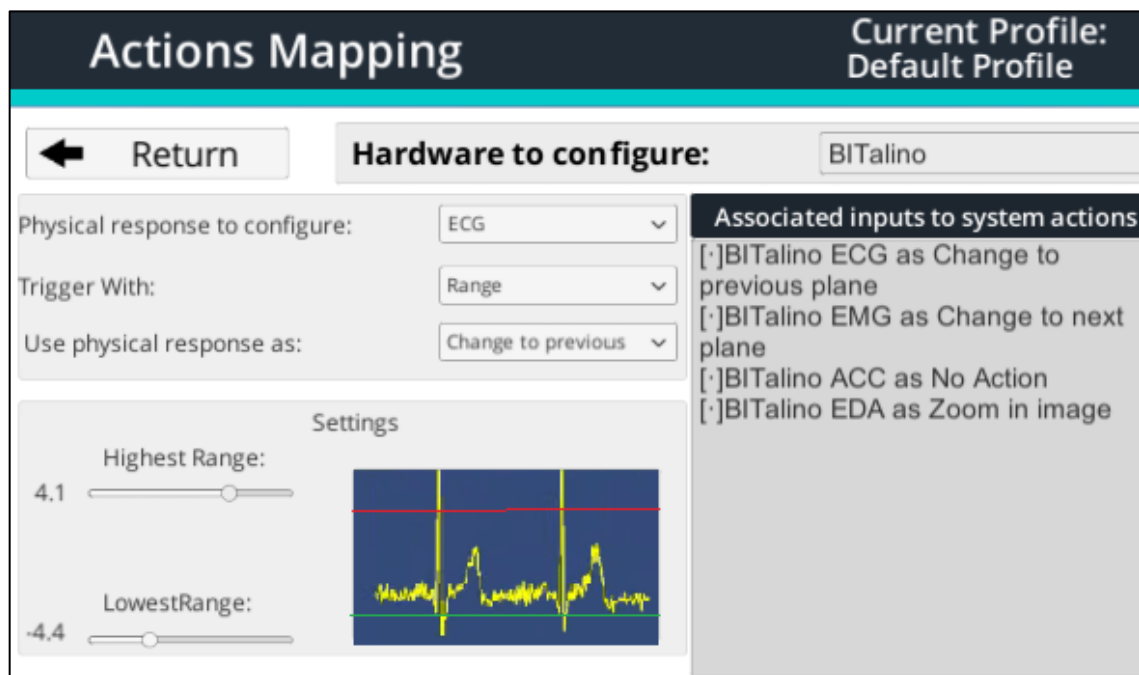


Figura 3.26: Menú de *Action Mapping* para *BITalino*.

Fuente: Elaboración propia, 2017.

En resumen, se cumplieron los requisitos funcionales RF011, RF012 y RF013 de forma completa al agregar *BITalino* al menú *Action Mapping* y permitiendo que estas relaciones tengan

consecuencias reales en una simulación de MOTIONS. Además, se muestran gráficos que facilitan la configuración del umbral o rango de acción. Los requisitos funcionales RF014 y RF015 también se cumplen de forma completa, al agregar opciones de configuración en el menú Hardware, que influyen efectivamente en los parámetros de funcionamiento de *BITalino*, además se ofrecen valores predeterminados de configuración.

3.2.7 Séptimo prototipo: Nueva escena y mejoras

Dentro de la Tabla 3.10, se encuentra un resumen de las características más importantes de la implementación del prototipo, y los requisitos funcionales y no funcionales abordados por éste. En este prototipo se crea otra escena, siguiendo un proceso similar al que permitió crear la primera, sin embargo, se desarrolló dentro de MOTIONS, y no como un proyecto aparte que luego se debe integrar. Esto, por una parte, sirve de prueba que no existen problemas respecto a la modularidad de la pieza de software, lo que se refleja en que su implementación no requiere mucho esfuerzo, salvo el de diseñar la estructura que sostendrá, de forma virtual, al participante, y, por otra parte, permite tener más variedad en los ambientes de experimentación.

Tabla 3.10: Datos de implementación para el séptimo prototipo.

Fuente: Elaboración propia, 2017.

ID Prototipo	P007
Nombre Prototipo	Nueva escena y mejoras
Objetivos	Añadir a MOTIONS, una nueva escena donde se pueda realizar una simulación y mejorar el menú de selección de niveles de inmersión.
Descripción	En este prototipo se añade una nueva escena, sin obstruir ninguna funcionalidad actual de la herramienta, incluyendo el control de los niveles de inmersión. Además, se debe mejorar el menú de selección de niveles de inmersión, de forma que cada nivel entregue información al usuario de la configuración.
Requisitos Funcionales Abordados	RF005, RF008
Requisitos no Funcionales Abordados	RNF002, RNF003, RNF004

Como se aprecia en la Figura 3.27, la configuración de los niveles de inmersión es funcional, y logra notarse una clara diferencia entre un nivel tres de inmersión (parte superior de la Figura 3.27) y un nivel siete de inmersión (parte inferior de la Figura 3.27).

Por otra parte, se mejora el menú de configuración de niveles de inmersión que ya se había diseñado en prototipos anteriores. En la Figura 3.28 puede apreciarse un primer acercamiento a esta mejora, donde por cada nivel de inmersión, un panel muestra las características técnicas más importantes que conlleva dicho nivel. Luego de esto, se añadió la capacidad seleccionar

manualmente algunas características de los niveles de inmersión. Finalmente, se añade un *checkbox* “Boost” que permite aumentar la calidad de los recursos gráficos del máximo nivel de inmersión. De esta forma, es posible llegar a todos los niveles de inmersión con características de hardware no tan exigentes, y al mismo tiempo, si se desea, es posible llegar a un nivel un poco superior, a un costo de procesamiento mayor. La idea detrás de este nuevo panel, es que configurar niveles de inmersión puede realizarse por alguien sin conocimientos de la lógica con que *Unity3D* maneja los recursos gráficos, o por alguien que este familiarizado con estos conceptos y que desee una configuración personalizada.

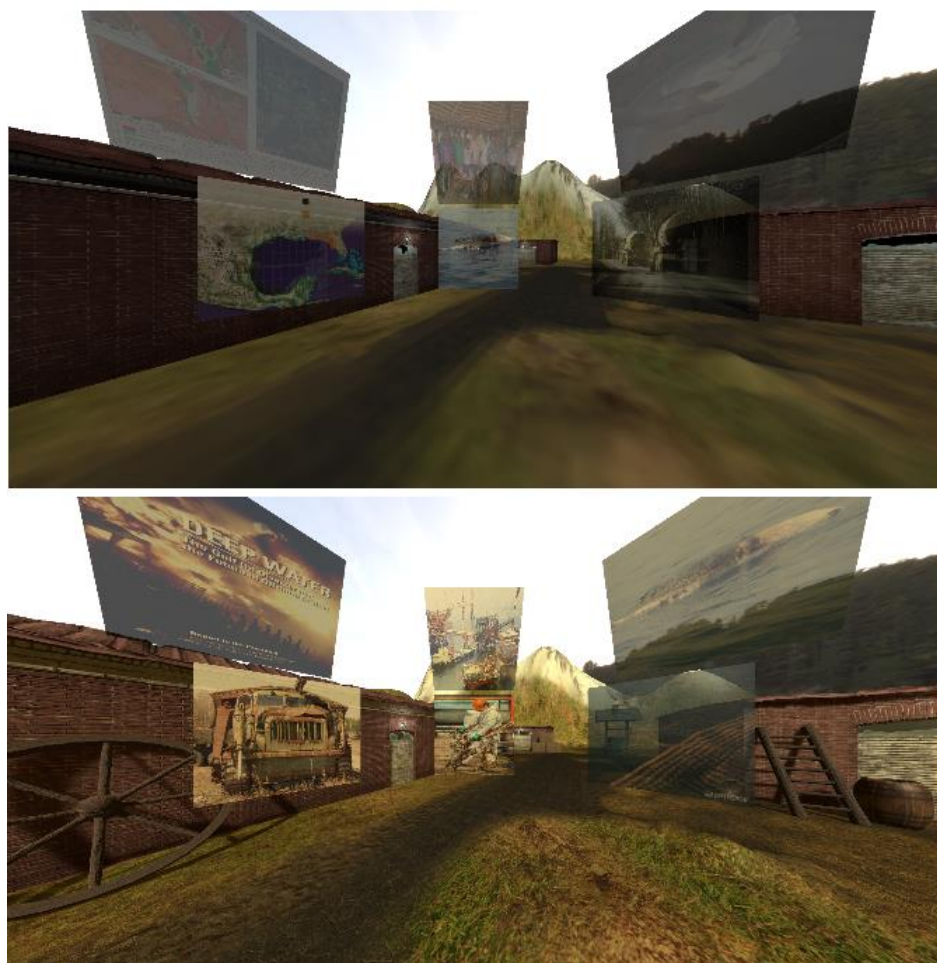


Figura 3.27: Comparación entre niveles de inmersión de la nueva escena.

En la parte superior de la imagen se muestra una escena configurada en el tercer nivel de inmersión visual. En la parte inferior se muestra una escena configurada en el séptimo nivel de inmersión visual.

Fuente: Elaboración propia, 2017.

Visual Immersion <input type="range"/> Low Select the scene Farm Scene ▼	- Skybox available. - Structure holds the participant. - Hard Shadows Only. - Low Resolution Shadows. - Shadow Projection set to Stable Fit. - Shadow distance set to 20. - Full resolution Textures. - Anisotropic Textures per Texture. - Anti Aliasing disabled.
Auditive Immersion <input type="range"/> Min	- No auditive feedback from the simulation.

Figura 3.28: Nueva información en el panel de configuración de inmersión.

Fuente: Elaboración propia, 2017.

En resumen, se cumplen los requisitos funcionales RF005 y RF007, ya que se implementó dos escenas inmersivas completas, y se añadió información respectiva a cada nivel de inmersión para ser vista por el usuario.

3.3 RESUMEN

En este Capítulo se logró completar un levantamiento de requisitos funcionales y no funcionales, en base a los antecedentes y necesidades de la herramienta MOTIONS, el cual puede revisarse en las Tablas 3.1 y 3.2. A lo largo del Capítulo se muestra también, el proceso de prototipado, que permite completar requisitos funcionales y no funcionales, y a la vez, generar nuevos requisitos a partir de necesidades que surgen en el proceso. En la Tabla 3.11 y 3.12 puede verse el cumplimiento de los distintos requisitos funcionales y no funcionales respectivamente, por cada prototipo.

Tabla 3.11: Cumplimiento de requisitos funcionales por prototipo.

Fuente: Elaboración propia, 2017.

	P01	P02	P03	P04	P05	P06	P07
RF01	X	X					
RF02			X				
RF03		X	X				
RF04	X	X	X				
RF05							X
RF06		X	X				
RF07	X						
RF08							X
RF09		X					
RF10					X		
RF11						X	
RF12						X	
RF13						X	
RF14						X	
RF15						X	

Tabla 3.12: Cumplimiento de requisitos no funcionales por prototipo.

Fuente: Elaboración propia, 2017.

	P01	P02	P03	P04	P05	P06	P07
RNF01					X	X	
RNF02				X	X	X	X
RNF03	X	X	X	X	X	X	X
RNF04		X	X	X		X	X

CAPÍTULO 4. DISEÑO E IMPLEMENTACIÓN

En este Capítulo se presenta a nivel general la arquitectura del sistema MOTIONS en general, y de los módulos de software creados para extender sus funcionalidades. Además, se presentan las clases que articulan el funcionamiento de estos nuevos módulos, y como es su flujo de ejecución. Al final de esta sección, se presentan las decisiones de diseño y sus motivaciones.

4.1 ARQUITECTURA GENERAL

La implementación actual de MOTIONS, presenta una arquitectura modular y por niveles, lo que permite que cada parte de la herramienta funcione de forma independiente, como se puede ver en las Figuras 4.1 y 4.2 (que están divididas para facilitar su visualización, en el Anexo A se encuentra la versión completa de este diagrama) donde se muestran seis niveles, siendo el nivel uno el de más alto y el nivel seis el más bajo. Los módulos de más baja jerarquía son los que presentan funcionalidades específicas para cada comportamiento, a medida que se sube de jerarquía, los módulos poseen características generales y, a la vez independientes de los módulos de nivel inferior. De esta forma, se facilita el proceso de añadir nuevos módulos, ya que se no se interrumpe el funcionamiento de los componentes adyacentes ni superiores (Pollmann, 2017). Basándose en este diseño, se agrega, en el nivel cuatro, el módulo *InmersiónManager*, que es el encargado de manejar la lógica que rige a los grados de inmersión. En el nivel cinco, dentro del *InterfaceManager* se añade el módulo *PhysiologicalManager*, que se encarga de dar la orden para inicializar las interfaces de recepción de señales fisiológicas y servir la información desde éstas hacia los niveles superiores. En el nivel seis, dentro del módulo *PhysiologicalManager*, se agrega el módulo *BITalino*, que es el que finalmente inicializa los servicios de una placa *BITalino* y extrae los datos recibidos por los sensores para exponerlos al resto de los módulos. En la Figura 4.3 se puede ver la adición de estos módulos al diagrama presentado en las Figuras 4.1 y 4.2.

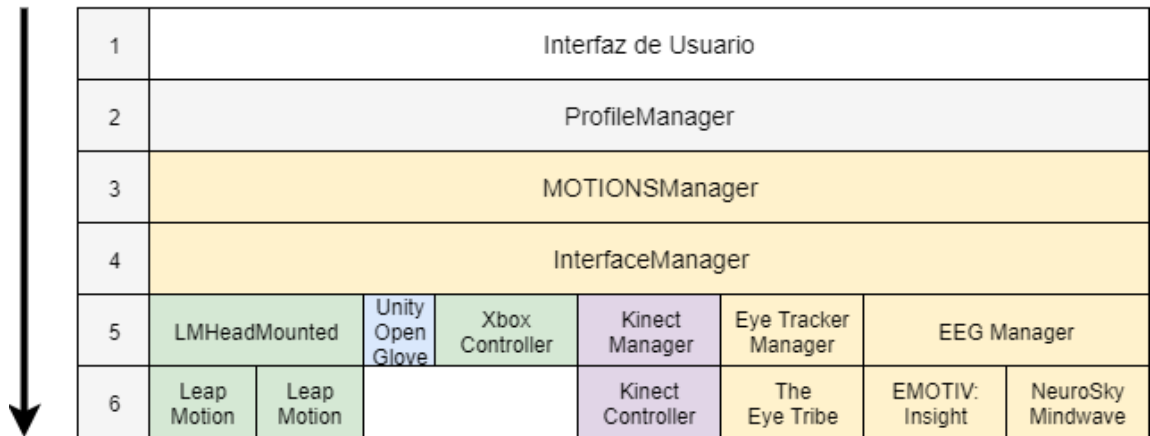


Figura 4.1: Diagrama arquitectural de MOTIONS, parte 1.

Fuente: Angus Pollmann (2017).

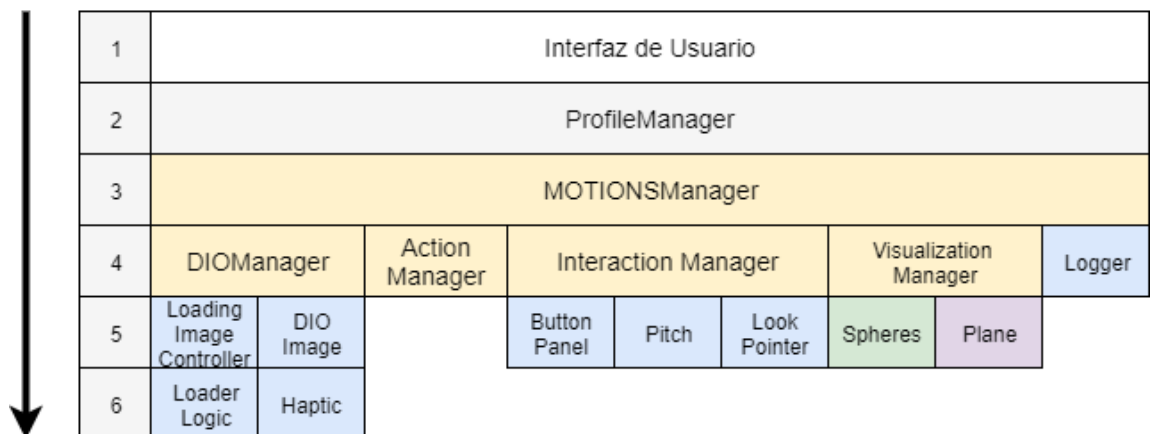


Figura 4.2: Diagrama arquitectural de MOTIONS, parte 2²⁴.

Fuente: Angus Pollmann (2017).

²⁴ DIO como abreviatura de “*Digital Information Object*”

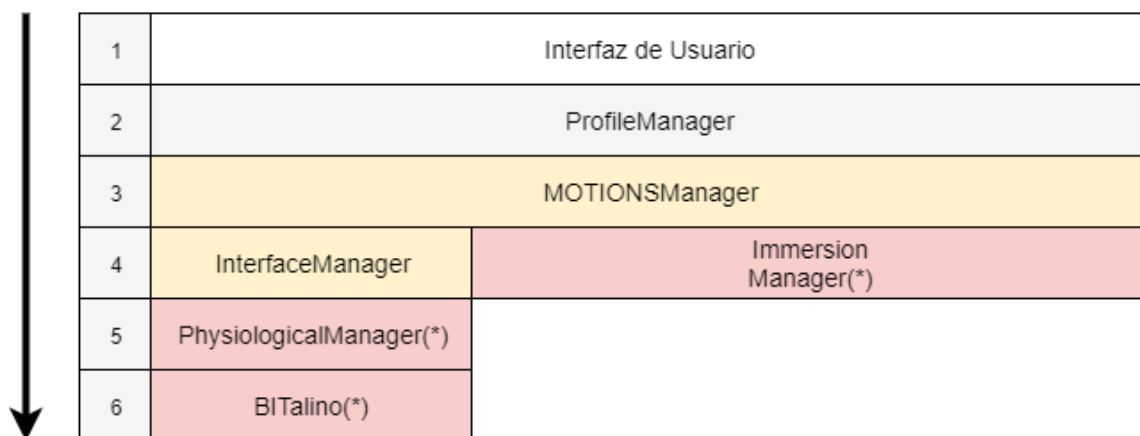


Figura 4.3: Extensión al diagrama arquitectural de MOTIONS.

(*) Módulos creados en este proyecto.

Fuente: Adaptado de Pollmann (2017).

4.2 ESTRUCTURA

En esta sección se explica cómo se produce la interacción entre los módulos de la arquitectura y una explicación de las clases más importantes que articulan su funcionamiento.

4.2.1 Clases

Dentro de la Tabla 4.1, se pueden apreciar las clases utilizadas, para el control de niveles de inmersión y para la conexión con *BITalino*, además de una descripción y el módulo al que pertenece. Cada clase es un script en C# utilizado dentro de *Unity3D*.

4.3 COMPORTAMIENTO

En esta sección se explica cómo se relacionan los módulos y sus extensiones, considerando los servicios que dichas extensiones comprenden. Además, se explica cómo funciona la asociación de respuestas fisiológicas con acciones de MOTIONS a través del módulo de asociación de acciones. Por otra parte, se da una explicación del proceso de guardado de preferencias por usuario.

4.3.1 Secuencia

Cada módulo comprende una funcionalidad que puede ser variada según las intenciones del investigador que desea realizar la evaluación. Estas funcionalidades son independientes, pero a

la vez se relacionan a través de paso de peticiones y respuestas, como se puede apreciar en la Figura 4.4. El modelo facilita la integración de nuevos módulos y características dentro de los módulos ya existentes, como, por ejemplo, una nueva interfaz.

Tabla 4.1: Descripción de clases

Fuente: Elaboración propia, 2017.

Nombre	Descripción	Pertenece a
ImmersionManager	Es el encargado de manejar los niveles de inmersión visual y auditiva.	ImmersionManager
AudioPreSettings	Configura aspectos de la inmersión auditiva que, por limitaciones de <i>Unity3D</i> , deben ser definidos antes de iniciar la escena de evaluación.	MOTIONSManager
ImmersionCavas-Controller	Controla la lógica del menú de configuración de niveles de inmersión. Se encarga de actualizar los valores mostrados por pantalla, y de almacenar la configuración elegida, para su posterior uso.	ImmersionManager
Physiological-Manager	Encargado de manejar la inicialización de las extensiones de dispositivos de recepción de señales fisiológicas. Además, expone, al resto de los módulos, los datos recibidos por las extensiones.	Physiological-Manager
BITalinoCtrl	Controla la conexión con la interfaz <i>BITalino</i> , inicializando los servicios de <i>BITalinoReader</i> , <i>BITalinoSerialPort</i> y <i>ManagerBITalino</i>	Extensión <i>BITalino</i>
BITalinoReader	Encargado de dar la instrucción a <i>ManagerBITalino</i> para iniciar una conexión con la placa <i>BITalino</i> , además, en esta clase se leen los datos recibidos en <i>ManagerBITalino</i> y se realiza el guardado de los datos recibidos en un log.	Extensión <i>BITalino</i>
ManagerBITalino	Encargado de comenzar la adquisición de datos desde los sensores de la placa <i>BITalino</i> . Permite además escribir un log sobre el estado de la conexión y de la adquisición.	Extensión <i>BITalino</i>
BITalinoSerialPort	En esta clase se configuran las preferencias de lectura de datos del <i>BITalino</i> , tales como el puerto de conexión, la frecuencia de muestreo y el tamaño del buffer, entre otras.	Extensión <i>BITalino</i>
BITalinoMapping-Loader	En esta clase se relacionan las acciones de la evaluación con las señales recibidas de la placa <i>BITalino</i> .	Módulo de asociación de acciones.
BITalinoConfig-CanvasController	Esta clase se encarga de recibir datos de configuración de la placa <i>BITalino</i> provistos por el usuario, y guardarlos para su posterior uso.	MOTIONSManager

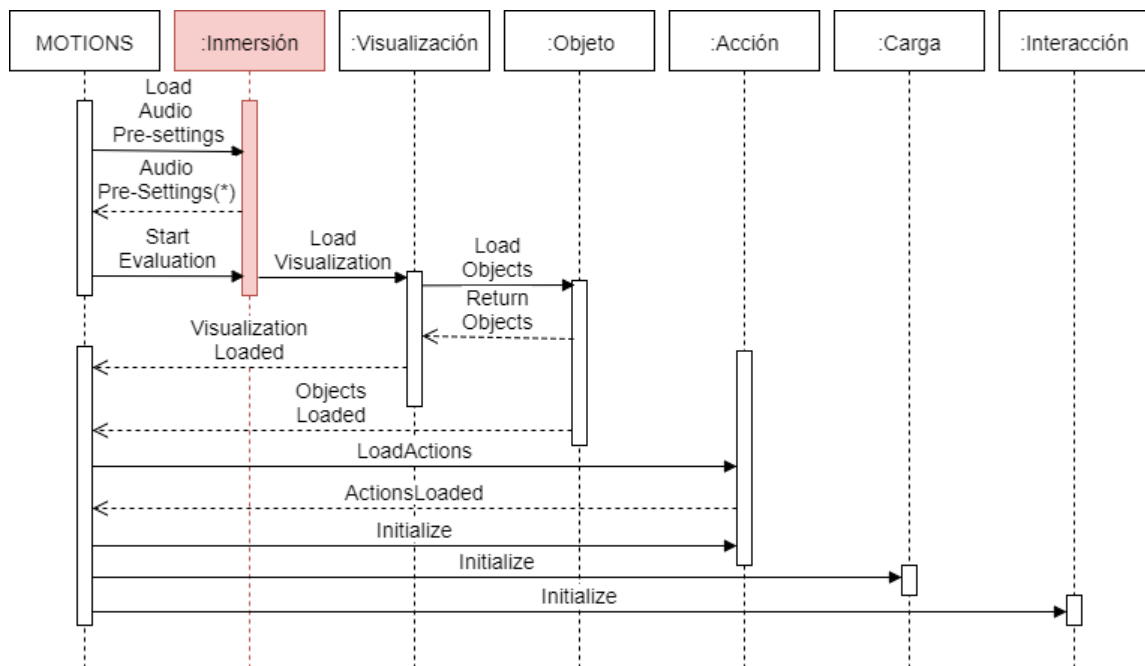


Figura 4.4: Extensión al diagrama de secuencia de módulos.

Se destaca con color rojo el módulo agregado en este proyecto.

Fuente: Adaptado de Pollmann (2017).

La Figura 4.4 está basada en el diagrama propuesto por Pollmann (2017), donde MOTIONS es el módulo encargado de iniciar la simulación. Sin embargo, en esta iteración, se añade la tarea previa de enviar una petición al módulo de inmersión, para que éste realice las configuraciones de audio previas respectivas, de acuerdo al nivel de inmersión seleccionado. Una vez realizado esto, se da paso al inicio de la simulación, donde el módulo de inmersión es el encargado de seleccionar la escena sobre la cual tendrá lugar la evaluación, y establecer el nivel de inmersión de la misma, basado en la cantidad y calidad de recursos gráficos y auditivos, además, basado en la visualización escogida, se posiciona al usuario en alguna parte de la escena, esto con el objetivo de no entorpecer la tarea de experimentación con recursos gráficos que la obstruyan. Luego, el módulo de inmersión da paso al módulo de visualización, el cual, en base a la visualización elegida, genera las figuras geométricas virtuales de acuerdo a las cuales se ordenan las imágenes, las que pueden ser esferas o planos. Posteriormente, se deben cargar los objetos digitales de información en estas figuras, objetos que son generados y entregados por el módulo de objetos, o *DIOManager*.

Como las acciones del sistema son dependientes de la visualización y de los objetos digitales de información, se espera a que los módulos de visualización y de objeto terminen y retornen su respuesta para cargar las acciones. Las acciones, como, por ejemplo, “hacer fuerza” (detectado con la toma de EMG de la placa *BITalino*) se guardan en un índice de acciones a través del

ActionManager, de esta forma cualquier interfaz puede ser relacionada a cualquier acción basándose en este índice, lo que se realiza en el módulo de carga y que, finalmente, permite la interacción del usuario con los objetos digitales de información a través de interfaces.

Este flujo es útil para comprender la modularidad del sistema, además, esclarece la secuencia que sigue cualquier interfaz, incluyendo la placa *BITalino* la cual se añade en este proyecto. Por otra parte, se da una visión general del funcionamiento del módulo de inmersión.

4.3.2 Inmersión

En la Figura 4.5, puede verse el flujo desde el inicio del programa MOTIONS, hasta que se establecen las configuraciones de niveles de inmersión en la escena.

Cuando inicia MOTIONS, el módulo del mismo nombre, permite establecer los niveles de inmersión visual y auditiva, y la escena donde se desea iniciar una evaluación (todo esto realizado dentro del panel de configuración de inmersión). Si el usuario no desea realizar configuraciones, se cargarán valores predeterminados en su perfil, que son inmersión visual mínima, inmersión auditiva mínima, y la primera escena disponible, a menos que el perfil que se está utilizando posea alguna configuración previa. Una vez obtenida la configuración de inmersión deseada, dentro del mismo módulo de MOTIONS se realizan las configuraciones previas de audio, dejando establecidos valores como frecuencia de muestreo y el tipo de reproducción a utilizar (*Stereo* o *Mono*), los que, por limitaciones de *Unity3D* no pueden ser cargados en tiempo de ejecución dentro de la escena donde se desea observar estos cambios.

Cuando se inicia la evaluación, el *ImmersionManager* obtiene los datos del perfil del usuario referentes a la inmersión. En base a las configuraciones obtenidas, primero se establece las configuraciones de calidad (*QualitySettings*), las que definen, entre otras cosas, aspectos de calidad en cuanto a texturas y sombras. Una vez definida la calidad, se pueden configurar las luces ambientales (*AmbientLight*), que corresponden a luces que se aplican de forma general a la escena, independiente de las propiedades físicas de los objetos que la reciben, lo que da una sensación más realista respecto a cómo funciona la obstrucción de luz que produce un objeto sobre otro. Dentro de esta configuración se elige si se desea o no usar luces ambientales, su intensidad, y su color. Posteriormente, se define el camino de renderizado (*RenderingPath*), que define de qué forma son renderizados los objetos en la escena, y como se aplica la luz sobre estos. Finalmente se cargan los recursos de inmersión, primero visual y luego auditiva, los que típicamente son *GameObjects* que añaden contexto a la escena. En el caso de los recursos de inmersión visual, suelen ser estructuras, objetos, fuentes de luz y *ReflectionProbes*. Para la inmersión auditiva, estos recursos son fuentes de sonido, las que pueden ser ambientales o estar relacionadas con un recurso visual (por ejemplo, el ruido que produce una lámpara al

balancearse), es por esto que los recursos auditivos se cargan después que los visuales, ya que no tendría sentido oír el ruido de un objeto que no existe. Todos los recursos (visuales y auditivos) poseen un *Tag* que indica el nivel de inmersión al que pertenecen, y, por lo tanto, en el cual son cargados.

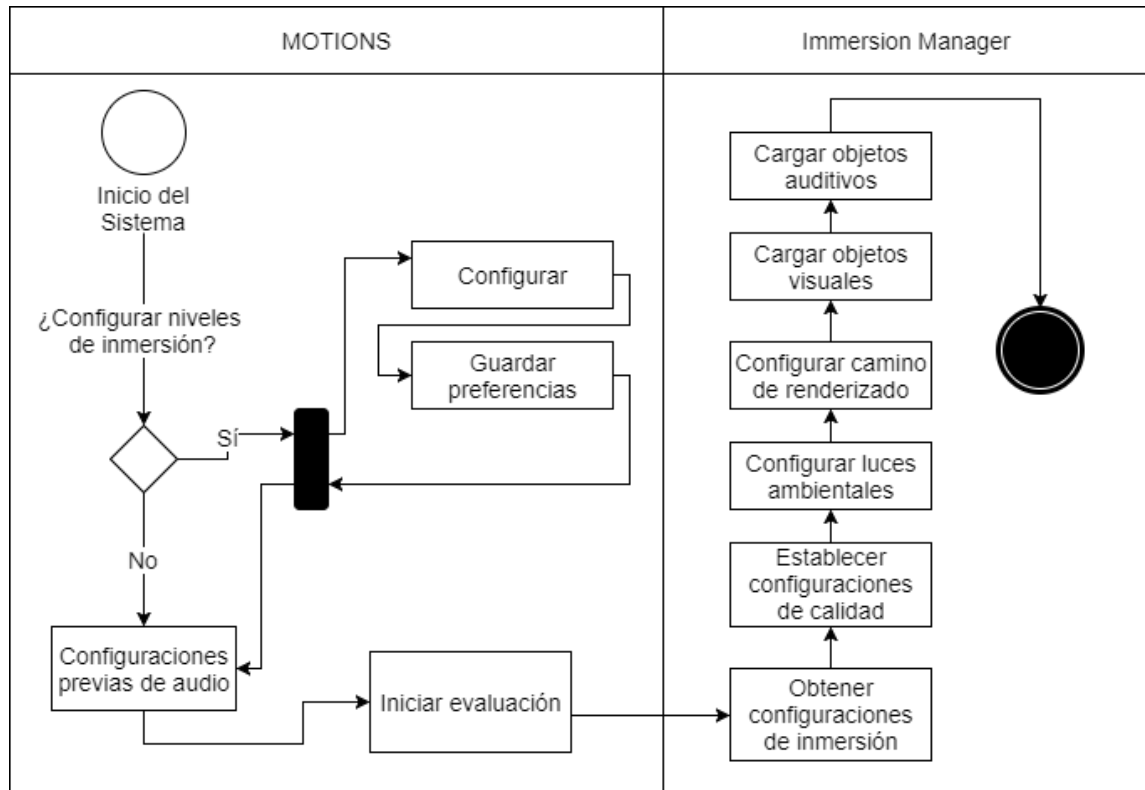


Figura 4.5: Diagrama de actividad de niveles de inmersión

Fuente: Elaboración propia, 2017.

4.3.3 Interfaz BITalino

La integración de *BITalino* en el sistema sigue el mismo flujo que las interfaces ya presentes en la aplicación, por esta razón, el diagrama de actividad que refleja este flujo está estrechamente relacionado con los diagramas de actividad propuestos por Pollmann (2017) para las interfaces cerebrales y de rastreo ocular, como se puede apreciar en la Figura 4.6. Para comenzar a utilizar la interfaz *BITalino*, se debe elegir utilizar señales fisiológicas, luego, el módulo *PhysiologicalManager* (que aparece en la Figura 4.6 como “Modulo Physiological”) inicializa el dispositivo fisiológico deseado. Cabe destacar que, si bien la interfaz a implementar es una placa *BITalino*, el módulo se llama *PhysiologicalManager*, ya que su diseño permite añadir, de forma simple, nuevas interfaces de recepción de señales fisiológicas. Para poder realizar esta inicialización, el módulo de Extensión *BITalino* llama a un intermediario encargado de establecer

la conexión con el dispositivo y recibir datos. Posteriormente, la clase *ManagerBITalino* comienza un ciclo de recepción y envío de los datos. El intermediario, al recibir los datos, los envía a la extensión *BITalino*, la cual, valida los datos, y los entrega al módulo *PhysiologicalManager*, donde son asignados a variables y expuestos al sistema, de forma que los módulos superiores puedan utilizar estos datos. Finalmente, en el módulo MOTIONS, se almacenan los datos capturados en un archivo de salida, para ser utilizados como logs.

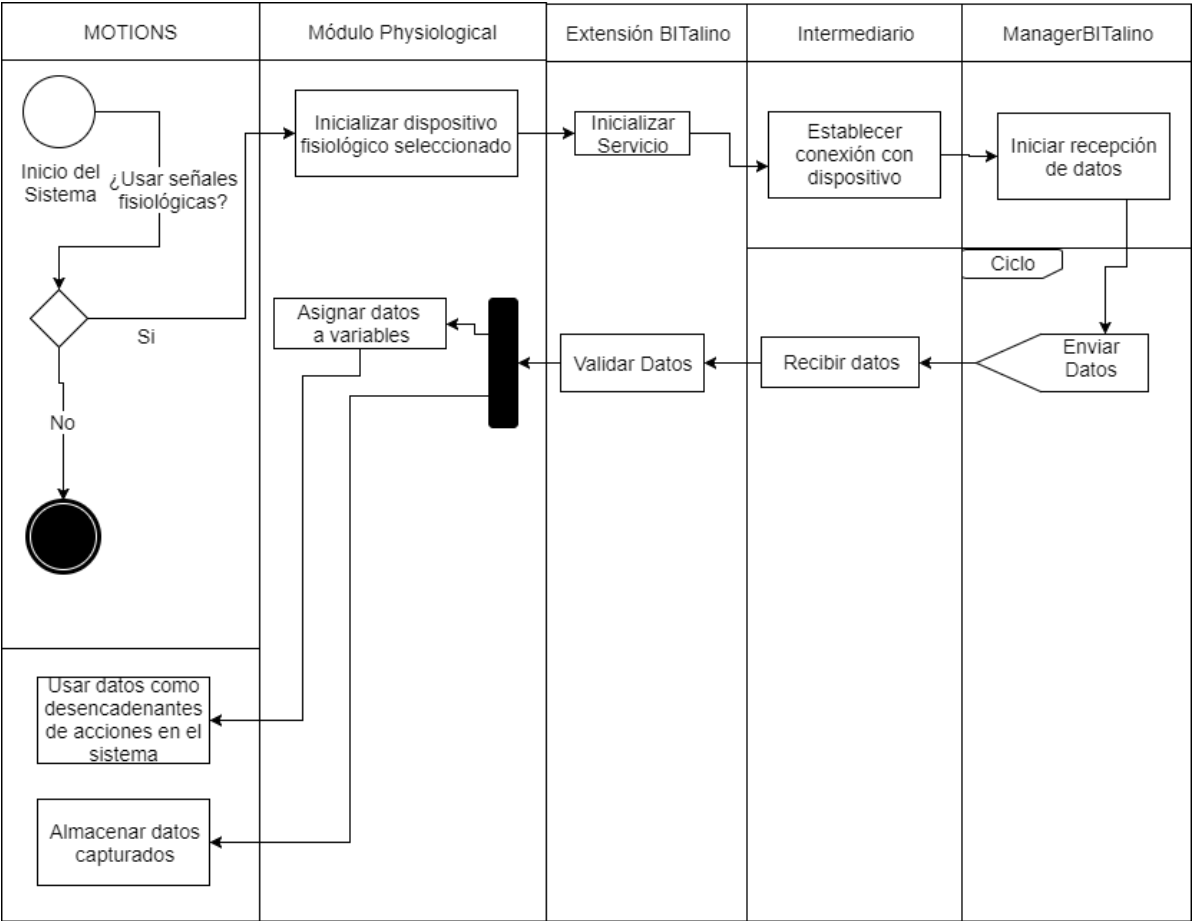


Figura 4.6: Diagrama de actividad de interfaces fisiológicas.

Fuente: Elaboración propia, 2017.

4.3.4 Clases Controller y Loader

La arquitectura de MOTIONS elaborada por Gaete (2016), y después refinada por Pollmann (2017) propone el uso de clases *Controller* y *Loader*, para conectar una interfaz a las configuraciones de evaluación del usuario y para asignar acciones a las interfaces, respectivamente.

Una clase Controlador se encarga de establecer una conexión entre los datos provistos por el Perfil del usuario y la interfaz visual de configuración de una interfaz. Para la interfaz *BITalino* el flujo es el mismo. En la Figura 4.7 puede apreciarse este intercambio, donde además de obtener el perfil, se debe obtener las acciones disponibles (referenciadas por su índice). Una vez obtenidas las acciones, se actualiza el índice de acciones en la interfaz visual. Cuando el usuario realiza una asociación entre interfaz y acción, este controlador es el encargado de guardar la relación en el perfil del usuario actual. Una vez se inicia una simulación, el controlador deja de ser necesario. Luego, las clases *Loader* recopilan los datos desde el Perfil del usuario, y asignan las acciones a la interfaz correspondiente (existe un *Loader* por interfaz) a través del módulo de asociación, como se aprecia en la Figura 4.8.

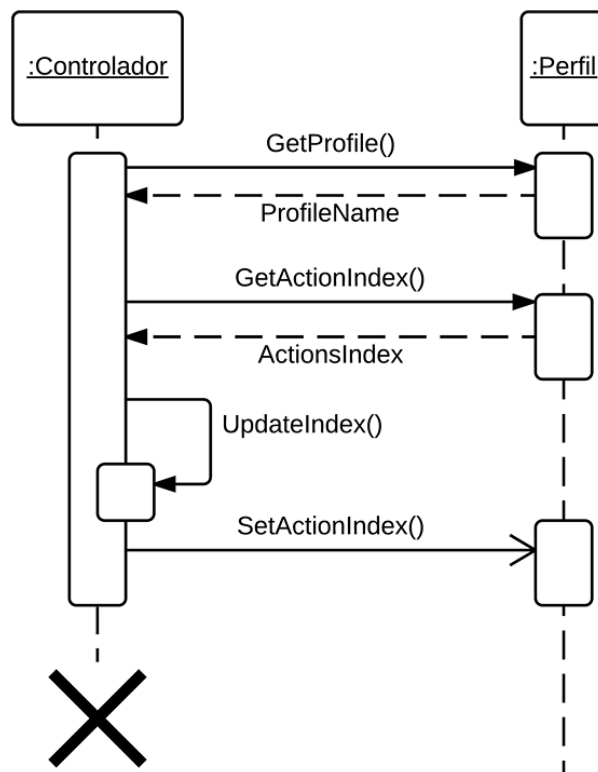


Figura 4.7: Diagrama de secuencia de las clases *Controller* con acciones

Fuente: Angus Pollmann (2017).

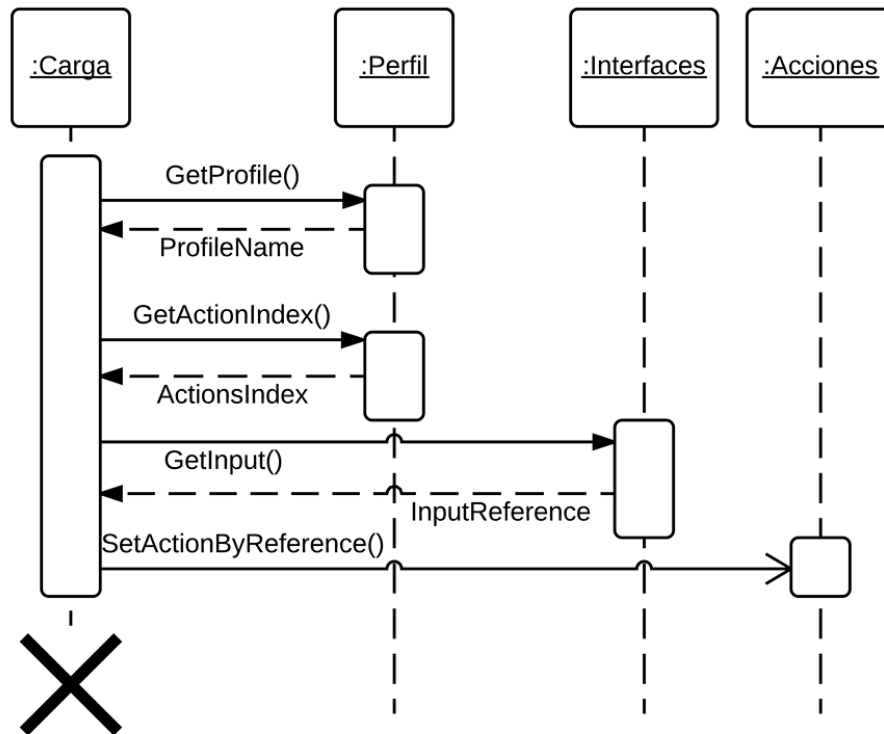


Figura 4.8: Diagrama de secuencia de las clases *Loader* con acciones

Fuente: Angus Pollmann, 2017.

4.4 ASPECTOS DE IMPLEMENTACIÓN

En esta sección se discuten decisiones de diseño e implementación, especificando las razones que las motivaron.

4.4.1 Recursos de inmersión en una escena

Como se menciona en la sección 3.1.2, para poder medir los niveles de inmersión se utilizan dos métricas, cantidad de recursos y calidad de recursos de inmersión. Para efectos de la cantidad de recursos de inmersión, un enfoque podría haber sido generar los objetos a partir de *scripts* de C#, creando únicamente los elementos que sean pertenecientes al nivel de inmersión elegido por el usuario. Generar recursos visuales y auditivos a través de *scripts* puede ser una tarea que utiliza mucho tiempo, considerando que deben programarse aspectos como posición relativa, vértices, texturas, materiales, *shaders*, recurso de audio, volumen, tipo de desvanecimiento, entre otros. Para solucionar este problema, *Unity3D* posee herramientas que permiten realizar esta tarea de forma mucho más sencilla, y sin perder eficiencia.

En *Unity3D*, todos los recursos lógicos, de iluminación, de posición, visuales, auditivos, interactivos y de UI, entre otros, deben estar contenidos en un *GameObject* en forma de *Componentes*. Los *GameObject* por su parte, existen únicamente dentro del contexto de una Escena, donde pueden estar activos, y por lo tanto, usables dentro de la Escena, o inactivos, lo que inutiliza todos los Componentes que éste comprende. Cada *GameObject* tiene de forma nativa la opción de poseer marcadores o *Tags*. En base a esto, para agregar los recursos estrictamente necesarios a la escena, basta con crear los objetos utilizando la interfaz visual de *Unity3D*, marcar con un *Tag* los recursos de inmersión a utilizar donde se especifique el nivel y tipo de inmersión al que pertenece, y dejar inactivos (a través de un *script* que realice una iteración) todos los que no pertenecen al nivel de inmersión en que se está realizando la simulación.

En esta implementación, para los *Tags* se utilizó el formato “*V_Immersion_X*” y “*A_Immersion_X*”, donde *V* representa recursos de inmersión visual, y *A*, recursos de inmersión auditiva y *X* el nivel al que pertenece. Un aspecto útil de la desactivación de *GameObjects* es que cuando un objeto está inactivo, inhabilita también a todos sus hijos, además, al desactivar sus componentes, su uso de recursos de hardware es casi nulo. Esta característica es importante debido a que, en algunos casos, los objetos de inmersión auditiva poseen una relación de dependencia con los de inmersión visual, es decir, si no se encuentra el recurso visual, tampoco debería encontrarse el auditivo, por lo tanto, basta con realizar la desactivación de los objetos de inmersión visual antes que la de los objetos de inmersión auditiva para que estos últimos queden inactivos, independiente de si la simulación está configurada en el nivel en que deberían estar activos.

4.4.2 Inmersión y hardware

Para lograr los distintos niveles de inmersión visual, es importante configurar la calidad de los recursos visuales en la escena. Esto se hace a través de la elección de un *RenderingPath*, y configurando los *QualitySettings*. Dentro del *RenderingPath* existen tres opciones, que son *Legacy Vertex Lit*, *Forward Path* y *Deferred Path*. Si una de estas opciones no fuera compatible con el hardware donde se está ejecutando la herramienta, el sistema automáticamente cambia a la siguiente con menor exigencia. Sin embargo, puede darse la situación de que el hardware que está siendo utilizado, si soporte el *Rendering Path* elegido, pero que su utilización implique una menor cantidad de *frames* por segundo (FPS). Por otra parte, los *QualitySettings* poseen una cantidad variada de opciones, que poseen rangos de exigencia de hardware mayores mientras más calidad visual otorguen. Esto también podría reflejarse en una caída en los FPS. Considerando que el objetivo de los niveles de inmersión más altos es hacer sentir al usuario que es parte de la escena donde está situado, es contradictorio experimentar detenciones provocadas por el trabajo de procesamiento, que finalmente obstruyen esta inmersión. La decisión de diseño

tomada es, configurar el nivel máximo de inmersión, no con los valores máximos de calidad visual que ofrece *Unity3D*, sino que un poco menores, esperando poder ofrecer todos los niveles de inmersión posibles a la mayor cantidad de usuarios, pero ofreciendo al usuario la capacidad de configurar de forma manual cada aspecto de inmersión visual de forma que el usuario tiene la posibilidad de llegar a todas las combinaciones posibles. Además, se añade un botón de “*Boost*” o mejora, que permite al usuario llegar a los valores máximos de calidad visual que ofrece *Unity3D* (esto únicamente para el último nivel de inmersión).

4.4.3 Asociación de acciones e interacciones

La arquitectura de MOTIONS está pensada para que la cantidad de interfaces sea creciente, y que otorgue libertad al investigador para elegir que interfaz utilizar, que acción relacionar y bajo que parámetros desea gatillar una acción. Es por esto que el sistema posee un módulo de asociación de acciones (Figura 4.9), el cual presenta un arreglo con las acciones propias de las visualizaciones y propias del objeto digital de información. Cabe destacar que se utiliza el nombre “objeto digital de información”, en lugar de imagen, debido a que el primero es más genérico y, por lo tanto, se adapta mejor a la idea de una herramienta extensible, donde en el futuro podrían existir tareas de experimentación con distintos objetos digitales de información, como, por ejemplo, videos o documentos. Las acciones disponibles se forman haciendo un nuevo arreglo, que solo contiene las acciones que están asociadas a alguna interfaz (Pollmann, 2017). Para obtener la modularidad deseada, debe existir un Controlador por interfaz, con sus entradas y configuraciones, el cual almacena la relación entre cada entrada y la acción asignada. Esta relación se guarda en el perfil del usuario, si cambia alguno de los parámetros, el registro original no se modifica, solo se crea uno nuevo, y el anterior queda obsoleto (aunque podría volver a utilizarse si se cumplieran nuevamente las condiciones bajo las cuales se creó). Posteriormente, como los controladores son eliminados al cambiar de escena, son los *Loader* los encargados de asociar las acciones a las interfaces en la evaluación. Todas las acciones asociadas a interfaces son ejecutadas en cada segundo de la evaluación, sin embargo, solo tienen un efecto real las que cumplen con la condición que se evalúa sobre la entrada del dispositivo, como un umbral, o un rango. Por ejemplo, si para seleccionar una imagen dentro de la evaluación se desea utilizar el sensor de EMG de la interfaz *BITalino*, se debe configurar valor mínimo de activación (umbral), de modo que la acción solo se realice en el instante en que la medición obtenida por el sensor sea igual o superior a dicho valor.

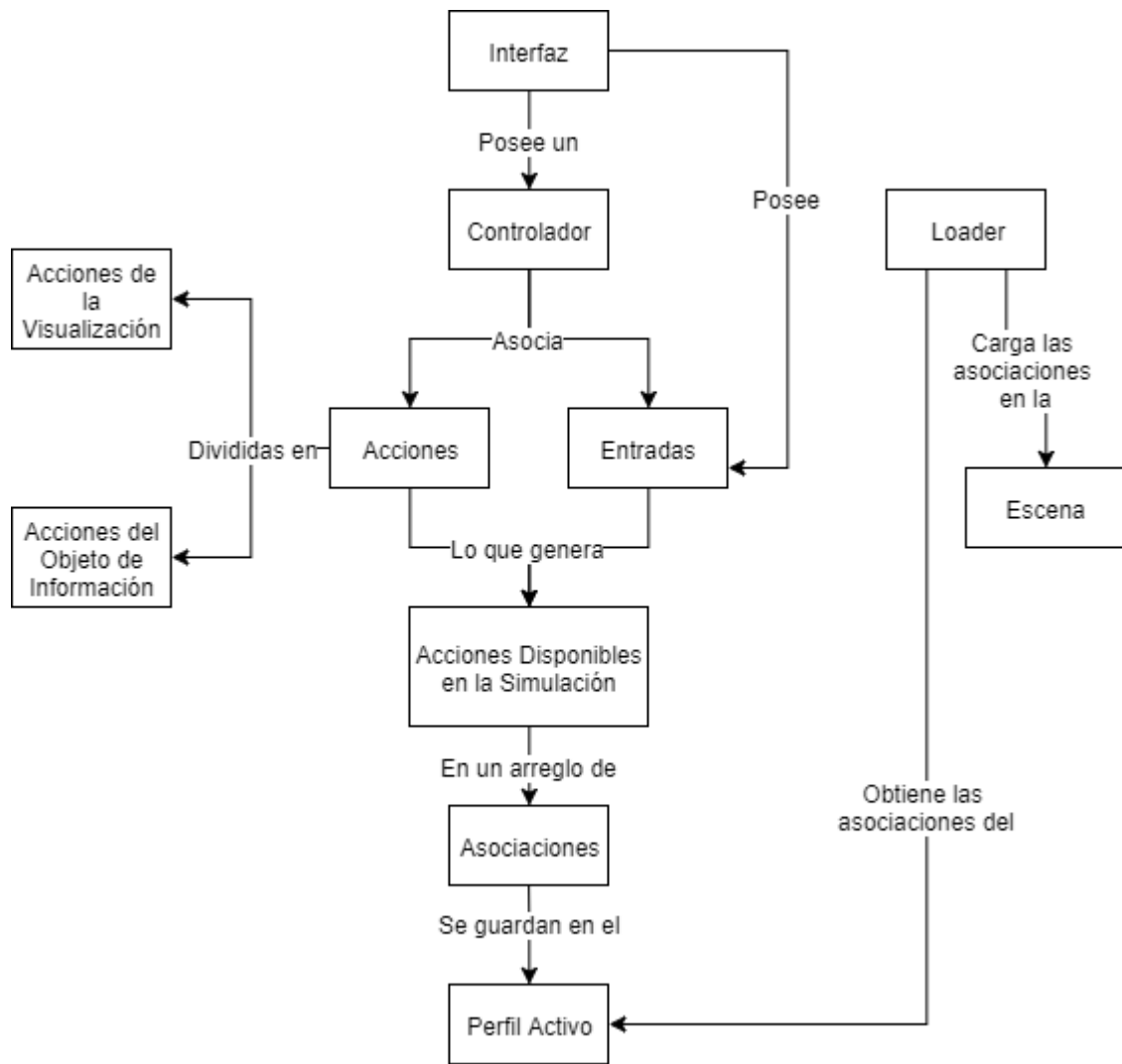


Figura 4.9: Módulo de asociación de acciones en un esquema conceptual.

Fuente: Elaboración propia, 2017.

4.5 RESUMEN

En este Capítulo se expone la arquitectura de MOTIONS, y las adaptaciones que se realizaron sobre ésta para soportar el módulo de control de niveles de inmersión y la recepción de señales fisiológicas a través de una placa *BITalino*. Esta arquitectura puede verse en las Figuras 4.1, 4.2 y 4.3, además de una breve explicación de las decisiones tomadas por su desarrollador original para llegar a ese diseño, y cómo los nuevos módulos y funcionalidades se rigen bajo estas decisiones también, con el objetivo de mantener la modularidad. Posteriormente, se muestra un diagrama de secuencia donde se explica el flujo de comunicación que poseen los módulos. A continuación, se presenta en la Tabla 4.1, las clases más importantes que permiten la

implementación de los nuevos módulos, junto con una breve descripción y el módulo al que pertenecen. Posteriormente se explica, a través del uso de diagramas de actividad, el funcionamiento del módulo de inmersión, y la conexión con *BITalino*, los que aparecen en las Figuras 4.5 y 4.6, respectivamente. Luego, a través de los diagramas de las Figuras 4.7 y 4.8, se muestra el funcionamiento general de las clases *Controller* y *Loader*. Finalmente se explican las decisiones de diseño para la carga de recursos de inmersión en una escena, para configurar la calidad de imagen teniendo en cuenta las capacidades de los distintos hardware, y para asociar acciones e interfaces.

CAPÍTULO 5. EVALUACIÓN

En este Capítulo se evalúan las mejoras realizadas sobre MOTIONS. Se explica el contexto de estas evaluaciones, y los resultados obtenidos de ellas.

5.1 CARACTERÍSTICAS GENERALES DE LA EVALUACIÓN

Dado que este trabajo se enfoca netamente en el desarrollo de una extensión de funcionalidades a un software existente, las pruebas que se realizan apuntan a la verificación de solución en si misma, con el objetivo de medir el cumplimiento de los requisitos funcionales y no funcionales planteados, además de evaluar la correcta integración con la plataforma original. Esta evaluación considera el módulo de inmersión, el cual se divide en inmersión visual y auditiva, y el módulo de recepción de señales fisiológicas.

5.2 PRUEBAS DE CUMPLIMIENTO DE REQUISITOS

Las pruebas realizadas en esta sección tienen como objetivo comprobar el cumplimiento de los requisitos funcionales y no funcionales planteados en la subsección 3.1.5. Las pruebas se dividen en casos de prueba, los que se definen como flujos de tareas a realizar por un participante dentro de MOTIONS (en Anexo B puede verse un caso de prueba en detalle). Cada caso de prueba posee un identificador, objetivos que se espera cumplir al realizar la tarea, y observaciones, en cuanto a problemas de usabilidad y errores detectados por el participante al desarrollar la tarea.

El primer grupo de casos de prueba evalúa las funcionalidades originales de MOTIONS, donde no se utilizan los nuevos módulos desarrollados, con el objetivo de verificar que dichas funcionalidades no han sido afectadas negativamente. Para no utilizar los nuevos módulos, los niveles de inmersión visual y auditiva se dejan configurados en su nivel mínimo (en el panel de configuración de inmersión), y la interfaz *BITalino* se desactiva (en el panel de configuración de hardware), todo esto, previo a la realización de cada caso de prueba. A continuación, se lista una breve explicación de cada caso de prueba:

1. **CP_MOTIONS_01:** La tarea consiste en cambiar la carpeta de salida de MOTIONS (donde se guardarán finalmente los logs). El evaluador debe abrir el menú “*Ouput Path*”, cambiar la carpeta y aplicar los cambios y volver al menú inicial.
2. **CP_MOTIONS_02:** La tarea consiste en configurar el directorio de los objetos digitales de información, su formato, y la cantidad de estos a utilizar. El evaluador debe abrir el menú “*Information Object*”, seleccionar allí la opción “*Images path and names*”, y cambiar el directorio de los objetos digitales de información. Luego debe escribir el prefijo con el

que están almacenados los objetos, y especificar el número. Una vez finalizado debe aplicar cambios y volver al menú inicial.

3. **CP_MOTIONS_03:** La tarea consiste en seleccionar cada opción dentro del menú de visualización. El evaluador debe entrar al panel "*Visualization*", y seleccionar la visualización planar. Luego, debe seleccionar la visualización esférica. Luego debe seleccionar la configuración de cámara y realizar algún cambio cualquiera. Posteriormente debe abrir el catálogo de las acciones disponibles para la visualización con el botón "*Actions available*". Finalmente debe volver al menú inicial y salir del programa.
4. **CP_MOTIONS_04:** La tarea consiste en la activación individual de cada una de las interfaces, y la apertura y cambio de parámetros de su panel de control en caso de poseerlo. El evaluador debe dirigirse al menú "*Hardware*". Posteriormente debe activar la primera interfaz y configurarla. Una vez realizado esto, debe volver al menú inicial e iniciar una simulación. Una vez en la simulación debe cerrar el programa y abrirlo de nuevo, repitiendo el proceso, pero con la segunda interfaz, así hasta realizarlo con la totalidad de las interfaces.
5. **CP_MOTIONS_05:** La tarea consiste en asociar acciones a teclas del teclado, e iniciar una simulación. El evaluador debe ingresar al menú "*Hardware*" y activar la interfaz "*Keyboard*", para luego volver al menú inicial. Luego debe dirigirse al menú "*Action mapping*", donde debe seleccionar el teclado (*Keyboard*) y asignar a cada acción al menos una tecla. Posteriormente debe volver al menú inicial e iniciar una simulación. Una vez en la simulación debe realizar cada una de las acciones posibles. Finalizado esto debe cerrar el programa.
6. **CP_MOTIONS_06:** La tarea consiste en configurar aspectos generales de la aplicación. El evaluador, en el menú principal, debe crear un nuevo perfil, crear una nueva evaluación y asignarse una ID con un número cualquiera. Posteriormente debe iniciar la evaluación. Una vez en la evaluación debe cerrar el programa.
7. **CP_MOTIONS_07:** La tarea consiste en iniciar una simulación con cada la visualización planar. El evaluador debe entrar al panel "*Object visualization*", y seleccionar la visualización Plana. Luego, debe volver al menú inicial e iniciar una simulación. Posteriormente debe realizar alguna acción a elección dentro de la simulación. Finalmente debe cerrar el programa.
8. **CP_MOTIONS_08:** La tarea consiste en iniciar una simulación con cada la visualización esférica. El evaluador debe entrar al panel "*Object visualization*", y seleccionar la visualización Plana. Luego, debe volver al menú inicial e iniciar una simulación. Posteriormente debe realizar alguna acción a elección dentro de la simulación. Finalmente debe cerrar el programa.

El segundo grupo de casos de prueba se enfoca en el módulo de inmersión visual, donde la tarea a realizar en cada caso de prueba es iniciar una simulación en cada uno de los niveles de inmersión visual, sin inmersión auditiva, con alguna interfaz a elección por el evaluador, primero en visualización planar y luego en visualización esférica. Para esto, en cada caso de prueba el evaluador debe ir al menú “*Inmersión*” desde el menú principal. Luego, el evaluador debe seleccionar el nivel de inmersión visual de acuerdo con el caso de prueba que está evaluando. A continuación, debe ir al menú “*Visualization*”, y seleccionar la visualización acorde al caso de prueba que está evaluando. Posteriormente debe realizar la configuración de la interfaz a utilizar. Para esto, debe activar la interfaz en el menú “*Hardware*”, y luego asignarle acciones en el menú “*Action mapping*”. Una vez realizado esto, debe volver al menú principal y comenzar una evaluación. Ya en la evaluación, debe realizar alguna acción con la interfaz configurada. Finalmente debe cerrar el programa.

El tercer grupo de casos de prueba es muy similar al segundo, la única diferencia es que, en lugar de variar el nivel de inmersión visual para cada simulación, debe variarse el nivel de inmersión auditiva. Esto también debe repetirse para ambas visualizaciones (planar y esférica).

El cuarto grupo de casos de prueba se enfoca en el módulo de recepción de señales fisiológicas. El objetivo de estas pruebas es verificar el funcionamiento de todos los requisitos que debía cumplir inicialmente el módulo. El dispositivo *BITalino* debe estar configurado y pareado en el equipo, y el evaluador debe conocer el puerto al cual está conectado. A continuación, se lista la explicación de los casos de prueba.

1. **CP_BITALINO_01:** La tarea consiste en configurar los parámetros de la interfaz *BITalino*. El evaluador, desde el menú principal, debe acceder al menú “*Hardware*”. Posteriormente debe activar el dispositivo *BITalino*, y abrir su menú de configuración. Debe seleccionar los valores recomendados y cerrar la ventana de configuración. Finalmente debe volver al menú principal y cerrar el programa.
2. **CP_BITALINO_02:** La tarea consiste en iniciar una simulación con el dispositivo *BITalino* conectado y funcionando. El evaluador debe acceder al menú “*Hardware*”. Posteriormente debe activar el dispositivo *BITalino* y configurarlo adecuadamente. Luego, debe dirigirse al menú “*Visualization*” y seleccionar el tipo de visualización planar. A continuación, debe volver al menú principal e iniciar una simulación. Finalmente debe cerrar el programa.
3. **CP_BITALINO_03:** La tarea es idéntica a la tarea del caso CP_BITALINO_02, solo que debe elegir la visualización esférica en lugar de la planar.
4. **CP_BITALINO_04, CP_BITALINO_05, CP_BITALINO_06 y CP_BITALINO_07:** Las tareas consisten en realizar acciones en una simulación a través del uso de la interfaz *BITalino*. Las tareas se diferencian en la señal fisiológica que gatillará la acción, siendo

estas EMG, EDA, ECG y ACC. El evaluador debe entrar al menú de “*Hardware*” y configurar adecuadamente el *BITalino*. Posteriormente, debe entrar al menú “*Action pairing*” y relacionar con alguna acción la interfaz *BITalino* seleccionando la señal correspondiente al caso de prueba que se está realizando. Posteriormente debe iniciar una simulación y gatillar la acción relacionada en el paso anterior. Finalmente debe cerrar el programa.

Todas las pruebas fueron realizadas por un evaluador externo, estudiante de último semestre de Ingeniería de Ejecución en Computación e Informática en la Universidad de Santiago de Chile.

5.3 RESUMEN

En este Capítulo se verifico el funcionamiento de los nuevos módulos integrados, a través de la ejecución de casos de prueba por un participante. Las pruebas realizadas se enfocaron en cuatro aspectos: Integridad del programa original, integración de niveles de inmersión visual, integración de niveles de inmersión auditiva e integración del módulo de recepción de señales fisiológicas.

Tabla 5.1: Casos de prueba: Evaluación de funcionalidades originales de MOTIONS.

(*) Esta observación también ocurría en la versión original.

Fuente: Elaboración propia, 2017.

Evaluación de funcionalidades originales de MOTIONS		
Identificador	Objetivos	Observaciones
CP_MOTIONS_01	Evaluar el panel de configuración de salidas (<i>Ouput Path</i>).	(*) Se genera más de un cuadro de dialogo al presionar el botón <i>Ouput Path</i> .
CP_MOTIONS_02	Evaluar el panel de configuración del objeto de información (<i>Information Object</i>)	(*) Se genera más de un cuadro de dialogo al presionar los botones de selección de la ruta de las imágenes y de selección de la ruta de los grupos de imágenes.
CP_MOTIONS_03	Evaluar el panel de visualización (<i>Visualization</i>)	(*) El botón “ <i>Camera configuration</i> ” realiza la misma función que “ <i>Actions available</i> ”.
CP_MOTIONS_04	Evaluar el panel de hardware (<i>Hardware</i>), incluyendo la configuración individual de las interfaces que lo requieren.	(*) El menú de configuración del <i>EMOTIV: Insight</i> escapa de los bordes de su contenedor, lo que no permite su configuración.

CP_MOTIONS_05	Evaluar el panel de relación de acciones e interfaces (<i>Action Mapping</i>)	Sin observaciones.
CP_MOTIONS_06	Evaluar el panel principal (<i>Evaluation Summary</i>)	(*) Crear un nuevo perfil e inicializar una simulación sin realizar configuraciones previas, produce el cierre del programa.
CP_MOTIONS_07	Evaluar simulación con vista planar, verificando el funcionamiento de todas las acciones disponibles.	Sin observaciones.
CP_MOTIONS_08	Evaluar simulación con vista esférica, verificando el funcionamiento de todas las acciones disponibles.	Sin observaciones.

Tabla 5.2: Casos de prueba: Evaluación de niveles de inmersión visual.

Fuente: Elaboración propia, 2017.

Evaluación de niveles de inmersión visual		
Identificador	Objetivos	Observaciones
CP_V_IMM_01	Evaluar el panel de control de niveles de inmersión (<i>Immersion</i>).	Sin observaciones.
CP_V_IMM_02	Evaluar primer nivel de inmersión visual en visualización planar.	Sin observaciones.
CP_V_IMM_03	Evaluar segundo nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_04	Evaluar tercer nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_05	Evaluar cuarto nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_06	Evaluar quinto nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_07	Evaluar sexto nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_08	Evaluar séptimo nivel de inmersión visual en una simulación con visualización planar.	Sin observaciones.
CP_V_IMM_09	Evaluar primer nivel de inmersión visual en una simulación con visualización esférica.	Sin observaciones.
CP_V_IMM_10	Evaluar segundo nivel de inmersión visual en visualización esférica.	Sin observaciones.
CP_V_IMM_11	Evaluar tercer nivel de inmersión visual en una simulación con visualización esférica.	Sin observaciones.
CP_V_IMM_12	Evaluar cuarto nivel de inmersión visual en una simulación con visualización esférica.	Sin observaciones.
CP_V_IMM_13	Evaluar quinto nivel de inmersión visual en una simulación con visualización esférica.	Sin observaciones.
CP_V_IMM_14	Evaluar sexto nivel de inmersión visual en una simulación con visualización esférica.	Sin observaciones.
CP_V_IMM_15	Evaluar séptimo nivel de inmersión visual en visualización esférica.	Sin observaciones.

Tabla 5.3: Casos de prueba: Evaluación de niveles de inmersión auditiva

Fuente: Elaboración propia, 2017.

Evaluación de niveles de inmersión auditiva		
Identificador	Objetivos	Observaciones
CP_A_IMM_01	Evaluar primer nivel de inmersión auditiva en visualización planar.	Sin observaciones.
CP_A_IMM_02	Evaluar segundo nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_03	Evaluar tercer nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_04	Evaluar cuarto nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_05	Evaluar quinto nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_06	Evaluar sexto nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_07	Evaluar séptimo nivel de inmersión auditiva en una simulación con visualización planar.	Sin observaciones.
CP_A_IMM_08	Evaluar primer nivel de inmersión auditiva en una simulación con visualización esférica.	Sin observaciones.
CP_A_IMM_09	Evaluar segundo nivel de inmersión auditiva en visualización esférica.	Sin observaciones.
CP_A_IMM_10	Evaluar tercer nivel de inmersión auditiva en una simulación con visualización esférica.	Sin observaciones.
CP_A_IMM_11	Evaluar cuarto nivel de inmersión auditiva en una simulación con visualización esférica.	Sin observaciones.
CP_A_IMM_12	Evaluar quinto nivel de inmersión auditiva en una simulación con visualización esférica.	Sin observaciones.
CP_A_IMM_13	Evaluar sexto nivel de inmersión auditiva en una simulación con visualización esférica.	Sin observaciones.
CP_A_IMM_14	Evaluar séptimo nivel de inmersión auditiva en visualización esférica.	Sin observaciones.

Tabla 5.4: Casos de prueba: Funcionalidades BITalino.

Fuente: Evaluación propia, 2017.

Funcionalidades BITalino		
Identificador	Objetivos	Observaciones
CP_BITALINO_01	Evaluar la configuración de <i>BITalino</i> en el menú de configuración de <i>Hardware</i> .	Sin observaciones.
CP_BITALINO_02	Evaluar <i>BITalino</i> en una simulación con visualización planar.	Sin observaciones.
CP_BITALINO_03	Evaluar <i>BITalino</i> en una simulación con visualización esférica.	Sin observaciones.
CP_BITALINO_04	Evaluar capacidad de realizar acciones utilizando el sensor de EMG de <i>BITalino</i> , en modo umbral y rango.	Resulta difícil lograr los cambios deseados en los valores obtenidos.
CP_BITALINO_05	Evaluar capacidad de realizar acciones utilizando el sensor de EDA de <i>BITalino</i> , en modo umbral y rango.	Resulta difícil lograr los cambios deseados en los valores obtenidos.
CP_BITALINO_06	Evaluar capacidad de realizar acciones utilizando el sensor de ECG de <i>BITalino</i> , en modo umbral y rango.	No es posible manejar a voluntad las acciones asociadas.
CP_BITALINO_07	Evaluar capacidad de realizar acciones utilizando el sensor de ACC de <i>BITalino</i> , en modo umbral y rango.	Resulta difícil lograr los cambios deseados en los valores obtenidos.

Para las observaciones generadas dentro del primero grupo de pruebas, referentes a MOTIONS en su estado original, su resolución involucra un estudio de los módulos involucrados, para identificar la causa de cada error derivado de la observación. Esto escapa del alcance de este proyecto, por lo tanto, no han sido modificadas.

El resto de las observaciones se presentaron únicamente en el módulo de recepción de señales fisiológicas. Para gatillar las acciones, la *API* de *BITalino* interpreta las señales recibidas y las transforma a valores numéricos. Estos valores son generalmente decimales entre -1 y 1, y cambian con facilidad, e incluso en estado de reposo están oscilando levemente. Esto es el origen de las observaciones de los casos de prueba CP_BITALINO_04, CP_BITALINO_05 y CP_BITALINO_07, ya que el selector de umbral o rango solo permitía seleccionar valores enteros entre -10 y 10. Se realizó una modificación que permite mayor densidad a la hora de elegir umbrales, donde ahora se permiten valores desde -5 a 5 considerando los decimales (se utiliza solo un dígito después del punto). Por otra parte, para la observación del CP_BITALINO_06, se propone realizar un detector de frecuencia de onda, y en base a esa frecuencia generar un umbral, debido a que los cambios en la amplitud de la señal ECG (valor entregado por la *API* de *BITalino*) no son representativos de cambios fisiológicos, por lo tanto, es imposible prever cuando se gatillará una acción asociada a este sensor.

CAPÍTULO 6. CONCLUSIONES

En este capítulo se presentan las conclusiones del trabajo, haciendo una revisión de los objetivos, sus alcances e implicancias. En la última parte de este Capítulo se discuten posibles trabajos futuros.

6.1 OBJETIVOS

A continuación, se presentan los objetivos, específicos y generales mencionados en la sección 1.4, y como se abordaron en el trabajo.

6.1.1 Objetivos específicos

1. Realizar un estudio del marco teórico que rodea el problema a solucionar.

A lo largo del capítulo 2, se presenta un análisis del marco teórico que rodea el problema al que se pretende dar solución, para esto se expone un marco conceptual, donde se exploran conceptos y tecnologías relacionadas con el proyecto. Se expone también el estado del arte, donde se mencionan distintos estudios relacionados a la problemática.

2. Diseñar un módulo para el control de inmersión que sea compatible con la arquitectura de MOTIONS.

En la sección 3.1, se realiza un análisis de los antecedentes que rodean el proyecto desarrollado en este trabajo, lo que permite conocer el estado de MOTIONS previo al desarrollo de los nuevos módulos, y, por lo tanto, tomar decisiones de diseño que permitan generar una solución que no interfiera en el funcionamiento de ninguna funcionalidad. En dicha sección, se da énfasis a los elementos que están estrechamente relacionados con los objetivos de este proyecto, estudiando, en la subsección 3.1.1, la historia de MOTIONS y sus características generales. Luego, en la subsección 3.1.2 se detalla qué se entiende por nivel de inmersión visual y auditivo en MOTIONS, y como era su implementación originalmente. Posteriormente, en las subsecciones 3.1.3 y 3.1.4 se estudian las interfaces y acciones en MOTIONS, y en base a esto, se listan, en la subsección 3.1.5 los requisitos funcionales y no funcionales que deberían cumplir los nuevos módulos para ser integrados en MOTIONS, sin interferir su correcto funcionamiento. En la sección 3.2 se detallan los distintos prototipos que permitieron llegar al producto final. Estos prototipos suponen por una parte la implementación, pero al mismo tiempo son parte del diseño de la solución, el cual se va refinando en cada iteración. En la sección 4.1 y 4.2 se hace un estudio de la arquitectura actual de MOTIONS,

y se presentan los aspectos a tener en consideración para añadir nuevos módulos que cumplan con las decisiones de diseño que dieron forma a la herramienta actual, con el fin de mantener su modularidad y extensibilidad.

3. Desarrollar un componente de control de nivel de inmersión visual.

A lo largo de la sección 3.2, se presentan distintos prototipos que van cumpliendo los requisitos generados en la subsección 3.1.5. La creación de estos prototipos permite el desarrollo de la componente de inmersión visual, específicamente en el primer prototipo y el segundo prototipo, presentados en las subsecciones 3.2.1 y 3.2.2, lugar en que se detalla la creación de siete niveles de control de inmersión visual, seleccionables por el usuario a través de una interfaz visual, donde cada nivel presenta una mejora en la cantidad y calidad de recursos gráficos dentro de una simulación, respecto del nivel anterior. Además, en el séptimo prototipo, explicado en la subsección 3.2.7, se mejora el menú de selección de niveles de inmersión en MOTIONS, lo que permite al usuario, de manera opcional, configurar, de forma personalizada las características que desea para cada nivel de inmersión visual. Esto permite al investigador controlar cada detalle de los niveles de inmersión visual, sin la necesidad de conocer el funcionamiento interno de la herramienta.

4. Desarrollar un componente de control de nivel de inmersión auditiva.

Al igual que para la inmersión visual, en la sección 3.2, se presentan prototipos que cumplen los requisitos generados en la subsección 3.1.5. relacionados con la inmersión auditiva. En particular, el tercer prototipo es el que permite cumplir este objetivo (subsección 3.2.3). En dicho prototipo, se ofrecen siete niveles de control de inmersión auditiva seleccionables por el usuario, donde cada nivel posee más y mejores recursos auditivos. La interfaz a utilizar para seleccionar el nivel de inmersión auditiva es la misma que para la inmersión visual, con el nuevo selector integrado. Luego, en el séptimo prototipo, explicado en la subsección 3.2.7, se mejora el menú de selección de niveles de inmersión en MOTIONS, permitiendo al investigador configurar de forma personalizada las características de cada nivel de inmersión auditivo.

5. Integrar los componentes de control de niveles de inmersión.

En la sección 3.2, de prototipado, el cuarto prototipo integra el control de niveles de inmersión en MOTIONS, como aparece en la subsección 3.2.4. Las mayores consideraciones vienen de parte de la arquitectura, tomando en cuenta que el proceso de integración de las nuevas funcionalidades podría haberse realizado de múltiples formas, es necesario invertir tiempo en

estudiar el diseño arquitectural de la herramienta, para elegir el modelo de integración que mejor se adapte al funcionamiento actual de MOTIONS, sin interferir en su correcto funcionamiento.

6. Diseñar e integrar entornos de realidad virtual.

En los prototipos P01 y P02, desarrollados en las subsecciones 3.2.1 y 3.2.2, se genera una escena inmersiva, y en el séptimo prototipo, desarrollado en la subsección 3.2.7 se genera una segunda escena inmersiva siguiendo el mismo procedimiento que para la primera. El proceso de integración de entornos de realidad virtual es simple, considerando que la pieza de software de control de niveles de inmersión es modular, basta con realizar pequeños ajustes en las escenas que se desea convertir en inmersivas, y es posible experimentar las configuraciones de estos niveles en una simulación.

7. Realizar pruebas de cumplimiento de requisitos funcionales y no funcionales.

Dentro del Capítulo 5, se definen las pruebas realizadas por un participante, para evaluar cada uno de los módulos desarrollados por separado, es decir, el módulo de control de inmersión, el cual fue dividido en pruebas para la inmersión visual y pruebas para la inmersión auditiva, y el módulo de recepción de señales fisiológicas. Además, en dicho Capítulo, se realizaron pruebas para verificar que la herramienta original no hubiese sido perturbada de forma que alguna de sus funcionalidades se viera perjudicada con la agregación de los nuevos módulos.

6.1.2 Objetivo general

El objetivo general de generar un módulo de control de niveles de inmersión para MOTIONS y agregar soporte para retroalimentación a través de interfaces que transmitan señales fisiológicas se cumplió completamente, ya que se generó de forma exitosa el módulo de control de niveles de inmersión visual y auditiva para MOTIONS, respetando su arquitectura y sin interferir en ninguna de sus funcionalidades. Por otra parte, se agregó soporte para retroalimentación a través de interfaces que transmiten señales fisiológicas, lo que queda demostrado con la implementación de la interfaz *BITalino*.

6.2 RESULTADOS OBTENIDOS

A lo largo de las pruebas realizadas, pudo verificarse que el programa original mantuvo su correcto funcionamiento luego de la agregación de los nuevos módulos. Además, cada módulo

por separado cumple con el mínimo suficiente para poder ser ejecutado sin mayores inconvenientes. Se evidenciaron problemas en cuanto a la interfaz visual de MOTIONS. Sin embargo, eran problemas que se arrastraban desde la implementación original, y no suponían una obstrucción mayor que no permitiera realizar una simulación. Para el módulo de control de niveles de inmersión, se evidenció que, al iniciar una simulación con más de 10 imágenes, dichas imágenes suponían una obstrucción visual, lo que no permitía apreciar grandes diferencias a medida que se cambiaba el nivel de inmersión visual, por otra parte, los recursos de inmersión auditiva que funcionaban con sonido en tres dimensiones, no lograron ser aprovechados totalmente, considerando que en una simulación, el usuario es posicionado en un lugar de la escena y luego no es posible moverse, a pesar de esto, los recursos auditivos de ambientación, lograron ser apreciados en su totalidad.

6.3 IMPLICACIONES Y ALCANCES

En esta sección se mencionan las implicaciones y alcances del trabajo que se elaboró.

6.3.1 Implicaciones

Este trabajo supone un aporte para el área de investigación humano-computador, en especial la interacción humano-información, ya que se añade una nueva variable de estudio, como es la inmersión, a una herramienta que ya es versátil por el hecho de presentar múltiples interfaces de interacción que se pueden utilizar de forma simultánea o individual, lo que amplía la cantidad de estudios que con ella se pueden realizar. Por ejemplo, se pueden hacer pruebas desde niveles bajos de inmersión, avanzando hasta los niveles más altos, y verificar en cada caso el cumplimiento de las hipótesis planteadas. Además, se añade un nuevo tipo de interfaz, que son los dispositivos de recepción de señales fisiológicas, permitiendo investigar, en ambientes inmersivos, las reacciones fisiológicas de un participante, e incluso, realizar acciones en base a cambios observados en éstas.

6.3.2 Alcances

En este trabajo el foco es la creación de módulos que permitan extender las funcionalidades de la herramienta MOTIONS, por lo tanto, no se realizan estudios para determinar el rendimiento de ésta, ni tampoco se realizan validaciones que permitan evidenciar la experiencia de los usuarios al utilizar la herramienta en experimentaciones, esto, a pesar de que las pruebas de cumplimiento de funcionalidades evidencian ciertos aspectos de usabilidad, ya que, por su propósito, dichas pruebas no son suficientes para realizar conclusiones acerca de la experiencia del usuario o

investigador que hace uso de MOTIONS y por lo tanto, se requiere un estudio aparte únicamente enfocado en este aspecto.

6.4 TRABAJO FUTURO

Este trabajo presenta un módulo de control de niveles de inmersión visual y auditiva sobre entornos creados manualmente, por lo tanto, al no ser el objetivo de este trabajo, la cantidad de escenas generadas es escasa y solo sirve para probar el funcionamiento del módulo. Se podría generar un sistema que cree ambientes de experimentación inmersivos de forma procedural, para así ampliar la cantidad de potenciales estudios. Si se desea escenas con propósito específico, la generación de ambientes de forma procedural permitiría, al menos, crear una base sobre la cual iterar de forma manual.

En cuanto al módulo de recepción de señales fisiológicas, se da un caso particular con la medición de ECG utilizando *BITalino*, ya que se ofrece la opción de gatillar acciones dentro de la simulación a través de umbral o rango, sin embargo, cuando se mide ECG, los cambios fisiológicos experimentados por el participante se ven reflejados en la frecuencia de la señal, más que en amplitud, por lo tanto, sería interesante agregar soporte para poder utilizar la frecuencia de la señal como detonante de una acción. Además, considerando que el módulo de recepción de señales fisiológicas es modular y extensible, es posible agregar nuevos dispositivos de recepción de señales fisiológicas, como, por ejemplo, *BioRadio*, que permite la recepción de señales emitidas por la respiración, además de poseer la característica de ser extensible en cuanto a sus receptores.

Queda planteado el desafío de obtener evidencia que asegure que cada nivel de inmersión desarrollado otorga mayor inmersión que el nivel previo. Si bien esto queda demostrado si es que la inmersión se entiende como cantidad y calidad de recursos inmersivos, no existen datos duros que prueben que esto se cumple para la inmersión como concepto general.

6.5 OBSERVACIONES FINALES

La herramienta MOTIONS, como se muestra a lo largo de este proyecto, es un trabajo conjunto de distintos memoristas. A modo de apreciación personal, esto posee ventajas y desventajas. Las ventajas son que, permite lograr objetivos más ambiciosos, y exige un trabajo ordenado y coordinado, donde las piezas de software deben ser modulares y la documentación clara. La desventaja es que, en ocasiones, a pesar de lo anterior, resulta necesario contactar a la persona que desarrolló el módulo base sobre el cual se está trabajando, lo que puede significar una

demora en el trabajo personal, además, se debe invertir una cantidad de tiempo considerable en lograr asimilar el trabajo previo y su arquitectura, de forma que las nuevas funcionalidades se adapten sin interferir negativamente en la herramienta original.

GLOSARIO

- 3D Engine: herramienta utilizada por diseñadores de videojuegos para desarrollar código y plantear un proyecto rápida y fácilmente, sin necesidad de comenzar desde cero.
- API: interfaz de programación de aplicaciones, en otras palabras, otorga recursos a través de una capa de abstracción que pueden ser utilizados por algún otro software.
- Escena: Son los contenedores de Game Objects dentro de Unity.
- Game Object: Clase base para todas las entidades de Unity.
- Inmersión: experimentación de presencia en un entorno generado.
- Interacción humano-información: es el proceso de realimentación recíproca entre las partes involucradas, donde la parte de información genera cambios en el cerebro humano, y el humano genera cambios a su vez en la información.
- Objeto de información digital: todo objeto que representa una pieza de información caracterizada debido a que posee atributos que le otorgan la capacidad de caracterizar de otras piezas y que cumple una función específica. Ejemplos: documentos, videos, imágenes y audios.
- Renderizado: proceso por el cual gráficos 3D son convertidos en imágenes 2D con efectos 3D fotorrealistas.

REFERENCIAS BIBLIOGRÁFICAS

- Almond, R. (2011). *Sensory And Emotional Immersion In Art, Technology And Architecture*.
- Alexander A., Narayanan S., Poys N., & Pradeep S. (2017). Immersion and Engagement in a VR Game. Virginia Tech.
- ASALE, R.-. (s. f.). Inmersión. Recuperado 18 de diciembre de 2017, a partir de <http://dle.rae.es/?id=LeYt5SL>
- Bevan, N. (1995). Measuring usability as quality of use. *Software Quality Journal*, 4(2), 115-130.
- BIOPAC Systems. (2017, octubre 26). Product Sheet. Recuperado a partir de <https://www.biopac.com/wp-content/uploads/BioNomadix-Smart-Center.pdf>
- BITalino. (s. f.). Frequently Asked Questions. Recuperado 18 de diciembre de 2017, a partir de <http://bitalino.com/en/support/faq>
- Chavan, S (2016). Augmented Reality vs. Virtual Reality: Differences and Similarities. *IJARCET*, 5(6).
- Dede, C. (2009). Immersive Interfaces for Engagement and Learning. *Science*, 323(5910), 66-69.
- Estay, F. (2017). Plataforma de apoyo a la experimentación en interacción humano-información a través de gestos corporales. Memoria en proceso de finalización para Ingeniería de Ejecución en Computación e Informática, Departamento de Ingeniería Informática, Universidad de Santiago de Chile.
- Federal Agencies Digitalization Guidelines Initiative. (s. f.). Sampling rate (audio). Recuperado 19 de diciembre de 2017, a partir de <http://www.digitizationguidelines.gov/term.php?term=samplingrateaudio>
- Gaete, E. (2016). Vortice: Ambiente de realidad virtual para apoyar la evaluación de interacciones con objetos de información digital. Memoria de Título de Ingeniería Civil en Informática, Departamento de Ingeniería Informática, Universidad de Santiago de Chile.
- González-Ibáñez, R., Varela-Otárola, J. L., & Barrera-Pulgar, C. (2016). Evaluating body-centered interactions in an image search task. In *Proceedings of the 13 - 17 Marzo 2016, ACM Conference on Human Information Interaction and Retrieval*, (pp. 269–272). Carrboro, North Carolina, USA.
- Great Lakes NeuroTechnologies [Great Lakes NeuroTechnologies] (2013, 22 de Agosto). BioRadio Webinar: Basics of Biomedical Data Acquisition [archivo de video]. Disponible en <https://www.youtube.com/watch?v=mKglhrK8V0E&feature=youtu.be>.
- Great Lakes NeuroTechnologies (s. f.). Glossary And FAQ's. Recuperado el 8 de diciembre a partir de 2017, desde <https://glneurotech.com/bioradio/support-page/bioradio-faq/>
- Jang, D. P., Kim, I. Y., Nam, S. W., Wiederhold, B. K., Wiederhold, M. D., & Kim, S. I. (2002). Analysis of Physiological Response to Two Virtual Environments: Driving and Flying Simulation. *CyberPsychology & Behavior*, 5(1), 11-18.
- Marchionini, G. (2008). Human-information interaction research and development, 30(3), 165-174.
- Martin, J. (1991). *Rapid application development*. Macmillian.
- Meneses, S. (2016). OpenGlove SDK: APIs de Alto Nivel Para C#, C++, JAVA y JavaScript. Memoria de Título de Ingeniería de Ejecución en Computación e Informática, Departamento de Ingeniería Informática, Universidad de Santiago de Chile.
- Mc Squared System Design Group. (s. f.). Sound Systems: Mono vs. Stereo. Recuperado 19 de diciembre de 2017, a partir de <http://www.mcsquared.com/mono-stereo.htm>
- Microsoft. (s. f.). What Is a Render Target? Recuperado 19 de diciembre de 2017, a partir de <https://msdn.microsoft.com/en-us/library/ff604997.aspx>
- Monsalve, R. (2015). Dispositivo de retroalimentación táctil para interfaces naturales y de realidad virtual. Memoria de Título de Ingeniería Civil en Informática, Departamento de Ingeniería Informática, Universidad de Santiago de Chile.
- Murray, C. D., & Sixsmith, J. (1999). The Corporeal Body in Virtual Reality. *Ethos*, 27(3), 315-343.
- Murray, C. D., Bowers, J. M., West, A. J., Pettifer, S., & Gibson, S. (2000). Navigation, Wayfinding, and Place Experience within a Virtual City. *Presence: Teleoperators and Virtual Environments*, 9(5), 435-447.

-
- Owens, B. (2013, octubre 28). Forward Rendering vs. Deferred Rendering. Recuperado 19 de diciembre de 2017, a partir de <https://gamedevelopment.tutsplus.com/articles/forward-rendering-vs-deferred-rendering--gamedev-12342>
- Pollmann, A. (2017). MOTIONS: Sistema de apoyo a la investigación de la interacción humano-Información a través de múltiples interfaces de usuario. Memoria en proceso de finalización para Ingeniería de Ejecución en Computación e Informática, Departamento de Ingeniería Informática, Universidad de Santiago de Chile.
- Steuer, J. (1992). Defining Virtual Reality: Dimensions Determining Telepresence. *Journal of Communication*, 42(4), 73-93.
- THE VOID - Step Beyond Reality. (s. f.). Recuperado 18 de diciembre de 2017, a partir de <https://www.thevoid.com/>
- Virtual Reality Society (2016, enero 10). History Of Virtual Reality. Recuperado 8 de mayo de 2017, a partir de <https://www.vrs.org.uk/virtual-reality/history.html>
- Unity3d. (2016). Manual: Antialiasing. Recuperado 19 de diciembre de 2017, a partir de <https://docs.unity3d.com/es/current/Manual/script-Antialiasing.html>
- Unity3d. (2017a, marzo). Scripting API: Material. Recuperado 19 de diciembre de 2017, a partir de <https://docs.unity3d.com/ScriptReference/Material.html>
- Unity3d. (2017b, marzo). Scripting API: Texture.anisoLevel. Recuperado 19 de diciembre de 2017, a partir de <https://docs.unity3d.com/ScriptReference/Texture-anisoLevel.html>

ANEXO A. ARQUITECTURA DE MOTIONS

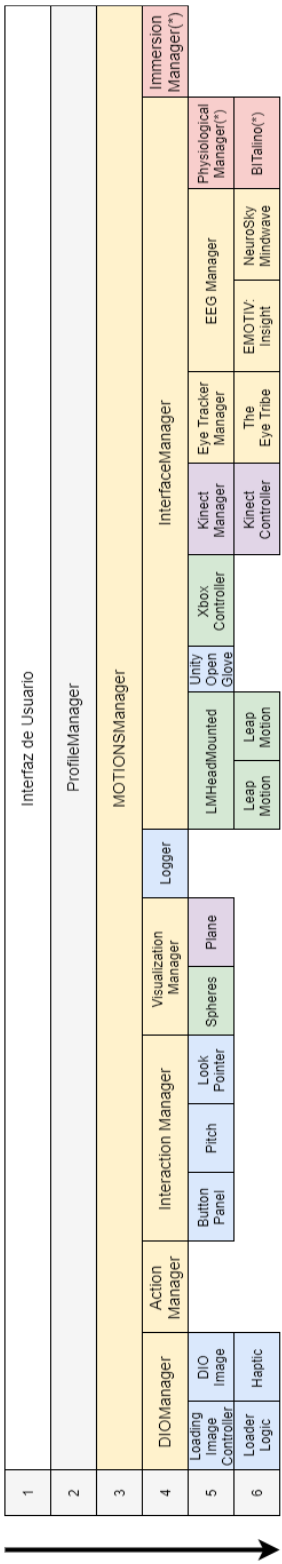


Figura A.1: Diagrama de Arquitectura de MOTIONS completo.
Fuente: Extendido de Pollmann (2017).

ANEXO B. CASO DE PRUEBA

Tabla B.1: Caso de prueba CP_V_IMM_08, parte 1.
Fuente: Elaboración propia, 2017.

Identificador	CP_V_IMM_08
Nombre del caso	Evaluación del séptimo nivel de inmersión visual en visualización planar.
Objetivos	Evaluar el correcto funcionamiento del séptimo nivel de inmersión visual en una simulación de MOTIONS utilizando la visualización planar.
Precondiciones	1. Ejecutar MOTIONS 2. Ingresar en uno de los perfiles disponibles. 3. Presionar el botón <i>Hardware</i> 4. Seleccionar <i>Mouse y Keyboard</i> 5. Presionar el botón <i>Return</i>. 6. Presionar el botón <i>Information Object</i>. 7. Presionar el botón <i>Image and Path names</i>. 8. Presionar el botón <i>Change Folder Path</i> y elegir la carpeta donde se encuentran las imágenes. 9. Escribir "<i>g_imagen_</i>" en el campo "<i>Images name prefix</i>" y "15" en el campo "<i>Image Amount</i>". 10. Presionar el botón <i>Apply Changes</i>. 11. Presionar el botón "<i>Return</i>" 12. Presionar el botón "<i>Visualization</i>" 13. Seleccionar "<i>Sphere</i>" en el dropdown "<i>Select visualization to preview</i>"
Paso	Resultado esperado
a. Seleccionar visualización planar	- El video que muestra un ejemplo de la visualización debería cambiar, para mostrar un ejemplo de la visualización planar en lugar de la esférica.
b. Presionar el botón " <i>Return</i> "	- El menú de configuración vuelve su estado inicial.
c. Presionar el botón " <i>Immersion</i> "	- El menú de configuración muestra el panel de inmersión.
d. Mover el <i>slider</i> " <i>Visual Immersion</i> " hasta la el extremo derecho (máximo nivel de inmersión visual).	- El panel de la derecha, donde aparecen todas las configuraciones variables para inmersión visual se actualiza de acuerdo a los valores predefinidos del máximo nivel de inmersión visual.
e. Presionar el botón " <i>Return</i> "	- El menú de configuración vuelve su estado inicial.
f. Presionar el botón " <i>Start Evaluation</i> "	- Se muestra una vista que indica el progreso de la carga de imágenes en la simulación. - A continuación, aparece la escena de la simulación, donde aparece un máximo de 12 imágenes por plano. - La escena presenta una estructura que sostiene al participante, un <i>skybox</i> que extiende la percepción del entorno, objetos relacionados con la escena, reflejos en materiales reflectivos y sombreado y texturas en alta definición.

Tabla B.2: Caso de prueba CP_V_IMM_08, parte 2.
Fuente: Elaboración propia, 2017.

Paso	Resultado esperado
g. Seleccionar una imagen	- La imagen seleccionada cambia su coloración.
h. Clasificar una imagen	- La imagen seleccionada pasa a estar contenida en la clasificación seleccionada.

ANEXO C. QUALITY SETTINGS

Tabla C.1: Configuraciones disponibles en el menú QualitySettings.

Fuente: Documentación de Unity3D, 2017.²⁵

Propiedad	Función
<i>Pixel Light Count</i>	El número máximo de píxeles cuando se usa <i>Forward Rendering</i> .
<i>Texture Quality</i>	Permite escoger si utilizar texturas en máxima resolución, o una fracción de ésta. Posee cuatro niveles: <i>Full Res</i> , <i>Half Res</i> , <i>Quarter Res</i> y <i>Eighth Res</i> .
<i>Anisotropic Textures</i>	Permite elegir como serán utilizadas las texturas anisotrópicas. Pueden usarse: <i>Disabled</i> , <i>Per Texture</i> y <i>Forced On</i> .
<i>AntiAliasing</i>	Establece el nivel de <i>antialiasing</i> a utilizar. Puede dejarse en 2x, 4x y 8x multi-muestreo
<i>Soft Particles</i>	Permite elegir si debe o no usarse <i>soft blending</i> para las partículas.
<i>RealTime Reflection Probes</i>	Permite elegir si debe o no actualizarse las <i>Reflection Probes</i> en tiempo de ejecución.
<i>Shadows</i>	Determinan que tipo de sombras se usarán. Posee tres opciones: <i>Hard and Soft Shadows</i> , <i>Hard Shadows only</i> y <i>Disable Shadows</i> .
<i>Shadow Resolution</i>	Determina la resolución de las sombras. Ofrece los niveles: <i>Low</i> , <i>Medium</i> , <i>High</i> y <i>Very High</i> .
<i>Shadow Projection</i>	Determina como se muestran las sombras producidas por luces direccionales. Posee dos opciones, <i>Close Fit</i> y <i>Stable Fit</i> .
<i>Shadow Cascades</i>	Determina el número de cascadas de sombras. Puede variar entre cero y cuatro.
<i>Shadow Distance</i>	Determina hasta que distancia son visibles las sombras.
<i>ShadowMask Mode</i>	Establece el comportamiento del <i>ShadowMask</i> .
<i>Blend Weights</i>	Determina el número de huesos que puede afectar a un vértice en una animación.
<i>VSync Count</i>	Determina cuantas veces se sincroniza el proceso de renderizado con la frecuencia en que se refresca el monitor.
<i>Particle Raycast Budget</i>	Determina el máximo número de <i>raycasts</i> a usar.

²⁵ <https://docs.unity3d.com/Manual/class-QualitySettings.html>